book 15

# game systems and architecture

an engineering handbook

Raeliosol

version 3.0, compiled 10 2026

# Contents

1. the game loop and time ..... 5
   1.1. variable or fixed ..... 5
   1.2. the accumulator ..... 5
   1.3. the framework runs this for you ..... 7
2. ecs theory ..... 8
   2.1. entities are handles with generations ..... 8
   2.2. the reference implementation, and its bill ..... 9
   2.3. the two storage religions ..... 10
3. an ecs implementation in c# ..... 12
   3.1. the store ..... 13
   3.2. systems as loops ..... 15
4. data oriented design ..... 17
   4.1. the two shapes ..... 17
   4.2. what the soa shape buys beyond speed ..... 18
   4.3. alignment and padding ..... 19
5. finite state machines ..... 21
   5.1. transitions as table rows ..... 21
6. behavior trees and utility scoring ..... 23
   6.1. the grammar ..... 23
   6.2. the ai in miniature ..... 25
   6.3. utility scoring ..... 26
7. deterministic math ..... 28
   7.1. Q31.32 in one struct ..... 28
   7.2. trig without a math library ..... 29
   7.3. what determinism costs ..... 30
8. physics ..... 32
   8.1. integration, velocity first ..... 32
   8.2. the heightfield and its questions ..... 33
   8.3. collision without tunneling ..... 34
9. terrain: heightmaps ..... 36
   9.1. value noise from a hash ..... 36
   9.2. octaves ..... 37
10. turn systems and game flow ..... 38
   10.1. initiative order ..... 38
11. randomness and determinism discipline ..... 41
   11.1. splitmix64 ..... 41
   11.2. unbiased integers ..... 42
   11.3. streams ..... 42
12. serialization and save systems ..... 44
   12.1. the format is the contract ..... 44
   12.2. the capstone save file ..... 45
13. testing games ..... 47
   13.1. the state hash ..... 47
   13.2. determinism as an assertion ..... 47
14. characters and progression ..... 49
   14.1. four primaries, four elements ..... 49
   14.2. growth as arithmetic ..... 50
   14.3. effective stats as one pure function ..... 52

14.4. the save file ..... 54
15. items, weapons, and crafting ..... 56
15.1. the weapon as data ..... 56
15.2. damage falls off linearly ..... 57
15.3. the terrain remembers the weapon type ..... 58
15.4. item stats and where each raw number may come from ..... 60
15.5. upgrades compound, costs double ..... 62
15.6. crafting as consume plus roll ..... 63
16. pets and companions ..... 65
16.1. the species as data ..... 66
16.2. growth mirrors characters ..... 67
16.3. hatching from a named stream ..... 68
16.4. mana gates the actives ..... 68
17. expeditions and bosses ..... 69
17.1. the stage as data ..... 69
17.2. enemy rosters and the two brains ..... 70
17.3. weighted loot from one stream ..... 71
17.4. the unlock chain ..... 72
18. embedded scripting: hosting lua ..... 73
18.1. why games embed a script runtime ..... 73
18.2. the host in c# ..... 74
18.3. the sandbox ..... 75
18.4. events in, decisions out ..... 76
18.5. warcraft iii reforged: lua in the world editor ..... 77
18.6. dota 2: vscript in source 2 ..... 78
18.7. our host against theirs ..... 78
18.8. the capstone wires it in ..... 79
19. capstone part 1: the battle engine ..... 85
19.1. components and the machine ..... 85
19.2. fire, then the tick owns the flight ..... 86
19.3. the weapon table ..... 88
19.4. the hybrid ai ..... 90
19.5. the match is still a pure function ..... 92
20. capstone part 2: the campaign core ..... 93
20.1. the composition root ..... 94
20.2. a stage becomes a battle ..... 95
20.3. the payout ..... 97
20.4. replay v3 ..... 99
21. capstone part 3: the playable mvp ..... 101
21.1. kni, taught before used ..... 101
21.2. the runtime font ..... 102
21.3. the screens and the input ..... 104
21.4. htmx, taught before used ..... 106
21.5. the thin companion ..... 107
21.6. the golden run ..... 108
22. capstone part 4: balance by simulation ..... 116
22.1. the matrix ..... 116
22.2. the tuning loop ..... 117
22.3. the bands and the report ..... 118
23. appendices: topic matrix and sources ..... 120
23.1. topic coverage ..... 120

23.2. pinned sources ..... 123

# 1. the game loop and time

Every real time game is one shape: a loop that reads input, advances a simulated world, draws that world, and presents the frame, over and over until the player quits. A server is a request loop, a gui toolkit is an event loop, a game is a loop that must finish all three passes in under sixteen milliseconds. That budget is the whole discipline of this book, and the loop itself is where its most consequential decision lives: what a step of time means inside `Update`.

## 1.1. variable or fixed

The naive loop measures the real time since the last frame and hands it to the simulation as the delta. Rendering at whatever rate the machine manages looks smooth, and for menus and turn counters it is fine. The moment anything integrates, a projectile's arc, a cooldown, a collision sweep, the variable delta leaks into the results: Euler integration over different sized steps lands in different places, so the same shot fired twice behaves differently, a replay recorded on a fast machine diverges from playback on a slow one, and a bug report becomes unreproducible.

The fixed timestep loop refuses the leak. The simulation always steps by the same constant dt, sixty steps a second by convention, and real time is only used to decide when to run the next step. The same input sequence produces the same world state on every machine, which is the property the capstone's replays and the testing chapter stand on. The price is that simulation time and render time come apart, a 60 hertz simulation shown on a 50 hertz display needs fractional steps, and that reconciliation is a real algorithm, not a rounding trick.

Integrated as trajectories the leak is two arcs where there should be one, and the fixed grid is one arc no matter which machine draws it:

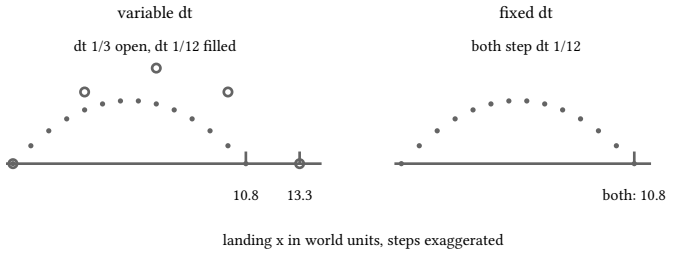



figure 1: the same shot under two step sizes lands apart, the fixed grid lands on one point

## 1.2. the accumulator

The standard machine, old enough that its canonical description is a 2004 article, is an accumulator. Each frame, add the measured real delta to a pot. While the pot holds at least one dt, run one simulation step and subtract dt. Whatever remains is carried to the next frame, and it is also the interpolation alpha, the fraction of the way the world has moved toward its next step. The renderer lerps the previous and current states by that fraction, a linear blend, and the visible result is smooth even though the simulation ticks on a fixed grid.

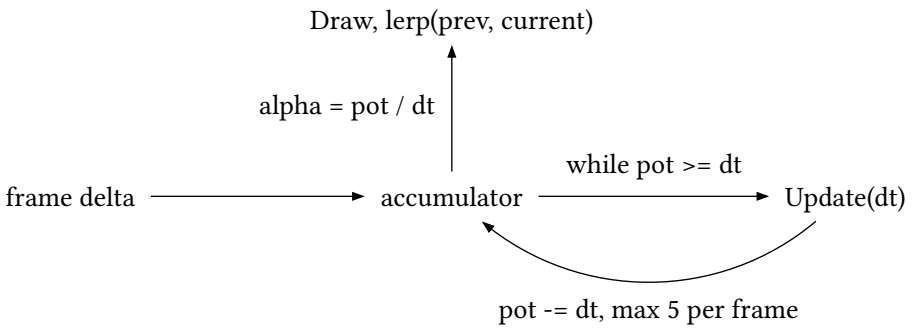



figure 2: the accumulator in one picture

```
/// <summary>
/// The accumulator at the heart of a fixed timestep loop. Feed it the real
/// frame delta, it answers how many simulation steps to run and how far
/// into the next step the render sits, the interpolation alpha.
/// </summary>
public sealed class FixedStepClock(double stepsPerSecond, int maxStepsPerFrame = 5)
{
    private TimeSpan _accumulator;

    public TimeSpan StepInterval { get; } = TimeSpan.FromSeconds(1.0 / stepsPerSecond);
    public int MaxStepsPerFrame { get; } = maxStepsPerFrame;
    public long StepsConsumed { get; private set; }

    /// <summary>Fraction [0,1) of the way from the last consumed step to the next.</summary>
    public double Alpha { get; private set; }

    public int Feed(TimeSpan realDelta)
    {
        _accumulator += realDelta;
        int steps = 0;
        while (steps < MaxStepsPerFrame && _accumulator ≥ StepInterval)
        {
            _accumulator -= StepInterval;
            steps++;
        }
        if (_accumulator ≥ StepInterval)
        {
            // the spiral of death guard: the backlog is bigger than the frame
            // budget can ever pay down, so it is dropped and the clock keeps
            // ticking from now
            _accumulator = TimeSpan.FromTicks(StepInterval.Ticks - 1);
        }
        StepsConsumed += steps;
        Alpha = (double)_accumulator.Ticks / StepInterval.Ticks;
        return steps;
    }
```

listing 1: fixed step clock: the accumulator, the step budget, the spiral guard (source: game-systems/samples/src/Ch01/Loop.cs, lines 3-38)

`Feed` is the entire loop minus the loop. The test suite drives it with scripted frame deltas instead of a wall clock, so the exact behavior is pinned without opening a window: a feed of exactly one interval consumes one step with alpha zero, and a full second of jittery delivery, lumps of two steps alternating with empty frames, still totals exactly sixty steps. The accumulator absorbs the jitter, it does not average it into the physics.

The `MaxStepsPerFrame` budget is the spiral of death guard. When a loading hitch or a debugger pause delivers a ten second delta, honest payment would mean six hundred catch up steps, each of which takes real time, which puts the frame further behind, which buys more backlog. The clamp runs five steps and drops the rest, the game hiccups once and continues, and the test pins the exact post clamp state: five steps consumed, alpha just under one, the backlog gone.

```
public static class Interp
{
    /// <summary>Blend two simulation states for rendering: alpha 0 is the previous step, 1 the current.</
summary>
    public static double Lerp(double from, double to, double alpha) => from + (to - from) * alpha;
}
```

listing 2: render side interpolation by the leftover alpha (source: game-systems/samples/src/Ch01/Loop.cs, lines 41-45)

`Lerp` is the render side of the contract. The simulation only ever produces states on the step grid, the renderer asks for the blend between the previous and current state at the alpha the clock reports, and the 50 hertz display test shows the pattern such a loop actually executes: frames of one, one, one, one, then two steps, six updates per five renders, forever.

## 1.3. the framework runs this for you

KNI, the framework the capstone renders through, inherits the XNA `Game` contract: a default fixed time step at sixty hertz (`IsFixedTimeStep` true, `TargetElapsedTime` one sixtieth), `Update` called zero or more times per frame as the accumulator drains, `Draw` called exactly once. That is the same machine as `FixedStepClock`, wired to the platform's timer and vsync instead of a test's delta stream. Building it by hand first is not redundant ceremony, it is the difference between knowing the framework catches up after a stall and trusting it.

Stacked side by side the two clocks are one machine with two drivers:

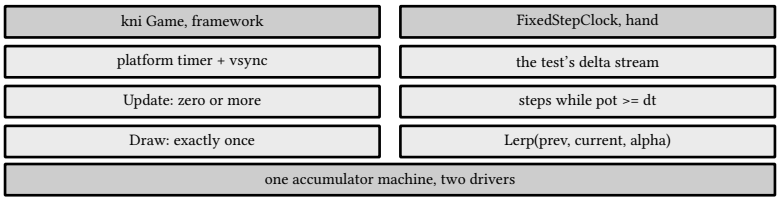



figure 3: the kni game contract stacked against the hand built clock

**WARNING: the alpha contract cuts both ways**

Interpolated rendering means `Draw` may not assume the world is at a step boundary. Anything derived from raw simulation state, camera follows, muzzle flashes, hit sparks, must blend or it will jitter exactly when the step and frame rates disagree. The capstone keeps its render state in previous/current pairs for this reason.

sources: gafferongames.com "fix your timestep" for the accumulator and interpolation alpha, docs.monogame.net game loop article for the xna `IsFixedTimeStep` and `TargetElapsedTime` contract KNI inherits, github.com/kniEngine/kni release v4.3.9001 verified 2026-09-08 by compiling the `Game` surface on net10.0. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 6 tests in `GameSystems.Samples.Tests.Ch01`.

# 2. ecs theory

The object oriented game engine starts with a class hierarchy: `Entity` at the root, `Actor` and `Prop` below it, `Tank : Actor`, `ExplosiveBarrel : Prop`, and it works until the designer asks for a barrel that is also a tank turret. Behavior in a real game is not a tree, it is a cross product of concerns, renderable, mobile, damageable, flammable, ai steered, and single inheritance forces every combination to be a class someone declared in advance. The usual escape is a god `Entity` with every field and an `IsActive` flag per feature, which moves the problem from design time to every cache line in the update loop.

Entity component system inverts the factoring. Three words, three definitions, no inheritance anywhere:

An **entity** is a name, usually one integer, that identifies a thing. It has no fields and no methods. It is not a reference to an object, it is a handle into storage someone else owns.

A **component** is plain data, a `Position`, a `Health`, a `Velocity`, with no behavior attached. An entity owns zero or more of them and gains and loses them at runtime, which is the composition that inheritance could not buy: a flammable tank is a tank entity that also carries an `Ignitable` component, declared by no one in advance.

A **system** is code that runs over every entity owning a particular set of components. A movement system asks for `Position` plus `Velocity` and cares about nothing else, so it processes tanks, barrels, and bullets through one loop with no virtual dispatch.

The FAQ that consolidated this vocabulary states the trade plainly: inheritance is a first class citizen in object oriented design, composition is a first class citizen in ecs. The capstone's artillery game is built this way, a shell is an entity with position, velocity, and a trail component, and nothing in the code knows what a shell "class" is.

## 2.1. entities are handles with generations

```
/// <summary>
/// An entity is not an object, it is a handle: a slot index plus a
/// generation that together name one live row of component storage.
/// </summary>
public readonly record struct Entity(int Index, int Gen);
```

listing 3: an entity is a slot index plus a generation (source: game-systems/samples/src/Ch02/NaiveEcs.cs, lines 3-7)

Because an entity is just a number, every ecs owes an answer to the stale handle question. A system caches an entity id, the entity dies, the slot is recycled for a new entity, and the cached id now names an innocent bystander. The standard defense is a generation counter: the handle is `(index, generation)`, destroying an entity bumps the slot's generation, and a handle from an older generation is refused instead of resolved. The test pins the whole lifecycle, same slot reused, new generation, the old handle rejected on use.

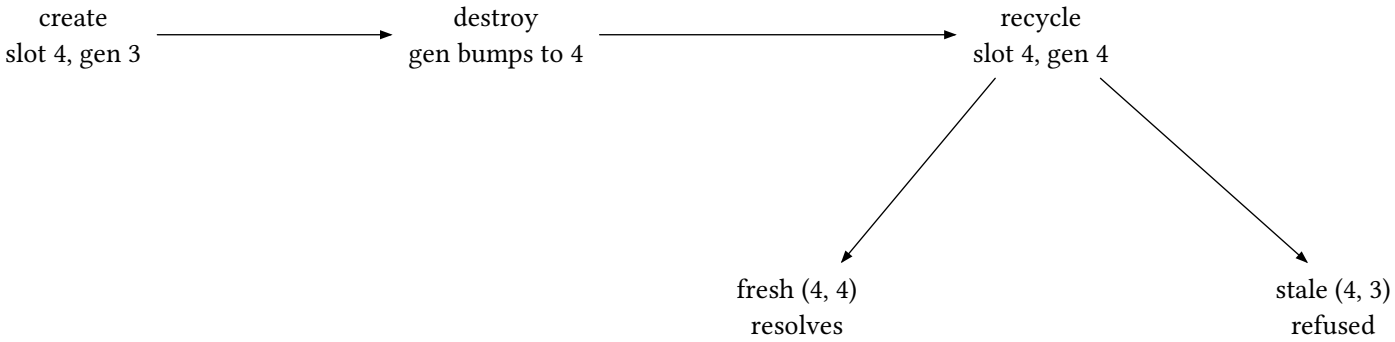



figure 4: one slot's life, and the stale handle refused on use

## 2.2. the reference implementation, and its bill

```
    public Entity Create()
    {
        if (_free.Count > 0)
        {
            int i = _free.Dequeue();
            Row row = _rows[i];
            row.Alive = true;
            row.Components.Clear();
            Count++;
            return new Entity(i, row.Gen);
        }
        _rows.Add(new Row { Alive = true });
        Count++;
        return new Entity(_rows.Count - 1, 0);
    }

    public void Destroy(Entity e)
    {
        Row row = RowOf(e);
        row.Alive = false;
        row.Gen++;
        row.Components.Clear();
        _free.Enqueue(e.Index);
        Count--;
    }

    public bool IsAlive(Entity e) =>
        e.Index < _rows.Count && _rows[e.Index].Alive && _rows[e.Index].Gen == e.Gen;
```

listing 4: slot recycling behind create and destroy (source: game-systems/samples/src/Ch02/NaiveEcs.cs, lines 28-55)

```
    public void Add<T>(Entity e, T component)
    {
        if (component is null)
            throw new ArgumentNullException(nameof(component));
        var bag = RowOf(e).Components;
        if (bag.ContainsKey(typeof(T)))
            throw new InvalidOperationException($"entity {e} already has a {typeof(T).Name}");
        bag.Add(typeof(T), component);
    }

    public T Get<T>(Entity e)
    {
        var bag = RowOf(e).Components;
        if (!bag.TryGetValue(typeof(T), out object? value))
            throw new KeyNotFoundException($"entity {e} has no {typeof(T).Name}");
        return (T)value;
    }

    public bool Has<T>(Entity e) => RowOf(e).Components.ContainsKey(typeof(T));
```

listing 5: component bags: correct, general, boxed (source: game-systems/samples/src/Ch02/NaiveEcs.cs, lines 57-76)

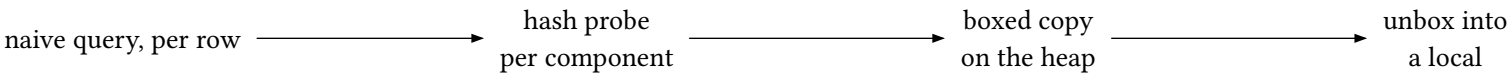
This is the first ecs nearly everyone writes, a dictionary of typed bags per entity, and it is worth owning deliberately before replacing it, because its semantics are the spec chapter 3 must meet: duplicate component types rejected, missing components throw, destroy clears the bags and recycles the slot. The bill arrives in the memory layout.

Every `Get<T>` is a dictionary probe, then a cast out of `object`, which boxes every struct component on `Add` and allocates a copy on every read. A query walks all rows and probes one hash table per component per row:

```
public IEnumerable<Entity> Query(Type first, params Type[] rest)
{
    for (int i = 0; i < _rows.Count; i++)
    {
        Row row = _rows[i];
        if (!row.Alive)
            continue;
        if (!row.Components.ContainsKey(first))
            continue;
        bool all = true;
        foreach (Type t in rest)
        {
            if (!row.Components.ContainsKey(t))
            {
                all = false;
                break;
            }
        }
        if (all)
            yield return new Entity(i, row.Gen);
    }
```

listing 6: a query as a full table scan with hash probes (source: game-systems/samples/src/Ch02/NaiveEcs.cs, lines 78-99)

For a hundred entities this is nothing. For the million entities the dod chapter measures, it is the difference between data in cache lines and data behind pointers, and it is why real storage engines pick a layout and defend it.



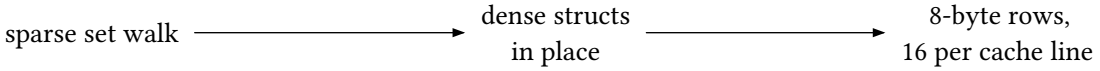




figure 5: one query row: dictionary probes and boxed copies against the contiguous walk

## 2.3. the two storage religions

Everything else in ecs design is a consequence of one question: where does component data live.

**Archetype storage** groups entities that own exactly the same component set into one contiguous block, Unity's entities package calls them archetypes stored in 16 kb chunks, one array per component type plus the entity ids. Iteration over a component set is a linear walk through memory at full bandwidth. The cost is structural churn: adding a component moves the entity's every component to a different block, so composition changes are the expensive operation.

**Sparse set storage** gives each component type its own dense array plus a sparse index, entity id to dense slot. Any component set intersection iterates the shortest array and probes the others. Component add and remove are swaps within one type's storage, cheap and isolated, at the price of iteration touching one array per component in the set instead of one block.

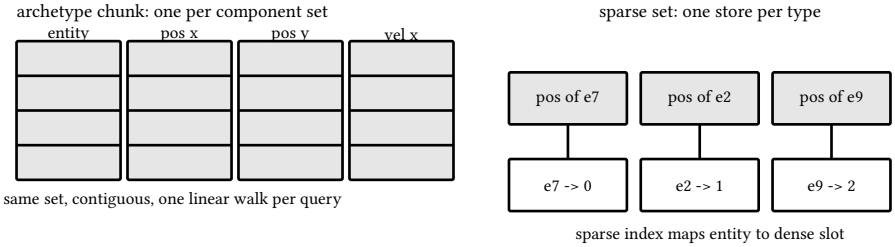



figure 6: the two storage religions, archetype chunk versus sparse set

Neither wins. Archetypes favor simulations where component sets are stable and queries are wide, sparse sets favor games where entities gain and shed components constantly, projectiles spawning and dying. Chapter 3 builds sparse sets, because the artillery capstone's load is spawn and death heavy, and the machinery is small enough to hold in your head. The honest rule is to know which religion your storage follows and write systems that respect its cheap path.

**NOTE: tags are components too**

A component with no fields, an `Enemy` marker or a `Frozen` marker, is a tag. It carries no data, but it participates in queries exactly like any component, and it is often the cheapest way to route an entity through a system without adding a field anyone has to maintain. The capstone tags wind-affected embers so one marker routes them through the wind system while plain debris falls straight.

sources: github.com/SanderMertens/ecs-faq for the entity, component, and system definitions and the composition over inheritance framing, docs.unity3d.com entities 1.0 archetypes and chunks concepts for the archetype storage model, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 7 tests in `GameSystems.Samples.Tests.Ch02`.

# 3. an ecs implementation in c#

Chapter 2 defined the contract and charged it to a dictionary of dictionaries. This chapter pays the same contract from sparse sets, and the payment plan is the chapter: one dense array per component type, one sparse index from entity to slot, one bitmask per entity, and system loops that touch nothing but arrays. The entity handle from chapter 2 is kept as is, only the storage under it is replaced, which is the ecs promise working once already, nothing above the world's api had to change shape.

```
/// <summary>System body over one component, ref access, no boxing.</summary>
public delegate void RefAction<T1>(Entity e, ref T1 a) where T1 : struct;

/// <summary>System body over two components, ref access, no boxing.</summary>
public delegate void RefAction<T1, T2>(Entity e, ref T1 a, ref T2 b)
    where T1 : struct where T2 : struct;

/// <summary>
/// Sparse set ecs: one store per component type, a dense component array
/// plus a sparse entity-to-slot index, and a bitmask per entity so queries
/// never touch a hash table.
/// </summary>
public sealed class World(int componentTypeCapacity = 64)
{
    private int[] _gens = new int[16];
    private bool[] _alive = new bool[16];
    private ulong[] _masks = new ulong[16];
    private int _slots; // live plus free, the high water mark
    private readonly Queue<int> _free = new();
    private readonly Dictionary<Type, int> _typeIds = new();
    private readonly object?[] _storeByTypeId = new object?[componentTypeCapacity];
    private int _componentTypes;
    private int _iterating;
```

listing 7: system delegates with ref access, the world's three arrays (source: game-systems/samples/src/Ch03/World.cs, lines 5-27)

The three parallel arrays at the top are the whole entity ledger: `_gens` for stale handle rejection, `_alive` for liveness, `_masks` for queries. A mask has one bit per registered component type, so a world answers "does this entity own a T" with one AND, and the honest limit is stated in the constructor, sixty four component types per `ulong`, more requires a wider mask, and the budget is enforced with an exception rather than silently wrapping.

## 3.1. the store

```
    // the store is nested private, so its public members are world only
    private sealed class Store<T>(int initialEntities) : IStore where T : struct
    {
        private T[] _dense = new T[8];
        private int[] _denseEntities = new int[8];
        private int[] _sparse = new int[Math.Max(initialEntities, 8)];
        private int _count;

        public int Population => _count;

        public int EntityAt(int slot) => _denseEntities[slot];

        public ref T At(int slot) => ref _dense[slot];

        public void Add(int entityIndex, T component, ulong bit, ulong[] masks)
        {
            if (Contains(entityIndex))
                throw new InvalidOperationException(
                    $"entity already has a {typeof(T).Name}");
            if (_sparse.Length ≤ entityIndex)
            {
                int size = _sparse.Length;
                while (size ≤ entityIndex)
                    size *= 2;
                Array.Resize(ref _sparse, size);
            }
            if (_count == _dense.Length)
            {
                Array.Resize(ref _dense, _dense.Length * 2);
                Array.Resize(ref _denseEntities, _denseEntities.Length * 2);
            }
            _dense[_count] = component;
            _denseEntities[_count] = entityIndex;
            _sparse[entityIndex] = _count;
            _count++;
            masks[entityIndex] |= bit;
        }
```

listing 8: dense components, dense entity ids, sparse index, append on add (source: game-systems/samples/src/Ch03/World.cs, lines 187-223)

`Add` appends to both dense arrays and links the sparse entry, so components of one type sit contiguous in memory, and the entity ids sit beside them so iteration can name what it is visiting. There is no boxing anywhere: `T` is constrained to `struct`, and `Get` returns `ref T` into the dense array, which is why a system can write `world.Get<Health>(e).Points -= 30` and the store sees it. Chapter 2's version returned a boxed copy per read, the kind of difference that is invisible in a seven entity test and that chapter 4 measures at one million entities, where layout alone costs a factor of 2.1.

```
        public bool Contains(int entityIndex) =>
            entityIndex < _sparse.Length && _sparse[entityIndex] < _count
            && _denseEntities[_sparse[entityIndex]] == entityIndex;

        public ref T GetRef(int entityIndex) => ref _dense[_sparse[entityIndex]];

        public bool Remove(int entityIndex, ulong bit, ulong[] masks)
        {
            if (!Contains(entityIndex))
                return false;
            SwapRemove(entityIndex);
            masks[entityIndex] &= ~bit;
            return true;
        }

        public void OnDestroy(int entityIndex)
        {
            if (Contains(entityIndex))
                SwapRemove(entityIndex);
        }

        private void SwapRemove(int entityIndex)
        {
            int slot = _sparse[entityIndex];
            int last = _count - 1;
            _dense[slot] = _dense[last];
            _denseEntities[slot] = _denseEntities[last];
            _sparse[_denseEntities[slot]] = slot;
            _count--;
        }
    }
}
```

listing 9: contains by identity, swap remove, destroy cleanup (source: game-systems/samples/src/Ch03/World.cs, lines 225-256)

Removal is the sparse set's signature move: the last dense row swaps into the victim's slot, one write each to the component array, the id array, and the sparse index of the moved entity. No shifting, nothing before the hole is touched, order is not preserved and the tests assert nothing about order for exactly this reason. Destroy walks the stores and removes the entity's row from each, then recycles the slot with a generation bump, so a stale handle is refused by the ledger before any store ever sees it.

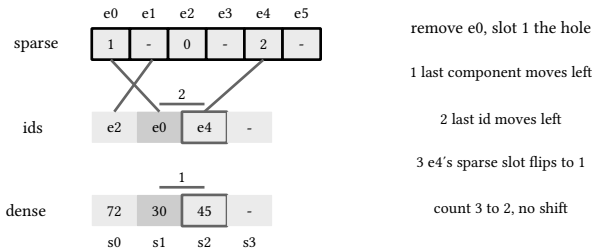



figure 7: the store as three arrays, and a removal paid in three writes

## 3.2. systems as loops

```
    public void ForEach<T1>(RefAction<T1> body) where T1 : struct
    {
        var s1 = StoreFor<T1>();
        _iterating++;
        try
        {
            for (int slot = 0; slot < s1.Population; slot++)
            {
                int i = s1.EntityAt(slot);
                body(new Entity(i, _gens[i]), ref s1.At(slot));
            }
        }
        finally { _iterating--; }
    }

    public void ForEach<T1, T2>(RefAction<T1, T2> body) where T1 : struct where T2 : struct
    {
        var s1 = StoreFor<T1>();
        var s2 = StoreFor<T2>();
        _iterating++;
        try
        {
            // walk whichever store holds fewer rows, probe the other
            if (s1.Population ≤ s2.Population)
            {
                for (int slot = 0; slot < s1.Population; slot++)
                {
                    int i = s1.EntityAt(slot);
                    if (s2.Contains(i))
                        body(new Entity(i, _gens[i]), ref s1.At(slot), ref s2.GetRef(i));
                }
            }
            else
            {
                for (int slot = 0; slot < s2.Population; slot++)
                {
                    int i = s2.EntityAt(slot);
                    if (s1.Contains(i))
                        body(new Entity(i, _gens[i]), ref s1.GetRef(i), ref s2.At(slot));
                }
            }
        }
        finally { _iterating--; }
    }
```

listing 10: two component queries walk the shorter store and probe the other (source: game-systems/samples/src/Ch03/World.cs, lines 98-141)

`ForEach` is the entire system interface. The two component version walks whichever store holds fewer rows and probes the other through its sparse index, so a `Health` by `Frozen` query over a hundred frozen entities in a world of a million healthy ones visits a hundred rows. The tags test rides exactly this shape, `Frozen` carries no data, routes one entity through the system, and costs one bit in a mask.

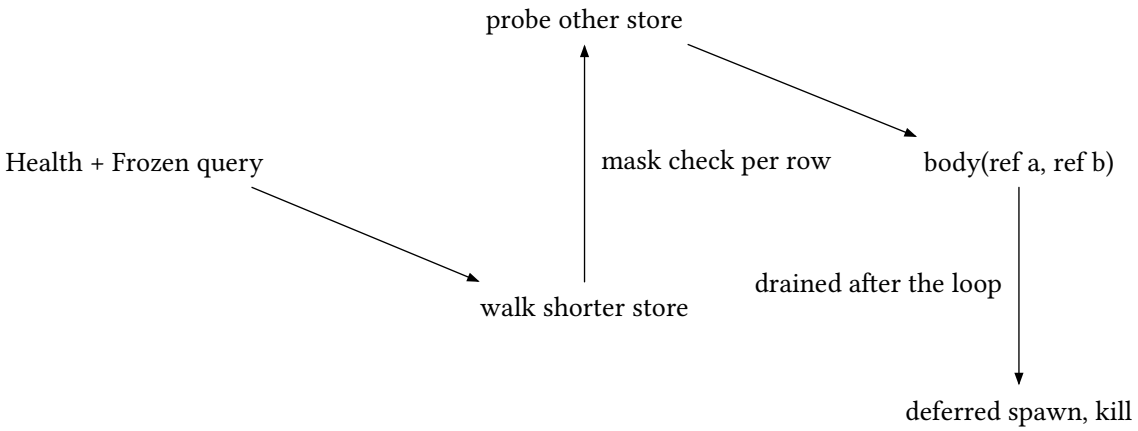



figure 8: a two component query walking the shorter store

The guard around the loops is the discipline sparse sets demand. Iteration holds dense indices into arrays that structural changes rearrange, so adding, removing, or destroying while a system runs is rejected with an exception. The honest pattern is deferred mutation: a system records the entities it wants to spawn or kill, and the frame's outer code applies the list after the loop returns. The capstone's impact resolution works this way, explosions queue entity deaths, particle spawns, and the cluster bomb's children, and the tick drains the queues between systems.

**PITFALL: what this world does not have**

No archetypes, so a wide five component query probes four sparse indexes per row instead of walking one block. No parallel scheduling, systems run in registration order on one thread. No change detection, a system that must react to edits polls or compares. Each omission is deliberate, this world is small enough to verify by reading, and each is a door into the industrial engines when the game outgrows it.

sources: design follows chapter 2's cited ecs-faq storage discussion and the sparse set literature it points to, the unity entities archetype docs remain the contrast case, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 8 tests in `GameSystems.Samples.Tests.Ch03`.

# 4. data oriented design

Chapter 3 chose a memory layout and promised it mattered. This chapter is the receipt. Data oriented design is the practice of organizing memory and code around how the data is actually transformed, in what order, how often, and at what width, instead of around a taxonomy of real world concepts. Its founding observation is arithmetic is nearly free and waiting for memory is not: a register add is a single cycle, while a load that misses every cache and goes to dram costs on the order of two hundred. A loop that touches memory badly does not do less math, it does the same math surrounded by stalls.

## 4.1. the two shapes

```
/// <summary>The array of structs shape: one heap object per entity.</summary>
public class EntityAos
{
    public double X, Y, Dx, Dy;
}

/// <summary>
/// The structure of arrays shape: each field in its own contiguous array.
/// </summary>
public sealed class SoaPositions(double[] x, double[] y, double[] dx, double[] dy)
{
    public double[] X = x, Y = y, Dx = dx, Dy = dy;
    public byte[] Flags = new byte[x.Length];

    public int CountFlagged()
    {
        int count = 0;
        foreach (byte f in Flags)
            count += f;
```

listing 11: array of structs against structure of arrays (source: game-systems/samples/src/Ch04/Dod.cs, lines 5-24)

The object oriented shape puts each entity's whole state behind one reference, so walking a list of entities walks a list of pointers, and every hop is a potential cache miss wherever the allocator put that object. The structure of arrays shape puts each field in its own contiguous array, so reading `X` for a thousand entities reads a thousand doubles in a straight line, sixteen per 128 byte cache line on this machine, and the prefetcher sees the pattern after two lines.

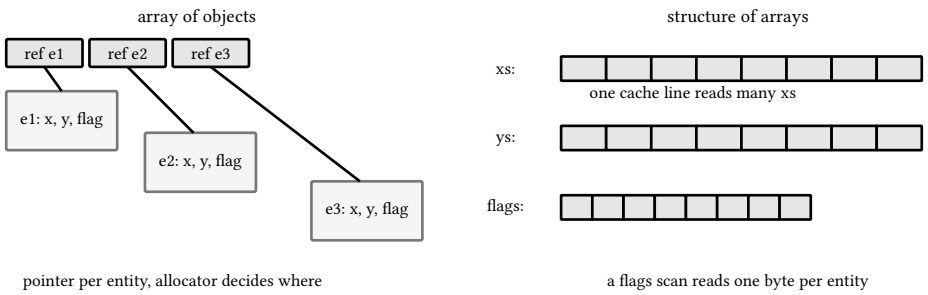



figure 9: the two shapes, array of objects versus structure of arrays

```
{
    public static double IntegrateAos(EntityAos[] entities, double dt)
    {
        double checksum = 0;
        foreach (EntityAos e in entities)
        {
            e.X += e.Dx * dt;
            e.Y += e.Dy * dt;
            checksum += e.X + e.Y;
        }
        return checksum;
    }

    public static double IntegrateSoa(SoaPositions soa, double dt)
    {
        double checksum = 0;
        double[] x = soa.X, y = soa.Y, dx = soa.Dx, dy = soa.Dy;
        for (int i = 0; i < x.Length; i++)
        {
            x[i] += dx[i] * dt;
            y[i] += dy[i] * dt;
            checksum += x[i] + y[i];
        }
        return checksum;
    }

    /// <summary>Warm both paths then time them, median of the later rounds.</summary>
    public static (double AosMs, double SoaMs) Bench(EntityAos[] aos, SoaPositions soa, double dt, int
rounds)
    {
```

listing 12: the same integration over both layouts (source: game-systems/samples/src/Ch04/Dod.cs, lines 46-74)

The arithmetic is identical, statement for statement, and the test proves it: both integrators over the same seeded data produce bitwise equal positions and checksums. What differs is only the memory traffic. Measured on this machine, .NET 10, release build, one million entities, median of thirty rounds after warmup, the array of objects takes 4.37 milliseconds per pass and the structure of arrays takes 2.09, a factor of 2.1. The benchmark test keeps a loose guard, the suite rerunning the same ordering at 300,000 entities: the soa path must never lose by more than fifty percent, so a future runtime change that inverts the honest ordering fails the suite instead of silently rotting the chapter.

## 4.2. what the soa shape buys beyond speed

```
    public byte[] Flags = new byte[x.Length];

    public int CountFlagged()
    {
        int count = 0;
        foreach (byte f in Flags)
            count += f;
        return count;
    }
```

listing 13: a flags scan reads one byte per entity (source: game-systems/samples/src/Ch04/Dod.cs, lines 18-26)

`CountFlagged` touches only the flags array, one byte per entity, while the array of objects version of the same question drags every pointer and every double into cache to read one flag. A query over a subset of fields pays for

exactly those fields, which is the same property the ecs of chapter 3 relies on when a two component query walks the shorter store. Sparse sets are dod applied to entity storage.

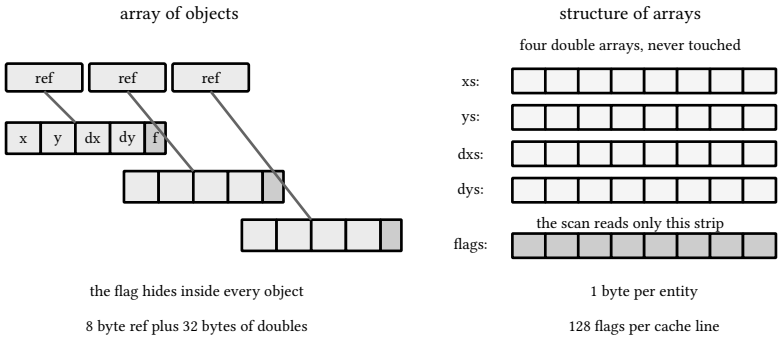



figure 10: which entities are flagged, the bytes each shape must touch

## 4.3. alignment and padding

```
    }
}

/// <summary>Two doubles and a byte, default alignment.</summary>
[StructLayout(LayoutKind.Sequential)]
public struct Wide
{
    public double X, Y;
    public byte Flags;
}

/// <summary>Same fields, packed to byte alignment.</summary>
[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct Packed
{
    public double X, Y;
    public byte Flags;
}
```

listing 14: seventeen bytes of fields, twenty four of struct (source: game-systems/samples/src/Ch04/Dod.cs, lines 26-44)

`Wide` holds two doubles and a byte, seventeen bytes of fields, and `Marshal.SizeOf` reports twenty four: the runtime pads the tail so the next element in an array starts on the largest member's alignment, eight bytes, because an unaligned double costs split accesses. `Packed` with `Pack = 1` really is seventeen, and the choice is a trade rather than a free win, packed arrays trade unaligned double access for density. The layout probe is a test because padding rules are runtime facts, not intuitions, and the numbers are asserted, not narrated.

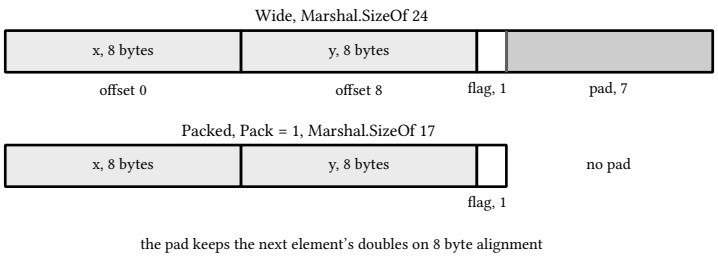



figure 11: wide at twenty four bytes against packed at seventeen, the pad labeled

**NOTE: when not to do this**

> Data oriented design pays where the same transform runs over many entities, projectiles, particles, tiles. It costs where entities are few and access is by name, one player, one camera, and a scattershot of one off systems. The capstone keeps the combatants and the turn machine as plain objects and reserves the ecs for the many: shells, embers, and particles. Optimize the loop the profiler says is hot, which chapter 13′s counting harness identifies without a profiler attached.

sources: the memory latency ladder and cache line framing follow Ulrich Drepper's "what every programmer should know about memory" and Mike Acton's 2014 cppcon talk on data oriented design, the numbers cited above are measured locally 2026-09-08 on windows/amd64, .NET 10, `Marshal.SizeOf` probes run in the test suite. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 4 tests in `GameSystems.Samples.Tests.Ch04`.

# 5. finite state machines

Every game is full of things that are always in exactly one mode. A turn is aiming, firing, or resolving, never two at once. A menu is open or closed, a unit is idle, moving, or attacking, a network session is connecting, live, or ended. The failure mode is also universal: the mode lives as a pile of bools and every code path that touches them has to keep the impossible combinations impossible. The finite state machine makes the mode a single value and the legal mode changes data.

## 5.1. transitions as table rows

```
public sealed class Fsm<TState, TEvent>(TState initial) where TState : notnull
{
    private sealed record Row(Func<bool>? Guard, Action? Action, TState To);

    private readonly Dictionary<(TState From, TEvent Ev), List<Row>> _table = new();

    public TState State { get; private set; } = initial;

    public void On(TEvent ev, TState from, TState to, Func<bool>? guard = null, Action? action = null)
    {
        if (!_table.TryGetValue((from, ev), out List<Row>? rows))
            _table[(from, ev)] = rows = new List<Row>();
        rows.Add(new Row(guard, action, to));
    }
```

listing 15: rows keyed by from state and event, guards and actions optional (source: game-systems/samples/src/Ch05/Fsm.cs, lines 10-23)

A row names its source state, its event, and its target, plus two optional hooks. A guard answers “is this change legal right now”, an action observes the change happening. Registration order matters because rows for the same `(state, event)` are tried in order and the first guard that passes wins, which is how one event routes to different targets on different conditions:

```
    public bool Fire(TEvent ev)
    {
        if (!_table.TryGetValue((State, ev), out List<Row>? rows))
            return false;
        foreach (Row row in rows)
        {
            if (row.Guard is { } guard && !guard())
                continue;
            row.Action?.Invoke();
            State = row.To;
            return true;
        }
        return false;
    }
}
```

listing 16: fire: first passing row moves the machine (source: game-systems/samples/src/Ch05/Fsm.cs, lines 25-39)

`Fire` returns a bool instead of throwing, and that is a deliberate choice worth defending in review. In a game, most events most of the time are irrelevant to most machines, the player presses jump while the menu is open, and the machine’s honest answer is “not now”, an answer the caller checks or ignores. The alternative, throwing on every

unhandled pair, turns the common case into exception-driven control flow. The trade is real though: a refused event is silent, so a wiring typo reads as “nothing happens”, which is why the tests pin refusals explicitly.

The artillery demo in the tests is the capstone’s actual turn machine in miniature: `Idle` to `Aiming` on start, `Aiming` to `Firing` on fire, `Firing` to `Resolving` on impact, and the turn end routed by a guard, more than one player alive loops back to `Aiming`, otherwise the machine lands in `GameOver` and stays. The last property is free: with no row for any event out of `GameOver`, every fire is refused, and a terminal state is simply a state with no exits.

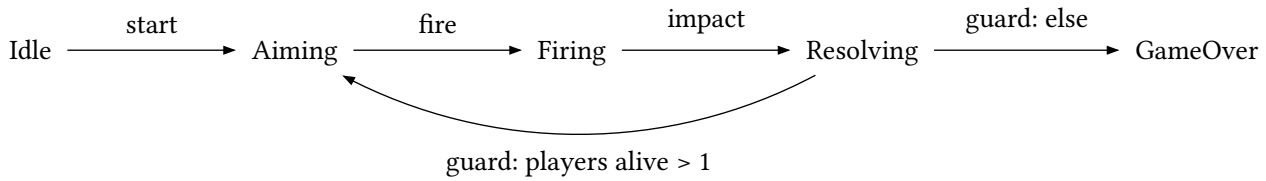



figure 12: the turn machine, one row per arrow

**NOTE: the doors out of a flat fsm**

Flat machines stop scaling when states share behavior, a pause overlay that any state can enter, or when history matters, returning to the state you came from. The standard exits are hierarchical state machines, where a state owns substates, and pushdown automata, which stack states so exiting restores the previous one. The capstone stays flat because a turn based artillery match has neither problem, and the table is auditable at a glance, every legal change is one row.

sources: gameprogrammingpatterns.com “state” chapter for the boolean pile against single state value framing, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 6 tests in `GameSystems.Samples.Tests.Ch05`.

# 6. behavior trees and utility scoring

The fsm of chapter 5 answers "what mode am I in" for one entity. The question that scales badly is "what should I do next" when the answer is a pile of conditions, fallbacks, and priorities, because nested ifs with early returns encode the same shape as a tree but cannot be inspected, reordered, or reused. The behavior tree makes the decision a data structure again, and its whole grammar is five words.

## 6.1. the grammar

A node's `Tick` returns one of three statuses: `Success`, `Failure`, or `Running` for work that spans more than one tick, a shell still flying, an animation still playing. Leaves do work, conditions turn predicates into statuses:

```
public enum Status { Success, Failure, Running }

/// <summary>One step of a decision. Trees are stateless here, ticked top to bottom.</summary>
public interface IBtNode
{
    Status Tick();
}

/// <summary>A named leaf doing the actual work.</summary>
public sealed class Leaf(string name, Func<Status> work) : IBtNode
{
    public string Name { get; } = name;
    public Status Tick() => work();
}

/// <summary>Success or failure straight from a predicate.</summary>
public sealed class Condition(Func<bool> test) : IBtNode
{
    public Status Tick() => test() ? Status.Success : Status.Failure;
}
```

listing 17: status, the leaf, and the condition (source: game-systems/samples/src/Ch06/Behavior.cs, lines 3-22)

Composites walk their children and short circuit. A `Sequence` is logical and, every child must succeed, the first failure ends the walk with failure. A `Selector` is logical or, the first success ends the walk with success, all failing fails. Both propagate `Running` upward untouched:

```
public sealed class Sequence(params IBtNode[] children) : IBtNode
{
    public Status Tick()
    {
        foreach (IBtNode child in children)
        {
            switch (child.Tick())
            {
                case Status.Failure:
                    return Status.Failure;
                case Status.Running:
                    return Status.Running;
            }
        }
        return Status.Success;
    }
}

/// <summary>Any child may succeed, the first success ends the walk, all failing fails.</summary>
public sealed class Selector(params IBtNode[] children) : IBtNode
{
    public Status Tick()
    {
        foreach (IBtNode child in children)
        {
            switch (child.Tick())
            {
                case Status.Success:
                    return Status.Success;
                case Status.Running:
                    return Status.Running;
            }
        }
        return Status.Failure;
    }
}
```

listing 18: sequence and selector, the two composites (source: game-systems/samples/src/Ch06/Behavior.cs, lines 25-60)

The decorator family is one node here, `Inverter`, which swaps success and failure and passes running through, and in the wild it is where the family grows, repeats, timeouts, cooldowns, parallels with success and failure policies. The compositor is deliberately dumb. It walks children in order and returns, which is what makes a hundred node tree still auditable, the priority of every option is literally its position.

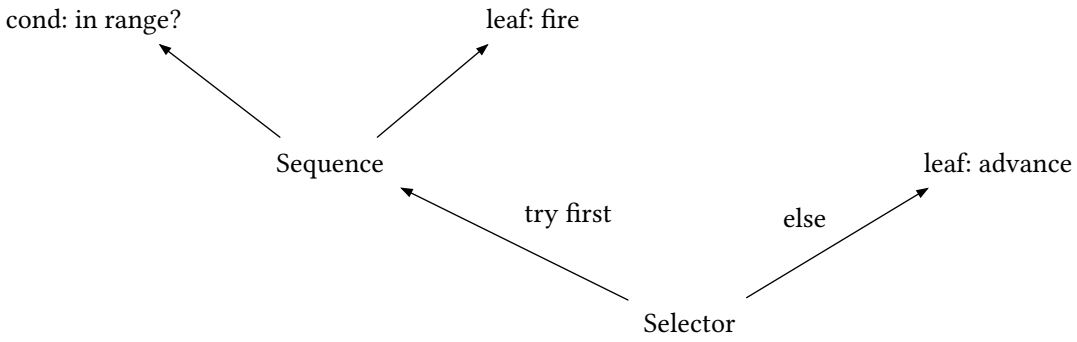



figure 13: the chapter's tree, selector takes the first succeeding child, sequence needs all

```
public sealed class Inverter(IBtNode child) : IBtNode
{
    public Status Tick() => child.Tick() switch
    {
        Status.Success => Status.Failure,
        Status.Failure => Status.Success,
        Status.Running => Status.Running,
        _ => Status.Running,
    };
}
```

listing 19: the inverter decorator (source: game-systems/samples/src/Ch06/Behavior.cs, lines 63-72)

## 6.2. the ai in miniature

```
/// <summary>
/// The capstone's ai in miniature: fire when the target is inside range,
/// otherwise advance. The tree is built once and ticked per decision.
/// </summary>
public sealed class ArtilleryAi
{
    private readonly double _range;
    private readonly IBtNode _tree;

    public double Distance { get; set; }
    public string LastAction { get; private set; } = "";

    public ArtilleryAi(double range)
    {
        _range = range;
        // the closures read Distance when ticked, not when built
        _tree = new Selector(
            new Sequence(
                new Condition(() => Distance ≤ _range),
                new Leaf("fire", () => { LastAction = "fire"; return Status.Success; })),
            new Leaf("advance", () => { LastAction = "advance"; return Status.Success; }));
    }

    public Status Tick() => _tree.Tick();
}
```

listing 20: fire in range else advance, built once, ticked per decision (source: game-systems/samples/src/Ch06/Behavior.cs, lines 74-98)

The tree closes over the ai's own mutable state, so building it once in the constructor and ticking it per decision re-reads `Distance` each time through the condition. The walk is reactive, every tick restarts from the root, which costs re-running cheap conditions and buys total statelessness, there is no memory to desynchronize when the world changes under the tree. The alternative, nodes with memory that resume where they left off, is worth having exactly when the early conditions are expensive, and the door is marked in the capstone where the ai re-solves its aim on every decision for free because aiming is arithmetic.

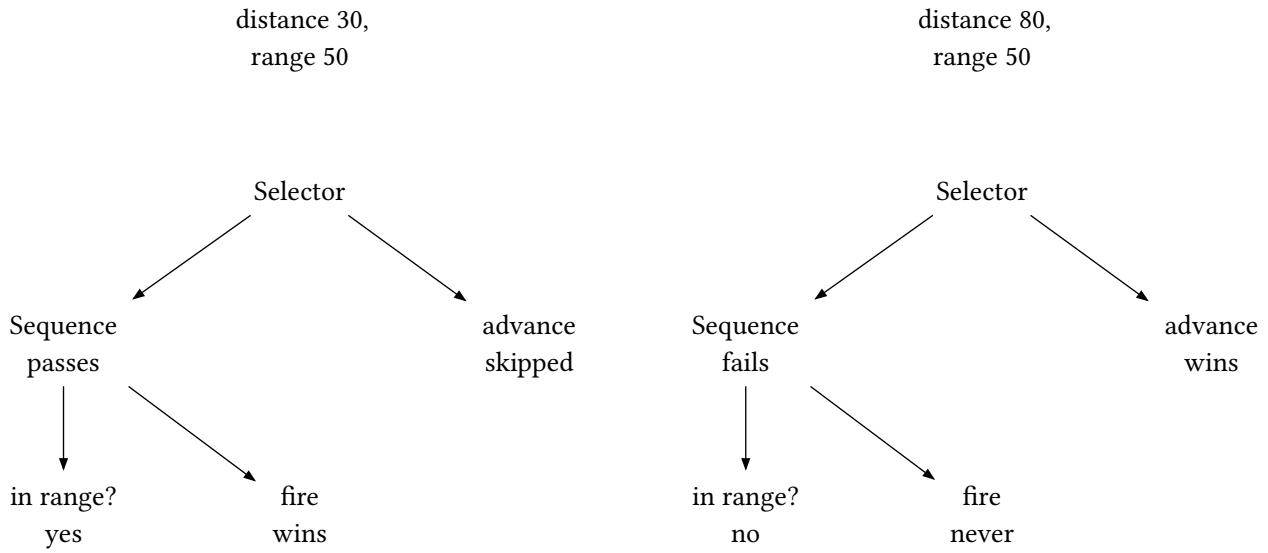



figure 14: the same tree ticked at two distances, in range fires, out of range advances

## 6.3. utility scoring

Trees decide by structure, first branch that applies wins. When the choice is instead "pick the best of N by how good each is", utility scoring is the honest tool: score every option as a number, take the max, and normalize the inputs so the numbers mean something:

```
public static class Utility
{
    /// <summary>Normalize x from [from, to] onto [0, 1], clamped outside.</summary>
    public static double Linear(double x, double from, double to) =>
        Math.Clamp((x - from) / (to - from), 0, 1);

    public static (string Name, double Score) Best(params (string Name, double Score)[] options)
    {
        (string name, double score) best = ("", double.MinValue);
        foreach (var o in options)
            if (o.Score > best.score)
                best = (o.Name, o.Score);
        return best;
    }
}
```

listing 21: linear response curves and argmax (source: game-systems/samples/src/Ch06/Behavior.cs, lines 101-115)

`Linear` maps a raw quantity, distance, health, ammo, onto 0 to 1 with clamped ends, and options combine several curves with weights before `Best` picks the winner. The capstone's ai picks shot angles this way, a score per candidate solution from expected damage and travel risk, and the framework is four lines because the discipline lives in choosing the curves, not the code. Curves beat bools for the same reason the fsm's single state beats the flag pile, they compose, 0.7 and 0.9 can be weighted against each other while "close enough" and "far" cannot.

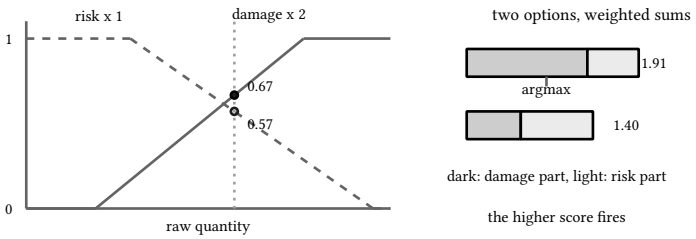



figure 15: response curves onto 0 to 1, weighted sums, argmax picks

**NOTE: trees and utilities compose**

The standard architecture is both, a behavior tree where some leaves are utility choices. The tree owns priorities and sequencing, utility owns comparisons, and the capstone's ai is exactly this hybrid.

sources: Chris Simpson, "behavior trees for ai: how they work", gamedeveloper.com, 2014, for the grammar, composite and decorator semantics, and the data driven framing, and the gdc vault talk "improving ai decision modeling through utility theory" for response curves and weighted scoring, both accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 9 tests in `GameSystems.Samples.Tests.Ch06`.

# 7. deterministic math

Chapter 1 chose the fixed timestep so the same input stream steps the same simulation. This chapter is the other half of that promise: the arithmetic inside each step must produce the same bits, and `double` does not promise that. Not because ieee 754 addition is sloppy, it is defined exactly, but because a real game does more than add. `Math.Sin` is implemented by the runtime's math library, transcendental results are not specified to the last bit, and a replay recorded on one machine can diverge on another after a few hundred calls. There is also the smaller honest point most engineers meet first:

```
0.1 + 0.1 + ... ten times
  double: 0.9999999999999999
  fixed:  raw 4294967300, bit identical everywhere
```

fragment, illustrative, not executed

Binary floating point cannot represent 0.1, so ten additions land a hair under one. That is not a bug, it is the format, and the test pins both sides of the comparison: the double misses, the fixed point lands within one part in a billion and, more importantly, lands on the same raw bits on every machine that runs the line.

## 7.1. Q31.32 in one struct

```
public readonly record struct Fp(long Raw)
{
    public static Fp Zero => new(0);

    public static Fp FromInt(long value) => new(checked(value << 32));

    public static Fp FromDouble(double value) =>
        new((long)Math.Round(value * 4294967296.0));

    public double ToDouble() => (double)Raw / 4294967296.0;

    public static bool operator <(Fp a, Fp b) => a.Raw < b.Raw;
    public static bool operator >(Fp a, Fp b) => a.Raw > b.Raw;
    public static bool operator <=(Fp a, Fp b) => a.Raw ≤ b.Raw;
    public static bool operator ≥(Fp a, Fp b) => a.Raw ≥ b.Raw;

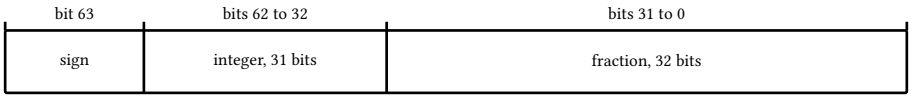
    public static Fp operator +(Fp a, Fp b) => new(checked(a.Raw + b.Raw));
    public static Fp operator -(Fp a, Fp b) => new(checked(a.Raw - b.Raw));
    public static Fp operator -(Fp a) => new(checked(-a.Raw));

    public static Fp operator *(Fp a, Fp b) =>
        new(checked((long)((Int128)a.Raw * b.Raw >> 32)));

    public static Fp operator /(Fp a, Fp b)
    {
        if (b.Raw == 0)
            throw new DivideByZeroException();
        return new(checked((long)(((Int128)a.Raw << 32) / b.Raw)));
    }
```

listing 22: one long, 31 integer bits, 32 fraction bits, checked everywhere (source: game-systems/samples/src/Ch07/Fp.cs, lines 9-37)

Fixed point is just integers with a convention: the low 32 bits are fraction. Addition and subtraction are integer operations, exact. Multiplication produces a 64.64 intermediate through `Int128`, which .NET 10 ships natively, then shifts back down, and division expands the numerator the same way. The `checked` context is the discipline: when a value leaves the representable range the operation throws instead of silently wrapping to garbage, because a wrapped coordinate in a replay is a corruption that surfaces three systems later as a mystery. The overflow test pins both sides, `2^20` squared throws, `128` to the fourth is exactly `2^28`.




raw 4294967300 is one integer plus four fraction units of 2^-32

figure 16: the 64 raw bits of one fixed point value

The conversions bracket the boundary honestly. `FromDouble` and `ToDouble` exist for input, display, and tests, and the rule the capstone follows is absolute: doubles may cross the skin of the simulation, never live inside it.

## 7.2. trig without a math library

```
            throw new DivideByZeroException();
        return new(checked((long)(((Int128)a.Raw << 32) / b.Raw)));
    }

    /// <summary>Cordic sine and cosine of an angle in radians, |angle| under 1.74.</summary>
    public static (Fp Cos, Fp Sin) SinCos(Fp angle)
    {
        long x = 0x9B74EDA8L; // 0.6072529... the cordic scaling constant
        long y = 0;
        long z = angle.Raw;
        for (int i = 0; i < 32; i++)
        {
            long tx = x, ty = y;
            if (z >= 0)
            {
                x -= ty >> i;
                y += tx >> i;
                z -= AtanTable[i];
            }
            else
            {
                x += ty >> i;
                y -= tx >> i;
                z += AtanTable[i];
```

listing 23: cordic rotation: shifts and adds (source: game-systems/samples/src/Ch07/Fp.cs, lines 35-58)

Sin and cos come from cordic, the shift-and-add algorithm from 1959 that hardware still uses in mode. Starting from the constant `0.60725...` on the x axis, each step rotates the vector by `atan(2^-i)` toward the target angle, and rotation by a power of two is just a shift. After 32 steps the vector is `(cos, sin)` to about nine decimal digits, the tests compare against `Math.Sin` and `Math.Cos` at `1e-6` and check the pythagorean identity holds inside fixed point. The catch that matters for replays: the atan table is generated once and frozen as hex literals in the source, so no runtime math library can ever move a digit of it. A table computed lazily from `Math.Atan` would reintroduce the exact dependency this chapter exists to remove.

The range limit is stated in the code and worth restating: this `SinCos` converges only for angles under about 1.74 radians, and the capstone never asks beyond a quarter turn, elevation angles live in zero to ninety degrees. General angles need range reduction first, and doing that in fixed point is its own exercise.

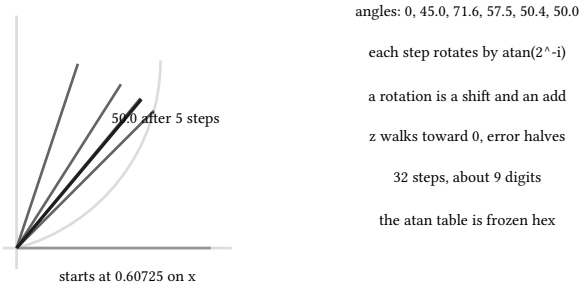



figure 17: cordic walking the vector onto the target angle, one shift-add per step

```
/// <summary>Integer newton iteration, seeded above the answer so it can only walk down.</summary>
public static Fp Sqrt(Fp v)
{
    if (v.Raw < 0)
        throw new ArgumentOutOfRangeException(nameof(v));
    if (v.Raw == 0)
        return Zero;
    Int128 n = (Int128)v.Raw << 32;
    // any raw value here has a root under 2^48, so the seed is always above it
    Int128 r = (Int128)1 << 48;
    while (true)
    {
        Int128 next = (r + n / r) >> 1;
        if (next ≥ r)
            break;
        r = next;
    }
    return new Fp((long)r);
}
```

listing 24: integer newton square root, seeded above the answer (source: game-systems/samples/src/Ch07/Fp.cs, lines 76-94)

Square root is newton iteration on integers, seeded at `2^48` which is above every root a legal raw value can have, so the iteration only walks down and its stopping rule, stop when the next estimate fails to decrease, is exact for integer square roots. Any seed above the answer converges to the same raw bits, which is the whole point.

## 7.3. what determinism costs

**WARNING: determinism is not exactness**

Fixed point does not make arithmetic exact, `0.1` is not representable in binary fixed point either, and mul followed by div does not round trip exactly. What it makes identical: the same operation sequence produces the same raw bits on every machine, every runtime, every optimization level, because there is no library call and no unspecified rounding anywhere in the path. Replays need identical, not exact, and identical is a property you can test: the projectile test runs a 300 step integration twice and compares raw bits, the same assertion the capstone's replay suite makes about whole matches.

The costs are the other side of the ledger. Range is finite and throws when exceeded, precision is 32 fraction bits everywhere with no denormals near zero, and every trig call is a 32 step loop. For a turn based artillery sim at 60 steps a second with hundreds of shells, all three costs are invisible, and chapter 13's counting harness proves it.

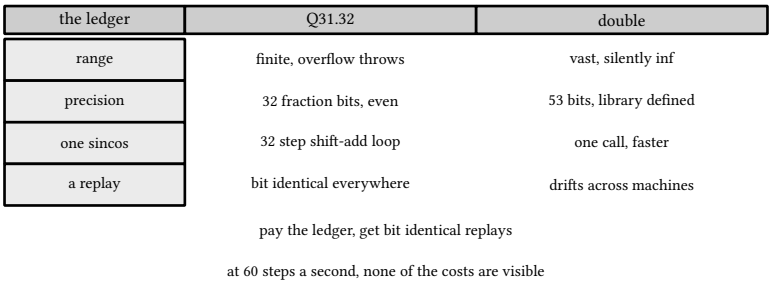



figure 18: the cost ledger, fixed point against double

sources: learn.microsoft.com System.Math.Sin remarks, “this method calls into the underlying C runtime, and the exact result or valid input range may differ between different operating systems or architectures”, which is the portability hole this chapter closes, the cordic algorithm and convergence constant from the cordic page on en.wikipedia.org checked against local .NET 10 runs, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 8 tests in `GameSystems.Samples.Tests.Ch07`.

# 8. physics

The physics an artillery game needs is one hundred and fifty years old: a point mass under constant gravity and a wind field, flying until it meets the ground. What makes it game physics instead of textbook physics are the three constraints around it: it must be cheap enough for hundreds of shells at sixty steps a second, it must be deterministic per chapter 7 so replays replay, and it must not tunnel, a fast shell stepping over a thin hill and hitting the far side that is not there.

## 8.1. integration, velocity first

```
public readonly record struct Projectile(Fp X, Fp Y, Fp Vx, Fp Vy)
{
    public Projectile Step(Fp dt, Fp g, Fp wind)
    {
        Fp vx = Vx + wind * dt;
        Fp vy = Vy + g * dt;
        return this with { X = X + vx * dt, Y = Y + vy * dt, Vx = vx, Vy = vy };
    }
}
```

listing 25: semi-implicit euler: velocity, then position with the new velocity (source: game-systems/samples/src/Ch08/Physics.cs, lines 10-18)

The one-line difference between explicit euler, position first, and semi-implicit euler, velocity first, is the difference between gaining energy every step and not. Semi-implicit euler is symplectic, the discrete orbit it traces has a bounded energy error instead of a growing one, and for a ballistics game under constant acceleration it is the whole correct answer. The tests check it against the analytic parabola three ways: height after five seconds of flight within half a meter of the closed form, a forty five degree shot at one hundred units per second landing within two percent of the analytic `v^2/g` range, and the apex within one of `250`, with all arithmetic in fixed point.

Wind is an acceleration like gravity, so a tailwind is literally the same `Step` with a positive `wind` term, and the tailwind outranges headwind test is a property of the integrator rather than a special case in it.

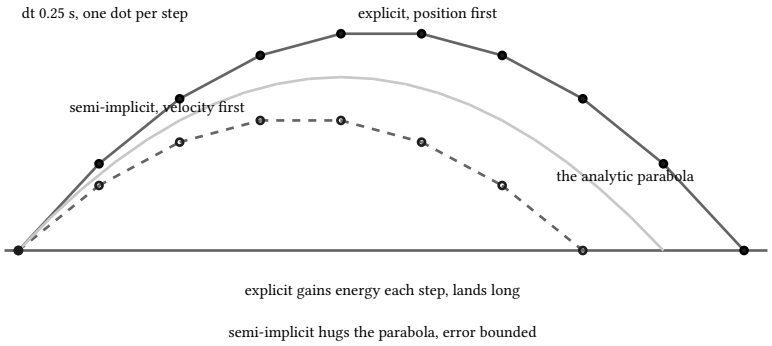



figure 19: one shot, two integrators, explicit gains energy every step

## 8.2. the heightfield and its questions

```
public sealed class Terrain(Fp[] columns)
{
    public int Width => columns.Length;

    /// <summary>The live height array: carving mutates it in place.</summary>
    public Fp[] Columns => columns;

    public Fp HeightAt(Fp x)
    {
        Fp col = x;
        int lo = (int)(col.Raw >> 32);
        if (lo < 0)
            return columns[0];
        if (lo ≥ columns.Length - 1)
            return columns[^1];
        Fp frac = new Fp(col.Raw & 0xFFFF_FFFF);
        return columns[lo] + (columns[lo + 1] - columns[lo]) * frac;
    }
```

listing 26: columns sampled linearly, how high is the ground at x (source: game-systems/samples/src/Ch08/Physics.cs, lines 21-39)

Terrain here is one height per integer column, the representation chapter 9 generates and the renderer draws. `HeightAt` answers "how high is the ground at x" by linear interpolation between the two neighbors, all in raw fixed point: the column index is the high 32 bits of x, the fraction is the low 32, and there is a rounding fact worth noticing, the fraction is an unsigned interpretation of bits that `Raw` holds signed, so the mask comes before anything else touches it.

```
    public void Carve(Fp x, Fp radius)
    {
        int center = (int)(x.Raw >> 32);
        int r = (int)(radius.Raw >> 32);
        for (int i = Math.Max(0, center - r); i ≤ Math.Min(columns.Length - 1, center + r); i++)
        {
            int dx = i - center;
            // parabola in integer raw units: depth = (r^2 - dx^2) / (4r)
            long depth = (r == 0) ? 0 : ((long)(r * r - dx * dx) << 30) / r;
            columns[i] -= new Fp(depth);
        }
    }
}
```

listing 27: carving a crater as an integer parabola (source: game-systems/samples/src/Ch08/Physics.cs, lines 41-53)

`Carve` answers "what does an explosion do to the shape": each column within the blast radius is depressed by a parabola, and the depth is computed with one integer expression, `((r^2 - dx^2) << 30) / r`. An earlier draft of this method used a `double` in the middle, quietly reintroducing exactly the nondeterminism chapter 7 exists to remove, and the code review rule that caught it is the book's rule: doubles may cross the skin of the simulation, never live inside it.

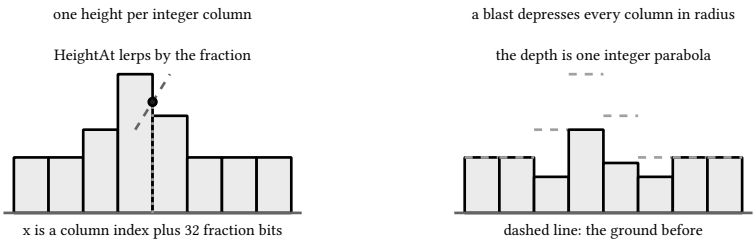



figure 20: the heightfield: lerp between columns, carve an integer parabola

## 8.3. collision without tunneling

```
public static class Collision
{
    /// <summary>
    /// March one flight segment against the heightfield: for every column
    /// the segment crosses, compare the segment's height there with the
    /// terrain's, and report the first crossing.
    /// </summary>
    public static (bool Hit, Fp X, Fp Y) SegmentTerrain(Fp x0, Fp y0, Fp x1, Fp y1, Terrain terrain)
    {
        if (x1 == x0)
            return (y1 ≤ terrain.HeightAt(x1), x1, y1);

        int from = Math.Max(0, (int)(x0.Raw >> 32));
        int to = Math.Min(terrain.Width - 1, (int)(x1.Raw >> 32));
        int step = x1 ≥ x0 ? 1 : -1;

        for (int col = from; step > 0 ? col ≤ to : col ≥ to; col += step)
        {
            Fp cx = Fp.FromInt(col);
            if (cx < (x0 < x1 ? x0 : x1) || cx > (x0 < x1 ? x1 : x0))
                continue;
            if (cx == x0)
                continue; // t == 0 is the launch, not a crossing
            Fp t = (cx - x0) / (x1 - x0);
            Fp y = y0 + (y1 - y0) * t;
            if (y ≤ terrain.HeightAt(cx))
                return (true, cx, y);
        }

        // endpoints themselves: a drop that lands inside one column
        if (y1 ≤ terrain.HeightAt(x1))
            return (true, x1, y1);
        return (false, x1, y1);
    }
}
```

listing 28: segment against heightfield, first crossing wins, the launch column skipped (source: game-systems/samples/src/Ch08/Physics.cs, lines 55-89)

The flight step gives a segment from the old position to the new one, and the collision test walks that segment across every terrain column it passes over, comparing the segment's height with the ground's at each column. Because the test is continuous along the segment rather than sampled at the endpoints, a fast shell cannot tunnel: the crossing column pins the impact between two heights, and the impact point is interpolated on the segment. The start of the segment is skipped, a shell launches from the surface, and `t == 0` is a starting condition, not a crossing. The tests pin the cases, a descending segment over a rising hill reports the crossing on the hill's side, an

ascending path over flat ground never hits, a vertical drop is handled without dividing by zero on the zero-width segment, and a segment launched from the surface itself neither lands at its launch column nor skips the crossing past it.

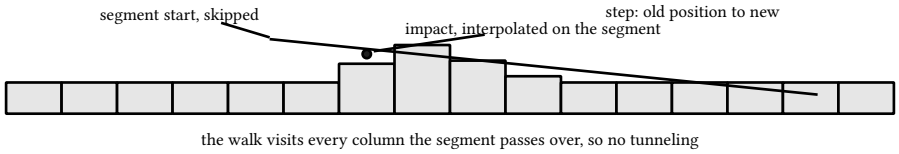



figure 21: one flight step against the heightfield, the crossing column wins

**NOTE: what this physics is not**

No rigid bodies, no rotation, no restitution, no friction beyond stopping, and no continuous collision between shells. The game is turn based artillery, the only bodies are points, and the only surfaces are the heightfield. Adding circle-shell collision later means distance checks against the same segment machinery, and restitution means the integrator gains a velocity reflection at the impact normal. Both are doors, not requirements.

sources: semi-implicit euler and its symplectic property follow Glenn Fiedler's integration chapters on gafferongames.com, the same series chapter 1 cites for the fixed timestep, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 9 tests in `GameSystems.Samples.Tests.Ch08`.

# 9. terrain: heightmaps

The terrain of the artillery game is one array: a height per column. That representation, a heightfield, is chosen before any generation happens, because everything downstream consumes it, chapter 8′s collision walks it column by column, the renderer of chapter 21 draws it as a polyline, craters rewrite individual entries, and a replay only needs the seed that produced it. Generation is where determinism earns its keep: the same seed must rebuild the same map bit for bit or the replay of a match played on a hill replays onto a different hill.

## 9.1. value noise from a hash

```
{
    /// <summary>Height at one lattice knot, straight from the hash.</summary>
    public static Fp Knot(ulong seed, int knot, Fp amplitude)
    {
        ulong z = Mix64(seed ^ (0x9E3779B97F4A7C15UL * (ulong)knot));
        // top 32 bits as the fraction [0,1) times the amplitude
        return new Fp(checked((long)(((Int128)(long)(z >> 32) * amplitude.Raw) >> 32)));
    }

    public static Fp[] ValueNoise(int width, ulong seed, int lattice, Fp amplitude)
    {
        Fp[] map = new Fp[width];
        for (int x = 0; x < width; x++)
        {
            int lo = x / lattice;
            Fp a = Knot(seed, lo, amplitude);
            Fp b = Knot(seed, lo + 1, amplitude);
            Fp t = new Fp(((long)(x % lattice) << 32) / lattice);
            map[x] = a + (b - a) * Smooth(t);
        }
        return map;
```

listing 29: a knot height straight from splitmix64, then smoothstep between knots (source: game-systems/samples/src/Ch09/Heightmap.cs, lines 11-31)

A lattice point, a knot every 16 columns, gets its height by hashing its coordinates. The hash is splitmix64′s finalizer, the avalanche stage whose absence bit book 12, and chapter 11 owns the generator: without it, adjacent knots share high bits and the terrain comes out as broad terraces instead of hills. The top 32 bits of the mixed value become a fraction of the amplitude, and between knots the height is the smoothstep of the interpolation parameter, `3t^2 - 2t^3`, which has zero slope at both ends and therefore no visible seam at the knots.

The tests pin the contract from both ends. At a knot the map equals the hash output exactly, no interpolation, and between knots the step from one column to the next is bounded by a quarter of the amplitude, the suite’s slack bound over the roughly nine percent smoothstep’s maximum slope of 1.5 allows across a 16 column cell, so a jump in the map is a bug in the generator, not noise being noisy.

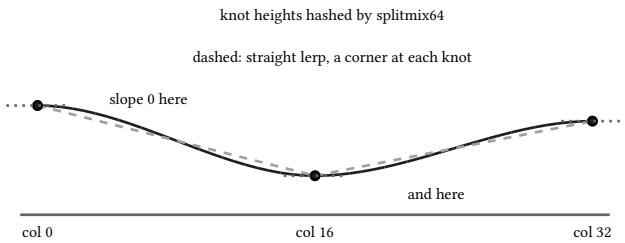



figure 22: two knots hashed by splitmix64, smoothstep between them

## 9.2. octaves

```
/// <summary>Fractal noise: octaves of value noise at doubling frequency, halving amplitude.</summary>
public static Fp[] Fbm(int width, ulong seed, int octaves, Fp amplitude)
{
    Fp[] map = new Fp[width];
    Fp amp = amplitude;
    int lattice = Math.Max(1, width / 4);
    for (int o = 0; o < octaves; o++)
    {
        Fp[] layer = ValueNoise(width, seed + (ulong)o * 0x9E3779B97F4A7C15UL, lattice, amp);
        for (int x = 0; x < width; x++)
            map[x] += layer[x];
        amp = new Fp(amp.Raw / 2);
```

listing 30: fractional brownian motion: stack octaves at halving amplitude (source: game-systems/samples/src/Ch09/Heightmap.cs, lines 33-45)

One noise layer is one hill every 32 columns, rolling but featureless. Fractal noise stacks layers, octave two at half the amplitude and twice the frequency, octave three at a quarter, and the sum reads as a mountain range: big shapes from the low octaves, texture from the high ones. The roughness test measures this as total variation over the map and demands that four octaves vary more than one, the same number a designer tunes by ear, octave count is the terrain's roughness dial, amplitude its height dial, seed its identity.

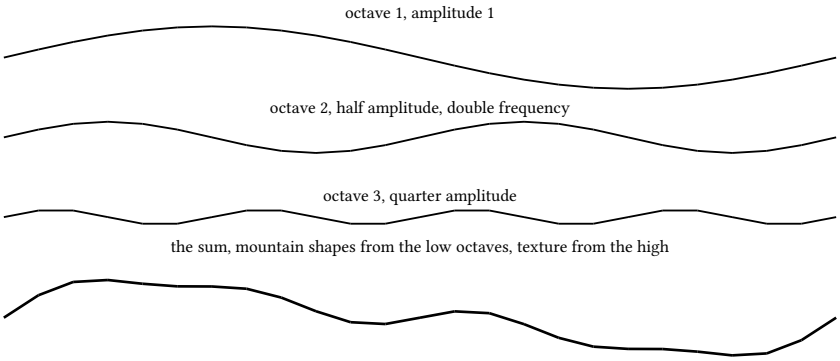



figure 23: octaves at halving amplitude and doubling frequency, and their sum

**NOTE: why the seed is the whole map**

A 512 column map is four kilobytes of state. A match with craters cannot be reproduced from the seed alone, because carving changes columns after generation, and the replay format of chapter 12 carries the seed plus every shot, so the craters rebuild themselves from the same inputs. The map is a function of its seed and its wound history, nothing else.

sources: value noise, smoothstep interpolation, and octave stacking follow the standard noise treatments, Iñigo Quílez's articles at iquilezles.org on value noise among them, and the splitmix64 finalizer constants from the splitmix paper as used in book 12 chapter 12, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 6 tests in `GameSystems.Samples.Tests.Ch09`.

# 10. turn systems and game flow

Real time games spend their complexity on the loop. Turn based games spend it on the queue: who acts, in what order, and what happens to the order when someone stops existing mid round. The artillery capstone is turn based, and its fairness lives entirely in this chapter's data structure.

## 10.1. initiative order

```
public sealed class TurnQueue(IEnumerable<(int Id, int Initiative)> entries)
{
    // OrderBy is stable, so initiative ties keep registration order
    private readonly List<int> _alive = entries
        .OrderByDescending(e => e.Initiative)
        .Select(e => e.Id)
        .ToList();

    private int _cursor;

    public int Round { get; private set; } = 1;

    public int Current => _alive[_cursor];

    public IReadOnlyList<int> Order => _alive;
```

listing 31: sorted by initiative, ties by registration, a cursor into the living (source: game-systems/samples/src/Ch10/TurnQueue.cs, lines 8-22)

The order is computed once from the roster, and the sort is stable, which is not decoration here: initiative ties resolved by registration order are the same discipline book 10′s sorting chapter defended for `OrderBy`, an unstable sort would make the tie break change between runs and the match would not replay. `Round` counts completed cycles of the living, `Current` is a cursor, and the structure never re-sorts, it only removes.

```
    public void Advance()
    {
        _cursor++;
        if (_cursor ≥ _alive.Count)
        {
            _cursor = 0;
            Round++;
        }
    }

    /// <summary>Drop an actor, alive or already gone, keeping the turn order intact.</summary>
    public void Remove(int id)
    {
        int at = _alive.IndexOf(id);
        if (at < 0)
            return;
        _alive.RemoveAt(at);
        if (at < _cursor)
            _cursor--;
        if (_cursor ≥ _alive.Count)
        {
            _cursor = 0;
            Round++;
        }
    }
}
```

listing 32: advance wraps, removal fixes the cursor (source: game-systems/samples/src/Ch10/TurnQueue.cs, lines 24-49)

`Remove` is where turn queues earn their keep. An actor dying mid round must not skip or double the next actor's turn: removing an entry before the cursor pulls the cursor back by one so the same actor stays current, removing the current actor leaves the cursor pointing at the successor, and a removal that empties the tail wraps with a round increment. The tests walk every case, including the one that locks most hand rolled implementations, an actor marked dead before their turn comes up is simply never visited, no skip logic anywhere.

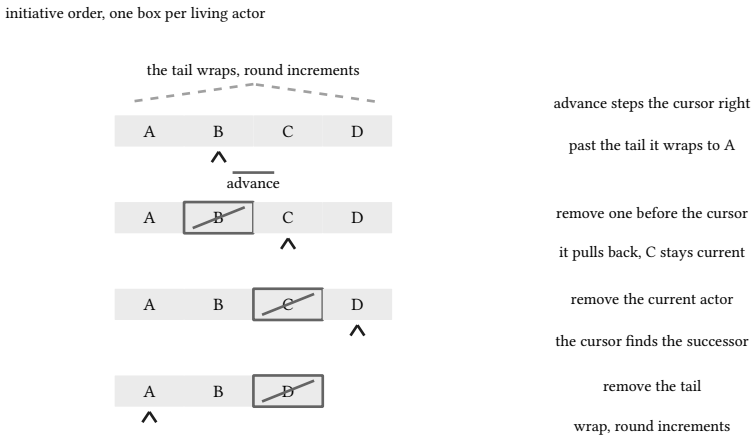



figure 24: one round on the queue, the cursor through advance and three removals

**NOTE: simultaneous and team turns**

The queue here is strict alternation, one actor fully resolves before the next. The two standard variants are simultaneous turn, everyone commits orders blind then all resolve together, which needs an order buffer and a deterministic resolution tiebreak, and team phase turn, all of one side moves then all of the other, which is

> this queue with the team id as sort key. The capstone keeps strict alternation because one shell in flight per turn is the game.

sources: initiative queues and the cursor discipline follow standard turn based design treatments, cross checked against the turn scheduling vocabulary in the gameprogrammingpatterns.com game loop chapter cited in chapter 1, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 7 tests in `GameSystems.Samples.Tests.Ch10`.

# 11. randomness and determinism discipline

A game rng has one job and it is not statistical elegance. It is that every machine drawing “random” numbers produces the same “random” numbers, because terrain, ai decisions, and wind gusts are all replay inputs. `System.Random` seeds itself from the clock when you do not pass a seed, and even seeded, its algorithm is an implementation detail the runtime may change between versions, which makes it the wrong tool for anything a replay touches. The fix is the same as chapter 7′s: own the arithmetic.

## 11.1. splitmix64

```
public sealed class Rng(ulong state)
{
    private ulong _state = state;

    public ulong NextUlong()
    {
        _state += 0x9E3779B97F4A7C15UL;
        ulong z = _state;
        z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9UL;
        z = (z ^ (z >> 27)) * 0x94D049BB133111EBUL;
        return z ^ (z >> 31);
    }
```

listing 33: a counter through an avalanche finalizer (source: game-systems/samples/src/Ch11/Rng.cs, lines 11-23)

Splitmix64 is a state counter plus two xorshift-multiply rounds and a final bare xor. There is nothing to initialize and nothing platform dependent, one `ulong` in, the same `ulong` sequence out, forever. It is not cryptographic and a game does not want cryptographic, players figuring out the wind pattern from saves is a feature, it wants period, speed, and bit-identical streams.

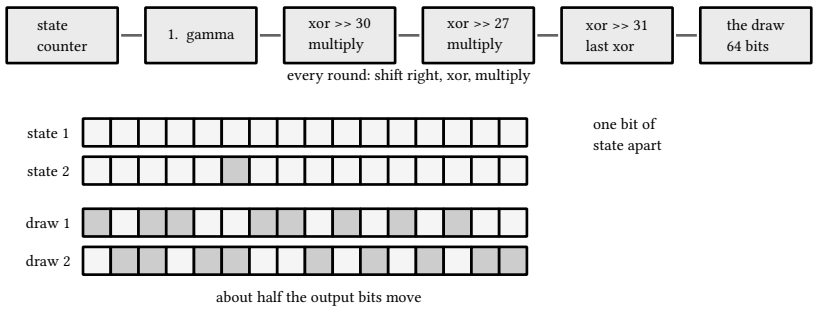



figure 25: splitmix64: a counter through two xorshift-multiply rounds and a final xor

## 11.2. unbiased integers

```
    public int NextInt(int maxExclusive)
    {
        if (maxExclusive ≤ 0)
            throw new ArgumentOutOfRangeException(nameof(maxExclusive));
        ulong range = ulong.MaxValue / (ulong)maxExclusive;
        ulong limit = range * (ulong)maxExclusive;
        ulong z;
        do { z = NextUlong(); } while (z ≥ limit);
        return (int)(z / range);
    }

    /// <summary>Uniform [0, 1) in fixed point, the top 32 bits of one draw.</summary>
    public Fp NextFp() ⇒ new((long)(NextUlong() ≫ 32));
```

listing 34: rejection sampling against a bucket limit (source: game-systems/samples/src/Ch11/Rng.cs, lines 25-37)

The tempting integer draw is `NextUlong() % max`, and modulo is biased whenever `2^64` is not a multiple of `max`, which is always. Seven buckets do not divide a power of two, so some buckets get more of the raw range than others. Rejection sampling fixes it: draw, discard draws at or above the largest exact multiple of the bucket count, and every bucket receives the same width of raw range. The uniformity test draws seventy thousand values into seven buckets and checks each against ten thousand plus or minus five hundred, and because the seed is fixed those bounds are not flaky, they are either true of this deterministic stream or the sampler is biased. Fixed point draws take the top 32 bits of one draw as a value in `[0, 1)`.

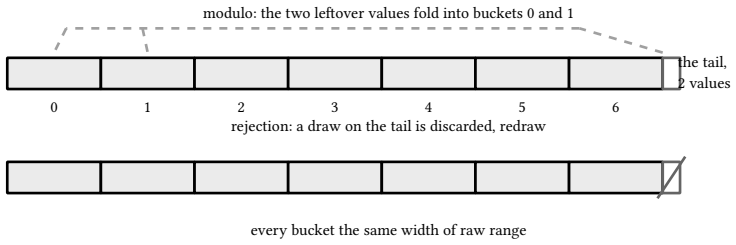



figure 26: seven buckets: modulo folds the leftover, rejection discards it

## 11.3. streams

```
    /// <summary>
    /// A named child generator: fnv-1a over the name, through the same
    /// finalizer book 12 chapter 12 taught, so subsystem streams never
    /// share lattice structure with each other.
    /// </summary>
    public Rng Stream(string name)
    {
        ulong h = 0xcbf29ce484222325UL;
        foreach (byte b in Encoding.UTF8.GetBytes(name))
            h = (h ^ b) * 0x100000001b3UL;
        h ^= _state;
        h = (h ^ (h ≫ 30)) * 0xBF58476D1CE4E5B9UL;
        h = (h ^ (h ≫ 27)) * 0x94D049BB133111EBUL;
        return new Rng(h ^ (h ≫ 31));
    }
```

listing 35: fnv over the name, finalizer, fork (source: game-systems/samples/src/Ch11/Rng.cs, lines 39-53)

One generator shared by every subsystem is a determinism bug with a delay fuse: fix the wind by drawing one extra number and the ai's decisions for the rest of the match change. The discipline is to fork named streams at

construction, wind from `Stream("wind")`, terrain from `Stream("terrain")`, so subsystems never consume each other's draws. The stream seed is the name hashed with fnv-1a and passed through the same splitmix finalizer, and the reason for the finalizer is book 12's chapter 12 finding again: raw fnv shares high bits across similar inputs, and seeded streams named `"a"`, `"b"`, `"c"` would land in correlated corners of the state space.

The last test states the discipline as an assertion: consuming from the parent before forking shifts every child, so the fork point is part of the replay contract. The capstone follows it twice over: the battle forks `"wind"` in its constructor, and the campaign forks `"battle"`, `"loot"`, `"craft"`, and `"hatch"` in its own, deriving per stage battle seeds without a parent draw.

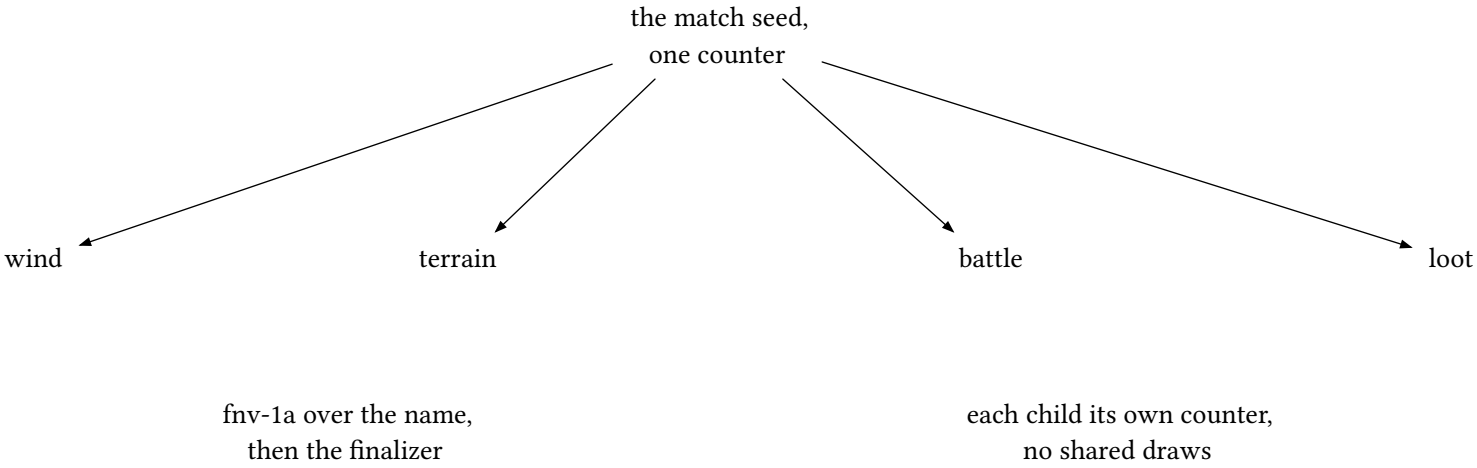



figure 27: the match seed forking named streams, children never share draws

**PITFALL: the shared static rng**

`static Random.Shared` is correct for cosmetic effects, muzzle sparks, screenshake, that no replay observes. The moment a draw changes game state it belongs to a named stream from the match seed, and the test suite's determinism check, same inputs, same state hash, is the guard that catches a smuggled `Random` call.

sources: splitmix64 constants from Steele, Lea, and Flood's "fast splittable pseudorandom number generators" paper, and learn.microsoft.com System.Random remarks, "the implementation of the random number generator in the Random class isn't guaranteed to remain the same across major versions of .NET", which is the hole this chapter's owned arithmetic closes, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 6 tests in `GameSystems.Samples.Tests.Ch11`.

# 12. serialization and save systems

A replay and a save file are the same question at different sizes: how does game state cross bytes without losing the determinism the previous five chapters built. The answer here is the input log. A match is recorded as its seed plus one entry per turn, and playback re-runs the same simulation on the same inputs, so the file is dozens of bytes and the state hash at the end, a 64-bit fold of the whole world that chapter 13 builds, proves the run matched. The alternative, snapshotting the whole world every turn, spends kilobytes per turn and buys a second, driftier representation of the game to keep in sync.

## 12.1. the format is the contract

```
public static byte[] Save(Replay replay)
{
    // 4 magic, 1 version, 4 width, 8 seed, 4 count, then 12 per turn
    var bytes = new byte[21 + replay.Turns.Count * 12];
    Magic.CopyTo(bytes, 0);
    bytes[4] = Version;
    BinaryPrimitives.WriteUInt32BigEndian(bytes.AsSpan(5, 4), (uint)replay.Width);
    BinaryPrimitives.WriteUInt64BigEndian(bytes.AsSpan(9, 8), replay.Seed);
    BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(17, 4), replay.Turns.Count);
    int at = 21;
    foreach (TurnInput t in replay.Turns)
    {
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at, 4), t.PlayerId);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 4, 4), t.AngleDegrees);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 8, 4), t.Power);
        at += 12;
    }
    return bytes;
```

listing 36: magic, version, length prefixed fields, twelve bytes per turn (source: game-systems/samples/src/Ch12/ReplayCodec.cs, lines 21-38)

Every field is fixed width, big endian, written with `BinaryPrimitives`, and the sizes are stated where they are used. The magic answers “is this our file”, the version answers “can this build read it”, and the version policy is the part that keeps replays loadable: version 1 files load forever, later formats migrate forward, and nothing is ever parsed by guessing. The tests treat the codec as a protocol implementation, not a convenience: wrong magic, unknown version, and a turn list that claims more entries than the bytes hold are each rejected with a message naming the field that broke.

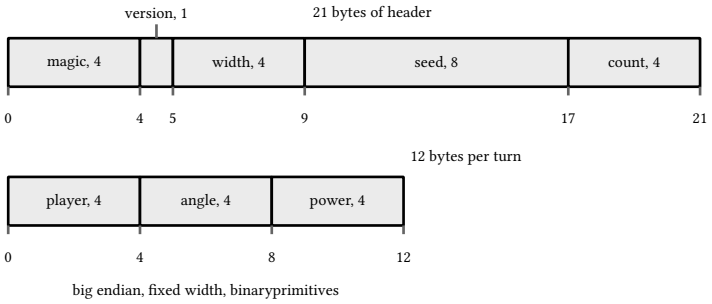



figure 28: the replay file as one byte map, 21 of header, 12 per turn

```
    public static Replay Load(byte[] bytes)
    {
        if (bytes.Length < 21)
            throw new InvalidDataException($"replay too short, {bytes.Length} bytes before the header
ends");
        for (int i = 0; i < 4; i++)
            if (bytes[i] != Magic[i])
                throw new InvalidDataException($"bad magic {bytes[..4]}");
        if (bytes[4] != Version)
            throw new InvalidDataException($"replay version {bytes[4]}, this build reads {Version}");
        int width = BinaryPrimitives.ReadInt32BigEndian(bytes.AsSpan(5, 4));
        ulong seed = BinaryPrimitives.ReadUInt64BigEndian(bytes.AsSpan(9, 8));
        int count = BinaryPrimitives.ReadInt32BigEndian(bytes.AsSpan(17, 4));
        if (bytes.Length < 21 + count * 12)
            throw new InvalidDataException($"turn list truncated, {bytes.Length - 21} bytes for {count}
turns");
        var turns = new List<TurnInput>(count);
        int at = 21;
        for (int i = 0; i < count; i++)
        {
            turns.Add(new TurnInput(
                BinaryPrimitives.ReadInt32BigEndian(bytes.AsSpan(at, 4)),
                BinaryPrimitives.ReadInt32BigEndian(bytes.AsSpan(at + 4, 4)),
                BinaryPrimitives.ReadInt32BigEndian(bytes.AsSpan(at + 8, 4))));
            at += 12;
        }
        return new Replay(seed, width, turns);
    }
```

listing 37: load validates before it allocates, reads exactly what save wrote (source: game-systems/samples/src/Ch12/ReplayCodec.cs, lines 40-66)

Two properties the tests pin are worth defending in review. The round trip is byte deterministic, the same replay saves to the same bytes every time, which is what lets a hash of the file stand in for a hash of the match. And the input log size test is the whole argument for the design, four turns cost 69 bytes against 16 kilobytes of per turn terrain snapshots, more than two orders of magnitude, and the gap widens with match length because the log grows per turn while snapshots grow per turn times world size.

**WARNING: when snapshots are the right answer**

Input logs require the simulation to be a function of its inputs. The capstone is, by construction, that is what deterministic math, owned randomness, and deferred structural changes buy. The moments a game stops being a pure function, a network correction rewinds state, a patch changes balance constants, the input log replays the wrong match, and the honest fix is periodic checkpoints inside the log, a full state snapshot every N turns so playback resynchronizes instead of drifting. Version the constants file like the format. The capstone exercises both predictions of this callout: its replay v2 bumped the magic and widened the turn record when weapons joined the log, and the character profile saves as a checkpoint, chapter 14's codec.

## 12.2. the capstone save file

The capstone's checkpoint rides the same discipline on SQLite: one file, six tables, written whole inside one transaction, a schema version standing where the binary codecs keep their magic. `Campaign.Save(path)` records the sheet, the loadout, the upgrade levels, the pets in slot order, the stages cleared per expedition, and the profile seed, because a loaded campaign has to draw the futures the seed always drew. Every call opens the file, commits, and closes, pooling off, so no handle outlives a save or a load and the writer can be gone by the time the reader arrives.

The paged inventory grid that chapter 15 builds is the delicate rebuild. Add fills the next empty cell and Take empties the first stacks, so the restore replays the cell vector through both, reserving each hole with a sentinel id until the real stacks sit where they belong, holes and page count included.

```
    private static InventoryGrid RebuildGrid(IReadOnlyList<(int Cell, int ItemId, int Count)> stacks, int
cellCount)
    {
        if (cellCount < InventoryGrid.CellsPerPage || cellCount % InventoryGrid.CellsPerPage ≠ 0)
            throw new InvalidDataException($"grid cell count {cellCount}, not whole pages");
        var grid = new InventoryGrid();
        int at = 0;
        int sentinels = 0;
        foreach ((int cell, int itemId, int count) in stacks)
        {
            if (cell ≥ cellCount || count < 1)
                throw new InvalidDataException($"grid cell {cell} x{count} outside {cellCount} cells");
            while (at < cell)
            {
                grid.Add(Hole, 1, 1);
                sentinels++;
                at++;
            }
            grid.Add(itemId, count, CampaignCatalog.Item(itemId).MaxStack);
            at++;
        }
        while (grid.Cells.Count < cellCount)
        {
            grid.Add(Hole, InventoryGrid.CellsPerPage, 1);
            sentinels += InventoryGrid.CellsPerPage;
        }
        if (sentinels > 0)
            grid.Take(Hole, sentinels);
        return grid;
    }
```

listing 38: the grid rebuilds through the public add and take, sentinel holes first, exact for every layout the game produces (source: game-systems/capstone/src/Persistence/SaveStore.cs, lines 240-268)

The round trip tests play a scripted campaign, hat bought and equipped so a hole stays behind, boss cleared, egg hatched, weapon upgraded, save, load, and assert the two campaigns agree cell for cell, pet for pet, counter for counter, and on the state hash, which covers the grid layout. A pinned bundle keeps the engine at sqlite 3.53.4, asserted by `select sqlite_version()` in the suite.

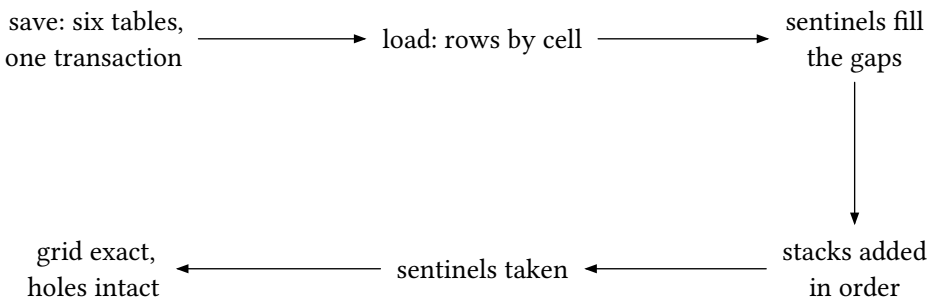



figure 29: the save as one transaction, the grid rebuilt through add and take

sources: length prefixed binary framing and magic plus version discipline follow conventional protocol practice, cross checked against System.Buffers.Binary.BinaryPrimitives on learn.microsoft.com for the exact size span contract that bit this chapter's first draft, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 6 tests in `GameSystems.Samples.Tests.Ch12`.

# 13. testing games

Games resist testing for one structural reason: the interesting behavior is a thousand step conversation between systems, and the habitual unit of tests, one function with inputs and an expected output, is the wrong shape. This chapter builds the three tools that fix the shape, a state hash, a headless harness, and a counting harness, and every one of them exists because a chapter before it paid for it. The hash works because state is raw words, the headless run works because the sim needs no window, and the counters work because systems are named loops.

## 13.1. the state hash

```
public static class SimHash
{
    public static ulong Of(params long[] raws)
    {
        ulong h = 0xcbf29ce484222325UL;
        foreach (long r in raws)
        {
            h ^= (ulong)r;
            h *= 0x100000001b3UL;
        }
        h = (h ^ (h >> 30)) * 0xBF58476D1CE4E5B9UL;
        h = (h ^ (h >> 27)) * 0x94D049BB133111EBUL;
        return h ^ (h >> 31);
    }

    public static ulong Of(Fp[] values) => Of(values.Select(v => v.Raw).ToArray());
}
```

listing 39: fnv over raw words, splitmix finalizer, avalanche by construction (source: game-systems/samples/src/Ch13/Harness.cs, lines 11-27)

`SimHash` folds the raw words of game state into one `ulong`. The fnv-1a core hashes, the splitmix finalizer avalanches, and the finalizer is not optional decoration: raw fnv moves few bits per input bit, so two nearly identical states could collide in their high bits and a determinism test would pass on luck. The avalanche test measures the property directly, flip one input bit at each of sixty four positions and count the output bits that move, and the average must sit between 24 and 40, the neighborhoods of half. Because the inputs are fixed the assertion is not a statistical flake, it is either true of this hash or the hash is broken.

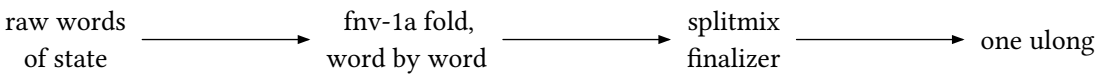



figure 30: raw words to one hash, one input bit moves half the output

## 13.2. determinism as an assertion

The headless flight test is the template for every game test that follows. A terrain from a seed, a wind drawn from a named stream, a shell integrated until impact, all constructed from the same chapters the capstone composes, run twice inside one test process, and the impact points compared as raw words. No rendering, no window, no timing dependence, and a failure means something concrete: some draw crossed the simulation's skin. The capstone's replay suite is this test scaled to whole matches, seed and turn inputs in, final state hash out, twice.

```
/// The counting harness: systems tick counters instead of a profiler,
/// so a test can assert "physics ran 600 times for 10 seconds of match"
/// without attaching anything.
/// </summary>
public sealed class Counters
{
    private readonly Dictionary<string, long> _counts = new();

    public void Increment(string system) =>
        _counts[system] = _counts.GetValueOrDefault(system) + 1;

    public IReadOnlyDictionary<string, long> Snapshot() => _counts;
```

listing 40: systems tick counters, tests assert the counts (source: game-systems/samples/src/Ch13/Harness.cs, lines 30-41)

The counting harness replaces the profiler for the questions tests actually ask. “Did the physics system run exactly 600 times for 10 seconds of match” is an assertion about wiring, that the fixed step clock feeds the system loop, and it catches the bug class where a system silently stops being called after a state change. Chapter 4 deferred measuring to the profiler and chapter 7 promised the cordic loop was cheap, this is the tool that turns both promises into test facts.

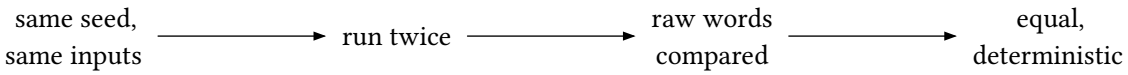



figure 31: determinism as assertions, run twice, compare words, count ticks

**NOTE: what game tests look like**

The suite that results has three layers. Unit tests over the pure pieces, the fixed point arithmetic, the codec, the turn queue, which are ordinary and most of the count. System tests over one subsystem with its harness, terrain determinism, flight reproducibility, counters. And scenario tests, whole matches with scripted inputs asserting invariants, someone always wins within the turn cap, no health below zero, the replay hash equals the live hash. The capstone ships all three.

sources: avalanche measurement follows the standard strict avalanche criterion treatment, fnv-1a and splitmix64 constants already pinned in chapters 9 and 11, accessed 2026-09-08. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 6 tests in `GameSystems.Samples.Tests.Ch13`.

# 14. characters and progression

The artillery game so far has two tanks and nothing to lose. The trajectory shooters this book's capstone draws its progression from, the DDTank lineage, put a character at the center: a level, primary attributes, gear, a bag, and one screen that shows all of it. This chapter builds that character as plain data and pure functions, integers only, because a character that can drift cannot be replayed.

## 14.1. four primaries, four elements

```
public enum Element { Earth, Water, Fire, Wind }

/// <summary>How many elements an elemental array carries, one per Element.</summary>
public static class ElementCount
{
    public const int Value = 4;
}

/// <summary>
/// The four primary attributes. Offense raises every damage number and crit
/// damage, defense raises health, armor percent and all resistances, agility
/// raises stamina regen and turn initiative, luck raises crit chance and
/// dampens the wind drawn on your turn.
/// </summary>
public sealed record PrimaryBlock(int Offense, int Defense, int Agility, int Luck)
{
    public static PrimaryBlock Zero { get; } = new(0, 0, 0, 0);

    public static PrimaryBlock operator +(PrimaryBlock a, PrimaryBlock b) =>
        new(a.Offense + b.Offense, a.Defense + b.Defense,
            a.Agility + b.Agility, a.Luck + b.Luck);
}

/// <summary>
/// What the 14 equipment slots contribute together. The raw fields are
/// exclusive by source, physical damage only from the weapon, raw elemental
/// damage only from the offhand, raw armor only from the seven armor pieces,
/// percent elemental damage only from the ring pair, percent resistances only
```

listing 41: the four elements every effect speaks, and the four primaries every stat derives from (source: game-systems/samples/src/Ch14/Attributes.cs, lines 4-31)

`PrimaryBlock` is four integers and nothing else, with component wise addition as its only operator, which is all composing bonuses ever needs. Offense raises every damage number and crit damage, defense raises health, armor percent, and all resistances, agility raises stamina regen and the turn order, luck raises crit chance and dampens the wind drawn on your turn. The `Element` enum fixes the vocabulary every weapon, resistance, and effect shares, so earth is the same integer in the catalog, the pipeline, and the save.

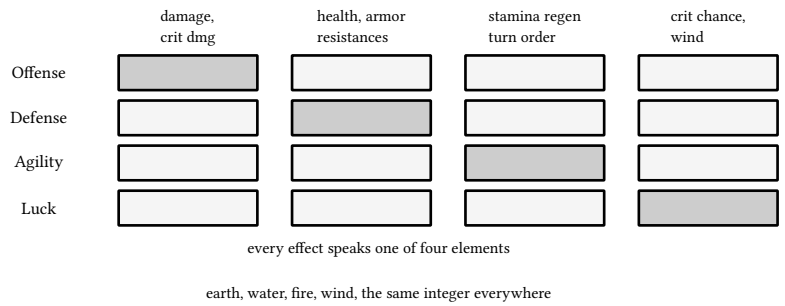



figure 32: which primary moves which number, the four by four map

## 14.2. growth as arithmetic

```
            @base.Vitality + GrowthOf(@base.Vitality) * steps,
            @base.Focus + GrowthOf(@base.Focus) * steps);
    }

    // half the stat per level, minimum one, and a zero stat stays zero:
    // a stat the base does not have must not grow from nothing
    private static int GrowthOf(int stat) => stat == 0 ? 0 : Math.Max(1, stat / 2);

    /// <summary>The same growth for the primary attributes of the v2 sheet.</summary>
    public static PrimaryBlock AtLevel(PrimaryBlock @base, int level)
    {
        ArgumentOutOfRangeException.ThrowIfLessThan(level, 1);
        int steps = level - 1;
        return new PrimaryBlock(
            @base.Offense + GrowthOf(@base.Offense) * steps,
            @base.Defense + GrowthOf(@base.Defense) * steps,
            @base.Agility + GrowthOf(@base.Agility) * steps,
            @base.Luck + GrowthOf(@base.Luck) * steps);
    }

    public static int XpForNext(int level) =>
        20 * level * level;

    public static (CharacterSheet Sheet, bool Leveled) GainXp(CharacterSheet sheet, int xp)
    {
        int level = sheet.Level;
        int total = sheet.Xp + xp;
        while (total >= XpForNext(level))
        {
            total -= XpForNext(level);
```

listing 42: stats scale off the base, xp escalates quadratically, one award can cross levels (source: game-systems/samples/src/Ch14/Characters.cs, lines 33-62)

Growth is derived, never stored: `AtLevel` scales the base sheet by half the base per stat per level, minimum one so no stat freezes at an even base, and a zero stat stays zero, a stat the base does not have must not grow from nothing. The same rule serves the legacy stat block and the primaries. The xp requirement is $20 \times \text{level}^2$. The loop in `GainXp` is the part worth noticing, one award may cross several levels in a single call because the expedition boss payout in the capstone is large enough to do exactly that. Storing grown stats instead of deriving them would invite the classic bug, a rebalance that leaves saved characters frozen at the old curve. Here the curve lives in the binary and the sheet holds only level and xp.

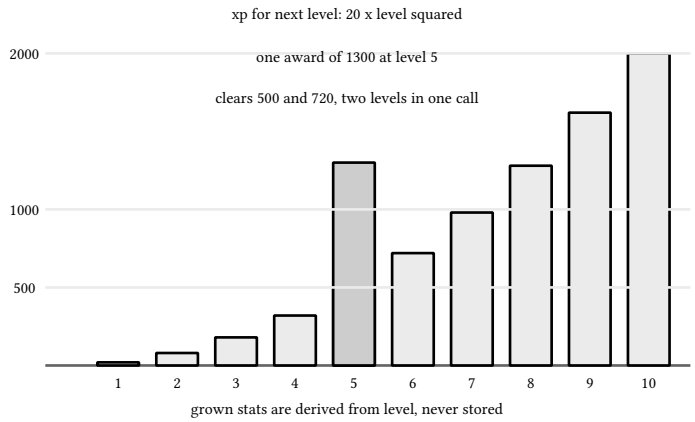



figure 33: the quadratic xp curve, one award crossing two levels

## 14.3. effective stats as one pure function

```
    public static DerivedStats Derive(
        PrimaryBlock @base, int level, EquipmentContrib equipment, PetContrib pet)
    {
        PrimaryBlock p = Growth.AtLevel(@base, level) + equipment.Inherent + pet.Inherent;
        int maxHealth = 100 + 8 * p.Defense;
        // raw stamina only comes from level ups, 100 at level 1
        int maxStamina = 100 + 5 * (level - 1);
        int[] elemRes = new int[ElementCount.Value];
        for (int e = 0; e < ElementCount.Value; e++)
            elemRes[e] = Math.Min(ElemResCap,
                At(equipment.ElemResPct, e) + At(pet.ElemResPct, e) + p.Defense);
        int primaries = p.Offense + p.Defense + p.Agility + p.Luck;
        int elemDamage = equipment.ElemDamage.Sum();
        int power = equipment.PhysicalDamage + equipment.Armor
            + maxStamina / 10 + pet.Mana / 10 + maxHealth / 5
            + elemDamage + elemRes.Sum() + 3 * primaries;
        var elemDamageOut = new int[ElementCount.Value];
        var elemDmgPctOut = new int[ElementCount.Value];
        for (int e = 0; e < ElementCount.Value; e++)
        {
            elemDamageOut[e] = At(equipment.ElemDamage, e);
            elemDmgPctOut[e] = At(equipment.ElemDmgPct, e);
        }
        return new DerivedStats(
            p,
            equipment.PhysicalDamage,
            equipment.Armor,
            maxHealth,
            maxStamina,
            pet.Mana,
            elemDamageOut,
            elemDmgPctOut,
            elemRes,
            CritChancePct: 5 + p.Luck,
            CritMultPct: 150 + 5 * p.Offense,
            DmgPct: 5 * p.Offense,
            ArmorPct: 2 * p.Defense,
            AllResPct: p.Defense,
            power);
    }

    // elemental arrays may arrive shorter than four, an absent element is 0
    private static int At(int[] values, int index) => index < values.Length ? values[index] : 0;

    internal static int[] Add(int[] a, int[] b)
    {
        var sum = new int[ElementCount.Value];
```

listing 43: derive folds growth, gear, and pet into every number the battle reads (source: game-systems/samples/src/Ch14/Attributes.cs, lines 96-142)

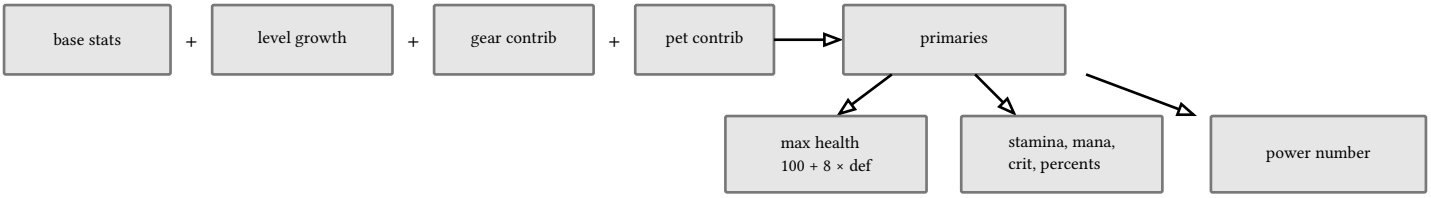



figure 34: the composition stack, growth and bonuses fold into the stats the battle reads

The battle engine never reads the sheet directly. `Derive` folds level growth, the summed gear contribution, and the active pet's passives into the primaries, then derives everything combat consumes: max health from defense, stamina from level only, mana from the pet only, elemental resistances capped at 75, crit chance from luck, crit damage and all damage percents from offense, and the one power number that stands for the whole sheet. The raw sources stay exclusive by construction: physical damage only arrives from a weapon, raw elements only from the offhand, armor only from garments, and the percents only from rings, the amulet, and the medal, enforced by the item catalog never setting an illegal field. Deriving beats storing twice over, the same sheet always composes to the same answer, and a bonus changing at the source reaches the battle with no cache to invalidate.

## 14.4. the save file

```
public static class ProfileCodec
{
    private static readonly byte[] Magic = "GSCH"u8.ToArray();
    private const byte Version = 2;
    private const int SlotCount = 14;

    // level, xp, pet, the 4 primaries, 14 slots, 14 upgrades, wallet, count
    private const int TailBytes = 3 * 4 + 4 * 4 + SlotCount * 4 + SlotCount * 4 + 8 + 4;

    public static byte[] Save(CharacterSheet sheet)
    {
        byte[] name = Encoding.UTF8.GetBytes(sheet.Name);
        int cells = sheet.Grid.Cells.Count;
        // 4 magic, 1 version, 2 name length, name, the tail, 8 per grid cell
        var bytes = new byte[7 + name.Length + TailBytes + cells * 8];
        Magic.CopyTo(bytes, 0);
        bytes[4] = Version;
        BinaryPrimitives.WriteUInt16BigEndian(bytes.AsSpan(5, 2), (ushort)name.Length);
        name.CopyTo(bytes, 7);
        int at = 7 + name.Length;
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at, 4), sheet.Level);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 4, 4), sheet.Xp);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 8, 4), sheet.ActivePet);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 12, 4), sheet.Base.Offense);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 16, 4), sheet.Base.Defense);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 20, 4), sheet.Base.Agility);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 24, 4), sheet.Base.Luck);
        at += 28;
        foreach (SlotId slot in Slots.All)
            BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + (int)slot * 4, 4),
                sheet.Loadout[slot] ?? 0);
        at += SlotCount * 4;
        for (int i = 0; i < SlotCount; i++)
            BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + i * 4, 4), sheet.Upgrades[i]);
        at += SlotCount * 4;
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at, 4), sheet.Gold);
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 4, 4), sheet.Diamonds);
        at += 8;
        BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at, 4), cells);
        at += 4;
        foreach (GridStack? cell in sheet.Grid.Cells)
        {
            BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at, 4), cell?.ItemId ?? 0);
            BinaryPrimitives.WriteInt32BigEndian(bytes.AsSpan(at + 4, 4), cell?.Count ?? 0);
            at += 8;
        }
        return bytes;
```

listing 44: magic, version, length prefixed fields, the whole v2 sheet in one flat array (source: game-systems/samples/src/Ch15/Sheet.cs, lines 36-82)

The profile codec is chapter 12's discipline pointed at a character instead of a match: magic first, then version, then length prefixed and fixed width fields, big endian throughout. The v2 sheet carries the loadout's 14 slots, the per slot upgrade levels, the paged grid, and the wallet. `Load` validates before it allocates and every failure names the field that broke, so a truncated file says `grid` instead of handing back a half built character, and a version 1 file

is refused outright, the binary is book-versioned. The tests pin the round trip, the corruption messages, and byte determinism, two saves of one sheet are equal and a single level of difference moves them apart.

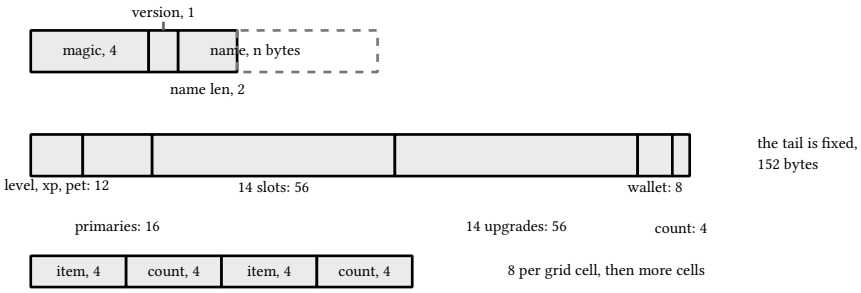



figure 35: the v2 sheet as bytes, name prefix, fixed tail, eight per cell

sources: the character and weapon feature framing follows the ddtank ii description on the dd tank us wiki games page, “fully customizable characters” and “versatile weapons with numerous upgrading options”, accessed 2026-09-09. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 17 tests in `GameSystems.Samples.Tests.Ch14`.

# 15. items, weapons, and crafting

The match so far fires one shell that does one thing. The genre this book's capstone now draws from treats the weapon as the star of the itemization: different weapons reshape the battlefield differently, and the gear a fight drops comes back through upgrades as more of the same piece. This chapter makes the weapon data, the terrain effects compositions, the item stats exclusive by slot, the upgrade curve a pure function, and the forge a pure function.

## 15.1. the weapon as data

```
public enum CarveProfileKind
{
    /// <summary>One crater of the base radius.</summary>
    Shell,
    /// <summary>Narrow and deep: Scale stacked carves of half the radius.</summary>
    Digger,
    /// <summary>Small craters at the impact column and at fixed offsets around it.</summary>
    Cluster,
    /// <summary>One crater of twice the radius: wide and shallow.</summary>
    Heavy,
}

/// <summary>The carve shape plus its scale knob, digger stacks this many carves.</summary>
public sealed record CarveProfile(CarveProfileKind Kind, int Scale);

/// <summary>Linear falloff damage: full at the impact, zero at the radius.</summary>
public sealed record DamageProfile(int MaxDamage, Fp Radius);

/// <summary>
/// A weapon as pure data: its element and the secondary element it splashes,
/// the physical damage profile shared with the crater radius, the carve
/// shape, and the elevation window it can fire in. The catalog lives with
/// the game binary, not the save file.
/// </summary>
public sealed record WeaponDef(
    int Id,
    string Name,
    Element Element,
    Element SecondaryElement,
    int SecondaryBonus,
    DamageProfile Damage,
    CarveProfile Carve,
    int MinAngle,
    int MaxAngle);
```

listing 45: four carve kinds, two profiles, and the elemental weapon record that carries them (source: game-systems/samples/src/Ch15/Items.cs, lines 9-43)

Everything a weapon does travels in one record. The damage profile is a max damage and a radius, the carve profile is a kind plus a scale knob, and the elemental fields say what the weapon is: its element, the secondary element it splashes, and the elevation window it can fire in, the earth ball stops at 55 degrees while the wind bolt flies near vertical. The capstone's four weapons are the four elements, each with 100 base physical damage and a different crater shape. The catalog a game ships is a list of these records, versioned with the binary, never serialized into saves, so adding a weapon is a data change and rebalancing one never invalidates a save file.

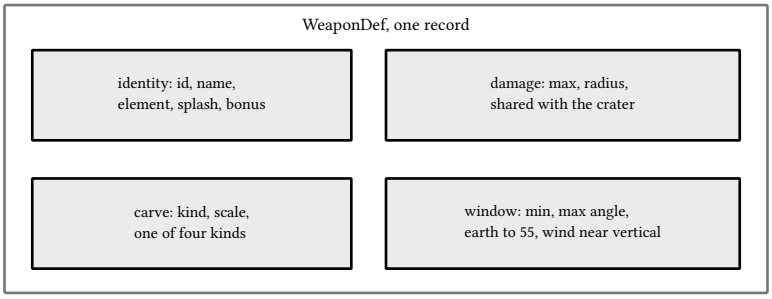



figure 36: the weapon record: identity, two profiles, and a fire window

## 15.2. damage falls off linearly

```
/// <summary>Falloff math shared by battle and tests, integer raw units throughout.</summary>
public static class Damage
{
    public static int Falloff(DamageProfile profile, Fp distance)
    {
        Fp reach = profile.Radius - distance;
        if (reach ≤ Fp.Zero)
            return 0;
        return (int)(reach.Raw * profile.MaxDamage / profile.Radius.Raw);
    }
```

listing 46: full damage at the impact, zero at the radius, integer raw units (source: game-systems/samples/src/Ch15/Items.cs, lines 44-53)

The falloff the artillery match used to own now belongs to the weapon: the reach is the radius minus the horizontal distance, and the damage scales by the fraction of reach remaining. Beyond the radius the answer is exactly zero, which is the boundary the tests pin, and the arithmetic stays in raw fixed point integers so a damage number cannot drift between builds.

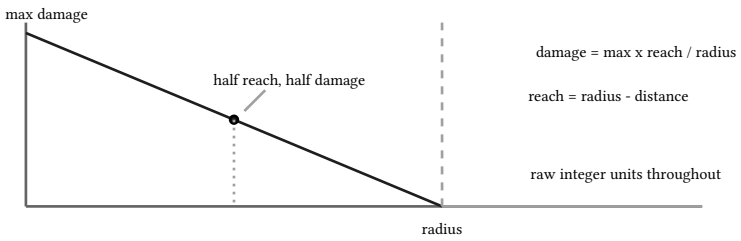



figure 37: linear falloff, full at the impact, exactly zero at the radius

## 15.3. the terrain remembers the weapon type

```
    public static void Apply(Terrain terrain, Fp x, Fp radius, CarveProfile profile)
    {
        switch (profile.Kind)
        {
            case CarveProfileKind.Shell:
                terrain.Carve(x, radius);
                break;
            case CarveProfileKind.Digger:
                Fp narrow = radius / Fp.FromInt(2);
                for (int i = 0; i < profile.Scale; i++)
                    terrain.Carve(x, narrow);
                break;
            case CarveProfileKind.Cluster:
                Fp small = radius / Fp.FromInt(3);
                terrain.Carve(x, small);
                for (int k = 1; k ≤ 2; k++)
                {
                    terrain.Carve(x + Fp.FromInt(k) * radius, small);
                    terrain.Carve(x - Fp.FromInt(k) * radius, small);
                }
                break;
            case CarveProfileKind.Heavy:
                terrain.Carve(x, radius * Fp.FromInt(2));
                break;
        }
    }
}
```

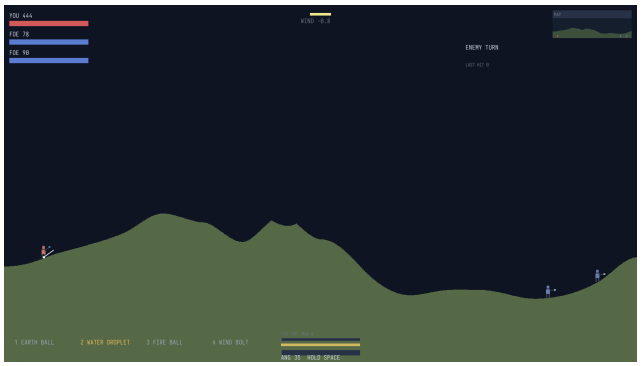
listing 47: each kind is a fixed recipe over the terrain's own carve (source: game-systems/samples/src/Ch15/Items.cs, lines 63-89)



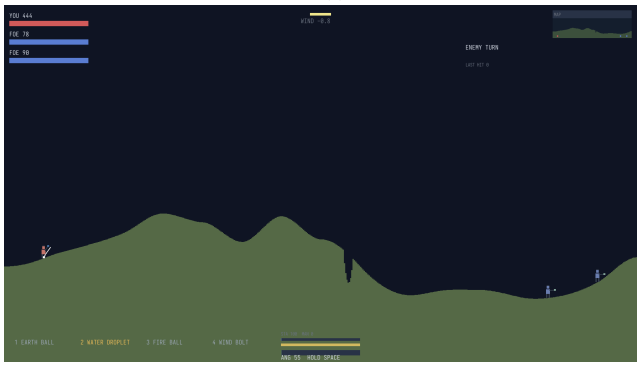

figure 38: the four carve shapes on the same ground, same base radius

No new terrain code exists in this chapter. Each kind is a recipe over the heightfield's own parabolic carve: the shell kind carves once at full radius and belongs to the water droplet, the digger kind stacks half-radius carves at the same column so depth accumulates while width stays narrow and belongs to the fire ball, the cluster kind drops small carves at the impact column and at fixed offsets to each side and belongs to the wind bolt, and the heavy kind carves once at double radius, trading depth per column for reach across them, and belongs to the earth ball. The heightfield never learns that weapons exist, which is the point, terrain deformation composes from one primitive instead of growing a switch per munition.

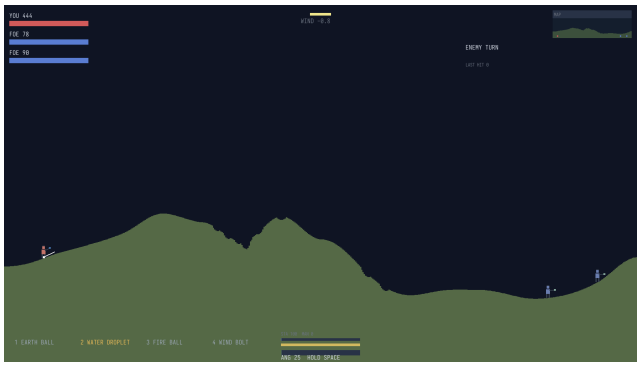
water droplet, shell kind



fire ball, digger kind



wind bolt, cluster kind



earth ball, heavy kind

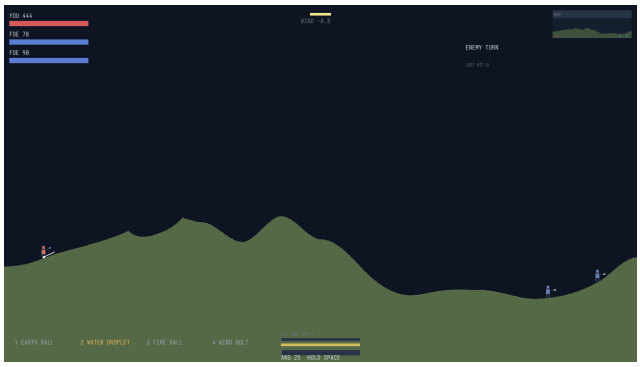


figure 39: the four carve profiles as the mvp renders them, one per element, fired from measured solutions on the seed 42 profile

## 15.4. item stats and where each raw number may come from

```
public enum ItemType { Weapon, Offhand, Garment, Ring, Amulet, Medal, PetEgg, Material }

/// <summary>
/// Which item goes where: an item fits a slot when their kinds match, and
/// eggs and materials fit nowhere. This is the whole legality table, there is
/// no per-item exception.
/// </summary>
public static class GearRules
{
    public static bool Accepts(SlotId slot, ItemType type) => type switch
    {
        ItemType.Weapon => EquipRules.KindOf(slot) is SlotKind.Weapon,
        ItemType.Offhand => EquipRules.KindOf(slot) is SlotKind.Offhand,
        ItemType.Garment => EquipRules.KindOf(slot) is SlotKind.Garment,
        ItemType.Ring => EquipRules.KindOf(slot) is SlotKind.Ring,
        ItemType.Amulet => EquipRules.KindOf(slot) is SlotKind.Amulet,
        ItemType.Medal => EquipRules.KindOf(slot) is SlotKind.Medal,
        _ => false,
    };
}

/// <summary>
/// What one item lends its wielder. The raw fields are exclusive by slot kind
/// and the catalog only ever sets the legal ones: physical damage on weapons,
/// raw elemental damage on offhands, armor on garments, percent elemental
/// damage on rings, percent resistances on the amulet and medal.
/// </summary>
public sealed record ItemStats(
    PrimaryBlock Inherent,
    int PhysicalDamage,
    int Armor,
    int[] ElemDamage,
    int[] ElemDmgPct,
    int[] ElemResPct)
{
    public static ItemStats None { get; } = new(PrimaryBlock.Zero, 0, 0,
        new int[ElementCount.Value], new int[ElementCount.Value], new int[ElementCount.Value]);

    public EquipmentContrib ToContrib() => new(
        Inherent, PhysicalDamage, Armor,
        (int[])ElemDamage.Clone(), (int[])ElemDmgPct.Clone(), (int[])ElemResPct.Clone());
}

/// <summary>One catalog entry; stackability follows the type, gear never stacks.</summary>
public sealed record ItemDef(int Id, string Name, ItemType Type, ItemStats Stats)
{
    public int MaxStack => Type is ItemType.Material or ItemType.PetEgg ? 99 : 1;
}

/// <summary>
/// Diamond upgrades. Every level grows every stat on the item by twenty
/// percent of the previous level, compounding, and the diamond price of level
/// n is 2^n, so the full road to +12 costs 2 + 4 + ... + 4096 diamonds.
```

listing 48: item types, the slot legality table, and the stats record whose raw fields are exclusive by source (source: game-systems/samples/src/Ch15/Gear.cs, lines 10-62)

One legality table decides what fits where, and one stats record decides what a piece lends its wielder. The raw fields are exclusive by slot kind exactly as the stat rules say: physical damage on weapons, raw elemental damage on the offhand, armor on the eight garment slots, percent elemental damage on the ring pair, percent resistances on the amulet and medal, and inherent primaries on everything. The catalog enforces this by never setting an illegal field, so the derivation in chapter 14 can trust its inputs.

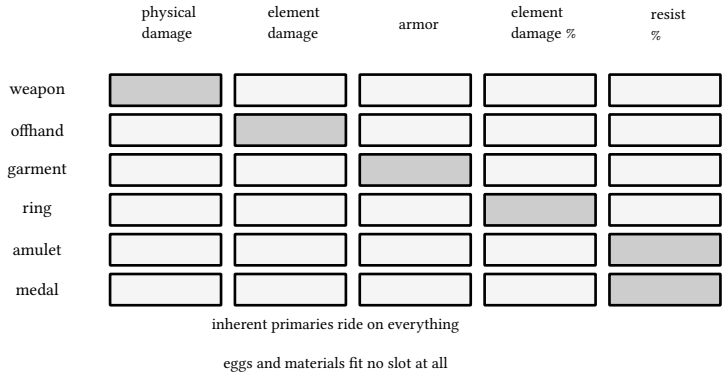



figure 40: which raw field each slot kind may carry, one legality table

## 15.5. upgrades compound, costs double

```
public static class UpgradeMath
{
    public const int MaxLevel = 12;

    public static Fp Multiplier(int level)
    {
        ArgumentOutOfRangeException.ThrowIfLessThan(level, 0);
        ArgumentOutOfRangeException.ThrowIfGreaterThan(level, MaxLevel);
        Fp m = Fp.FromInt(1);
        Fp step = Fp.FromInt(6) / Fp.FromInt(5);
        for (int i = 0; i < level; i++)
            m *= step;
        return m;
    }

    public static int Scale(int stat, int level)
    {
        if (stat == 0)
            return 0; // a stat the item does not have never grows
        Fp scaled = Fp.FromInt(stat) * Multiplier(level);
        // round to nearest: the fixed point 1.2 sits a hair under, and a
        // +1 upgrade printed as 119 out of 100 would read as broken
        return (int)((scaled.Raw + (1L << 31)) >> 32);
    }

    public static int Cost(int targetLevel)
    {
        ArgumentOutOfRangeException.ThrowIfLessThan(targetLevel, 1);
        ArgumentOutOfRangeException.ThrowIfGreaterThan(targetLevel, MaxLevel);
        return 1 << targetLevel;
    }

    public static ItemStats Scale(ItemStats stats, int level) => new(
        new PrimaryBlock(
            Scale(stats.Inherent.Offense, level),
            Scale(stats.Inherent.Defense, level),
            Scale(stats.Inherent.Agility, level),
            Scale(stats.Inherent.Luck, level)),
        Scale(stats.PhysicalDamage, level),
        Scale(stats.Armor, level),
        ScaleAll(stats.ElemDamage, level),
        ScaleAll(stats.ElemDmgPct, level),
        ScaleAll(stats.ElemResPct, level));

    private static int[] ScaleAll(int[] stats, int level)
```

listing 49: each level grows every stat by a fifth of the previous, and the diamond price is 2 to the level (source: game-systems/samples/src/Ch15/Gear.cs, lines 64-108)

Diamond upgrades are fixed point arithmetic in the open: every level scales every stat on the item by a compounding 1.2, rounded to nearest because the fixed point 1.2 sits a hair under and a +1 printed as 119 out of 100 would read as broken, and the price of level n is exactly 2 to the nth diamonds, so the whole road to +12 costs 2 + 4 + ... + 4096. A stat the item does not have never grows, which is the zero stays zero rule again.

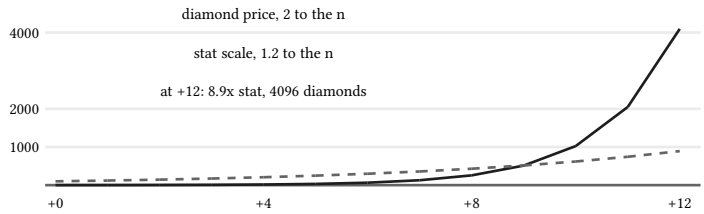


figure 41: the stat scale compounds 1.2, the diamond price doubles

## 15.6. crafting as consume plus roll

```
public sealed record Recipe(
    int Id,
    IReadOnlyList<(int ItemId, int Count)> Inputs,
    int OutputId,
    int OutputCount,
    int SuccessChancePercent);

/// <summary>
/// Crafting as consume plus roll. Inputs are consumed whether the roll
/// succeeds or not, the roll is one draw from a caller supplied stream so
/// determinism is the caller's contract, and the returned inventory keeps
/// the input ordering minus the stacks that hit zero.
/// </summary>
public static class Crafting
{
    public static bool CanCraft(Recipe recipe, IReadOnlyList<(int ItemId, int Count)> inventory) =>
        recipe.Inputs.All(i => Count(inventory, i.ItemId) >= i.Count);

    public static (IReadOnlyList<(int ItemId, int Count)> Inventory, bool Success) Craft(
        Recipe recipe, IReadOnlyList<(int ItemId, int Count)> inventory, Rng roll)
    {
        if (!CanCraft(recipe, inventory))
            throw new InvalidOperationException(
                $"craft {recipe.Id} wants inputs the inventory does not hold");
        var next = new List<(int ItemId, int Count)>();
        foreach ((int itemId, int count) in inventory)
        {
            int take = recipe.Inputs.Where(i => i.ItemId == itemId).Sum(i => i.Count);
            if (count - take > 0)
                next.Add((itemId, count - take));
        }
        bool success = roll.NextInt(100) < recipe.SuccessChancePercent;
```

listing 50: the gate, the consume, and one roll from a caller supplied stream (source: game-systems/samples/src/Ch15/Items.cs, lines 92-123)

`CanCraft` is the gate and `Craft` enforces it with a throw, so there is no silent partial consume. Inputs are consumed whether the roll succeeds or not, the genre's real risk, and the output stacks onto the inventory only on success. The roll is exactly one draw from a stream the caller owns, chapter 11's discipline at its second call site. The capstone's forge is now the diamond counter, but the consume plus roll machinery stays the cleanest teaching shape for stream ownership, and the campaign still forks its `"craft"` stream the day a recipe table returns.

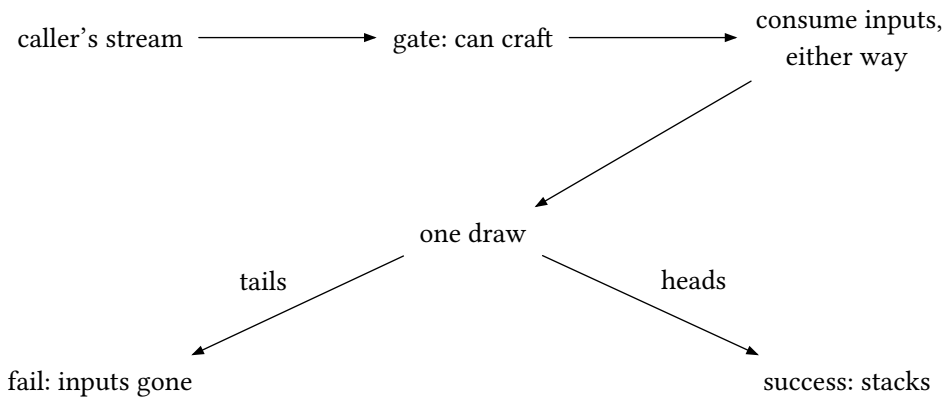



figure 42: crafting: gate, consume either way, one draw, stack on success

**NOTE: why the stream is a parameter**

A `Random.Shared` roll inside `Craft` would make crafting the first nondeterministic system in the game, unreplayable and untestable in the scenario suite. Passing the stream in keeps this chapter's code pure and pushes the ownership decision to the one place that has the seeding policy, the composition root.

sources: the weapon grouping and drop framing follow the dd tank us wiki weapons page, normal and special weapons earned from tasks, events, and rare battle drops, accessed 2026-09-09, and the games page pinned in chapter 14. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 23 tests in `GameSystems.Samples.Tests.Ch15`.

# 16. pets and companions

The trajectory shooters this capstone draws from hatch pets that fight beside the gunner: passives that compose straight onto the character sheet and actives that turn a fight. This chapter keeps both as plain data, three actives and two passives per species, mana gated in the battle, so a pet changes outcomes without introducing a single floating point number.

## 16.1. the species as data

```
public enum PetSkillKind
{
    /// <summary>Heal the owner by Magnitude.</summary>
    Heal,
    /// <summary>Restore the owner's stamina and mana by Magnitude.</summary>
    Restore,
    /// <summary>Raise the owner's armor percent by Magnitude for two turns.</summary>
    ArmorBuff,
    /// <summary>Hit every enemy for Magnitude, no falloff, no crit.</summary>
    DamageBurst,
    /// <summary>Drain every enemy's mana by Magnitude.</summary>
    Debuff,
}

/// <summary>One active skill: its effect, its mana price, and the size of that effect.</summary>
public sealed record PetSkill(PetSkillKind Kind, int ManaCost, int Magnitude);

/// <summary>
/// One innate passive as pure numbers: inherent primaries, raw elemental
/// resistances, and raw mana, the only places those two raw stats exist.
/// </summary>
public sealed record PetPassive(PrimaryBlock Inherent, int[] ElemResPct, int Mana)
{
    public PetContrib ToContrib() =>
        new(Inherent, (int[])ElemResPct.Clone(), Mana);
}

/// <summary>
/// The species as data, v2: exactly three active skills and exactly two
/// innate passives, both counts pinned at construction.
/// </summary>
public sealed record PetSpecies(
    int SpeciesId,
    string Name,
    PetSkill[] Actives,
    PetPassive[] Passives)
{
    public PetSkill[] Actives { get; } =
        Actives.Length == 3 ? Actives
            : throw new ArgumentException(
                $"a pet species carries 3 actives, got {Actives.Length}", nameof(Actives));

    public PetPassive[] Passives { get; } =
        Passives.Length == 2 ? Passives
            : throw new ArgumentException(
                $"a pet species carries 2 passives, got {Passives.Length}", nameof(Passives));

    public PetSkill Skill(int index)
```

listing 51: five skill kinds, the priced skill record, the passive record, and the species pinned at three and two (source: game-systems/samples/src/Ch16/PetSkills.cs, lines 6-53)

A species is exactly three active skills and exactly two innate passives, both counts enforced at construction so no catalog entry can drift. A skill is its kind, its mana price, and the size of its effect, all data, the battle engine dispatches on the kind rather than calling into pet code. A passive is the chapter 14 pet contrib record, inherent

primaries, the only raw elemental resistances in the game, and the only raw mana. An owned pet is its species, a name, and its own level and xp, a small character that never fires.

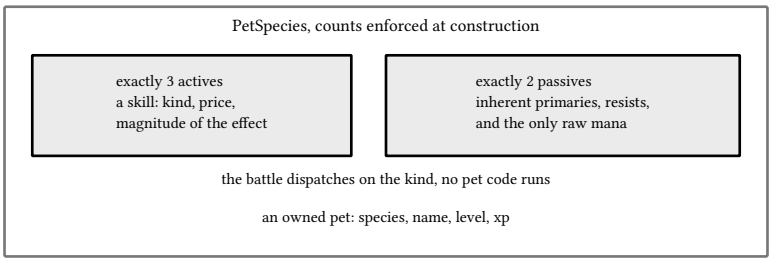



figure 43: the species pinned at three actives and two passives

## 16.2. growth mirrors characters

```
/// arithmetic facts.
/// </summary>
public static class PetGrowth
{
    public static int XpForNext(int level) => 20 * level * level;

    public static (PetInstance Pet, bool Leveled) GainXp(PetInstance pet, int xp)
    {
        int level = pet.Level;
        int total = pet.Xp + xp;
        while (total ≥ XpForNext(level))
        {
            total -= XpForNext(level);
            level++;
        }
```

listing 52: the same xp curve and the same multi level loop (source: game-systems/samples/src/Ch16/Pets.cs, lines 13-27)

`PetGrowth` is chapter 14's curve applied to pets: `XpForNext` and `GainXp` are the same quadratic requirement and the same multi level loop, so pets and their owner level up under one set of arithmetic facts and one rebalance moves both.

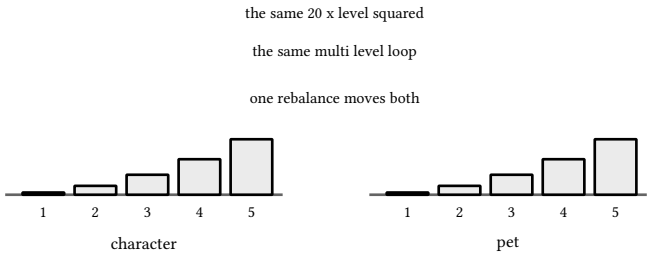



figure 44: the same quadratic curve under characters and pets

## 16.3. hatching from a named stream

```
public static class Hatchery
{
    public static PetInstance Hatch(Rng stream, IReadOnlyList<PetSpecies> pool)
    {
        PetSpecies def = pool[stream.NextInt(pool.Count)];
        return new PetInstance(def.SpeciesId, def.Name, 1, 0);
    }
}
```

listing 53: one NextInt draw picks the species (source: game-systems/samples/src/Ch16/Pets.cs, lines 37-44)

Hatching is deliberately the smallest possible random system: one draw from a caller supplied stream indexes the pool, the species names the pet, and it starts at level one. The stream is a parameter for the same reason crafting's roll was, the campaign forks a `"hatch"` stream once at construction, so the same profile seed hatches the same sequence of pets every time it is replayed, and the scenario suite can pin a boss reward down to the species it hatches.

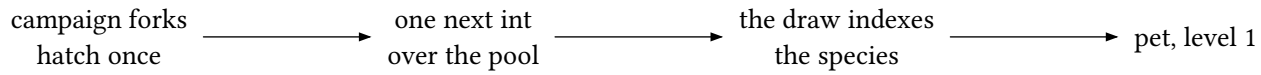



figure 45: hatching: one draw from the forked hatch stream picks the species

## 16.4. mana gates the actives

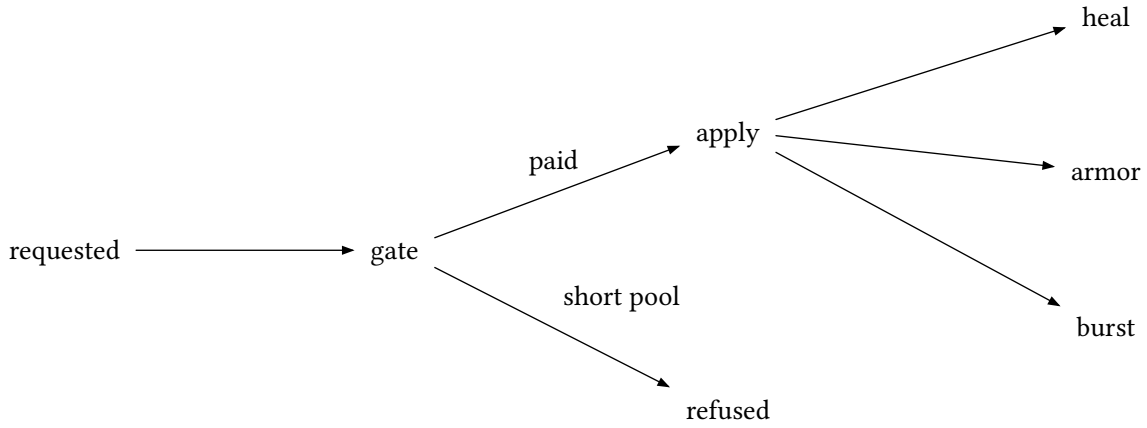



figure 46: a skill request routes through the mana gate to its effect

The once-per-battle bool is gone, mana is the gate now: the pools regen five a turn, the passives decide how deep they run, and a skill that cannot be paid for is refused by the gate rather than by the effects. Heal and restore feed the actor's own pools, armor up rides the same status list as the elemental afflictions for two turns, and burst and debuff hit every living enemy flat with no falloff and no crit. What a pet is worth is decided before the battle by derive, the passives feed the derivation, and inside it by what its actives cost against the pool it brought.

sources: the dd tank us wiki systems section lists pets among its game systems, and the baidu baike dd tank entry describes the pet raising system unlocking at level 25, accessed 2026-09-09. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 8 tests in `GameSystems.Samples.Tests.Ch16`.

# 17. expeditions and bosses

The campaign needs somewhere to spend the sheet. The genre's answer is the expedition: a fixed sequence of staged fights that ends at a boss, pays loot that feeds crafting, and unlocks the next one only when this one is cleared. This chapter makes the stage, the loot table, and the unlock chain data and pure functions, the same discipline as every system before it.

## 17.1. the stage as data

```
public sealed record StageDef(
    string Name,
    int EnemyHealth,
    int EnemyAttack,
    int EnemyDefense,
    int TerrainOctaves,
    Fp TerrainAmplitude,
    int EnemyWeaponId,
    IReadOnlyList<(int ItemId, int Count, int Weight)> Rewards,
    bool IsBoss,
    IReadOnlyList<EnemyDef> Enemies,
    int GoldLo,
    int GoldHi);

/// <summary>An expedition is its ordered stages, the last one is the boss.</summary>
public sealed record ExpeditionDef(int Id, string Name, IReadOnlyList<StageDef> Stages);
```

listing 54: one fight as a record, and the expedition as its ordered stages (source: game-systems/samples/src/Ch17/Expeditions.cs, lines 13-28)

A stage carries everything a battle needs to build itself: the enemy's health, attack, and defense, the terrain's octave and amplitude knobs, the weapon it fires, the weighted rewards it can drop, and the gold a win rolls between. The boss flag changes behavior in the battle engine, a desperation branch in its aim tree and four diamonds on the payout, without changing the stage's shape. An expedition is just the ordered list with the boss last, which is a convention the tests pin rather than a rule the type enforces.

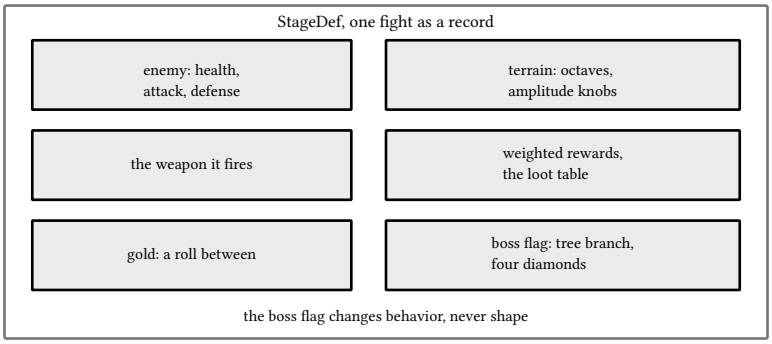



figure 47: the stage record feeding a whole battle build

## 17.2. enemy rosters and the two brains

```
/// <summary>How a stage enemy picks its shots.</summary>
public enum EnemyBrainKind
{
    /// <summary>The deterministic coarse grid of ScriptedFire, a creep.</summary>
    Script,
    /// <summary>The fuzzy behavior tree, a boss.</summary>
    Fuzzy,
}

/// <summary>
/// One enemy in a stage roster: its sheet numbers, its weapon, and its brain.
/// Creeps run the script, bosses run fuzzy, and the campaign turns each into
/// a combatant on its own team.
/// </summary>
public sealed record EnemyDef(
    string Name,
    int Health,
    int Offense,
    int Defense,
    int Agility,
    int Luck,
    int WeaponId,
    EnemyBrainKind Brain);
```

listing 55: an enemy as data with its brain kind, script for creeps, fuzzy for bosses (source: game-systems/samples/src/Ch17/Rosters.cs, lines 6-28)

A stage can also carry a roster, one `EnemyDef` per combatant, each with its own numbers, weapon, and brain. Creeps run the scripted brain, a deterministic coarse grid search with no randomness anywhere, and bosses run the utility tree of chapter 19. The roster field is what lets one stage hold creeps and a boss in the same fight once the campaign consumes it.

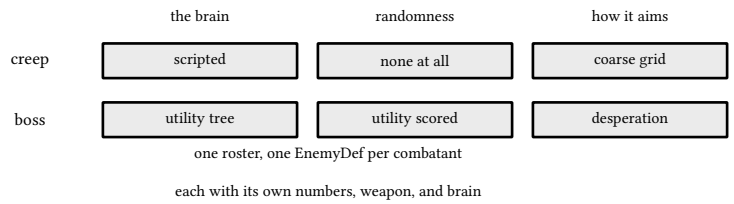



figure 48: creep against boss, the scripted grid against the utility tree

## 17.3. weighted loot from one stream

```
public static class Loot
{
    public static IReadOnlyList<(int ItemId, int Count)> Roll(
        IReadOnlyList<(int ItemId, int Count, int Weight)> table, Rng stream, int draws)
    {
        int total = table.Sum(e => e.Weight);
        if (total == 0 || draws == 0)
            return [];
        var loot = new List<(int ItemId, int Count)>();
        for (int d = 0; d < draws; d++)
        {
            int pick = stream.NextInt(total);
            foreach ((int itemId, int count, int weight) in table)
            {
                if (pick < weight)
                {
                    int at = loot.FindIndex(e => e.ItemId == itemId);
                    if (at < 0)
                        loot.Add((itemId, count));
                    else
                        loot[at] = (itemId, loot[at].Count + count);
                    break;
                }
                pick -= weight;
            }
        }
        return loot;
```

listing 56: one NextInt over the total weight per draw, repeats stack (source: game-systems/samples/src/Ch17/Expeditions.cs, lines 35-61)

Each draw is one `NextInt` over the total weight, then a walk that subtracts weights until the pick falls inside an entry, so a zero weight entry can never win no matter the seed. Repeated picks of the same entry stack their counts into one line of loot. The stream is a parameter again: the campaign forks a `"loot"` stream at construction and every reward in a replayed run lands identically, which is what lets the scenario suite assert a full clear down to the items.

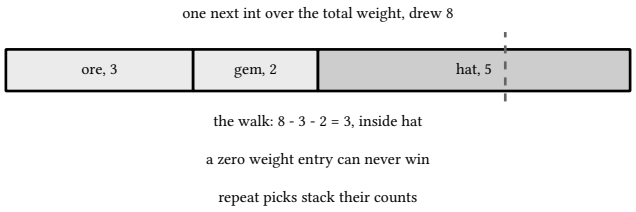



figure 49: one draw over the total weight, the subtracting walk picks

## 17.4. the unlock chain

```
public static class Progress
{
    public static bool Unlocked(IReadOnlyList<bool> clearedPerExpedition, int index)
    {
        ArgumentOutOfRangeException.ThrowIfLessThan(index, 0);
        ArgumentOutOfRangeException.ThrowIfGreaterThanOrEqual(index, clearedPerExpedition.Count);
        return index == 0 || clearedPerExpedition.Take(index).All(cleared => cleared);
    }
```

listing 57: index zero is always open, the rest need all earlier cleared (source: game-systems/samples/src/Ch17/Expeditions.cs, lines 70-78)

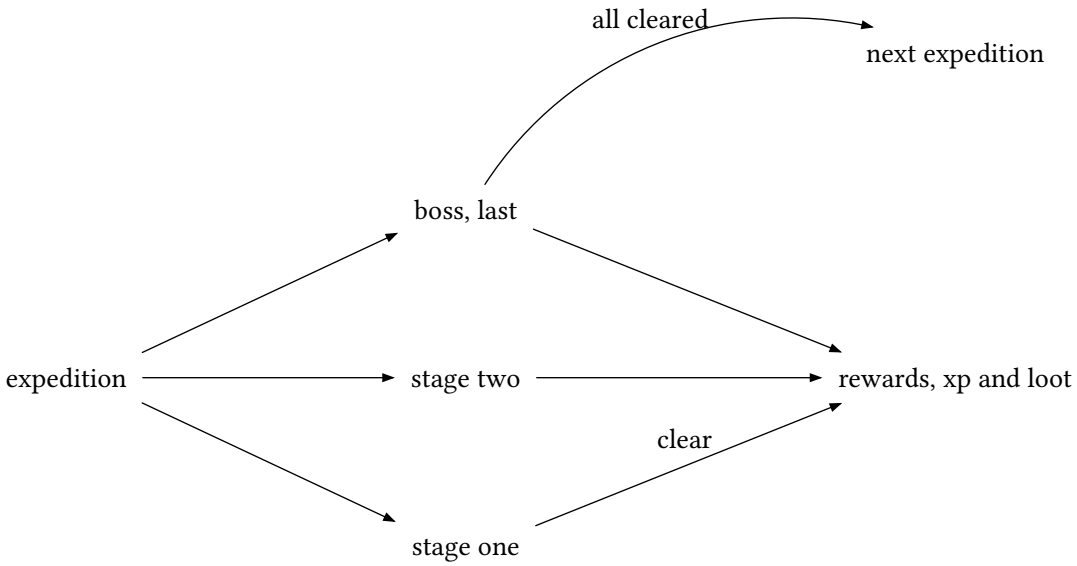



figure 50: an expedition paying out per stage, the boss clearing opens the next

The chain is strict order: the first expedition is always unlocked, every later one needs all earlier ones cleared, and clearing means the boss went down, because completion counts stages and the boss is a stage. Losing a fight keeps the cleared stages, the retry walks back in at the same place, so an unlock is permanent progress rather than a streak. That is the whole persistence story, and it needs no save machinery beyond the profile the character chapter already owns.

sources: the adventure mode framing follows the gamesindustry.biz dd tank overview naming adventure as one of the game's two main modes, accessed 2026-09-09, alongside the pins of chapters 14 through 16. Verified by `dotnet test books/game-systems/samples/GameSystemsBook.slnx`, 12 tests in `GameSystems.Samples.Tests.Ch17`.

# 18. embedded scripting: hosting lua

Seventeen chapters of systems, and every behavior in them is compiled c#. That is the right default: compiled code is fast, typed, and versioned with the binary that shipped it. But a game is also a pile of decisions somebody wants to change without a recompile, the designer tuning enemy weights, the modder rewriting a rule, the map author scripting a boss intro. The industry answer is an embedded script runtime, and this chapter builds a real one: the campaign keeps its engine, its state, and its randomness, lua answers exactly one kind of question, which choice. The chapter's rule runs through every section: scripts own policy, never state.

## 18.1. why games embed a script runtime

The recompile cycle is the whole argument. A designer iterating on how much a boss values desperation waits minutes per change in compiled code and seconds in a script, and a modder never has the build at all. The cost is everything this book spent 17 chapters defending: determinism, replay exactness, the state hash. A script that reads the clock, rolls its own dice, or mutates engine state directly breaks all three. So the embedding draws a hard line. The engine keeps the loop, every byte of state, the named rng streams of chapter 11, and the save file. The script receives facts as plain values, answers with an index or a command, and cannot reach anything else. Inside that line a script is a policy function like any other, and the equivalence proofs later in this chapter hold it to the same standard as the compiled policy it sits beside.

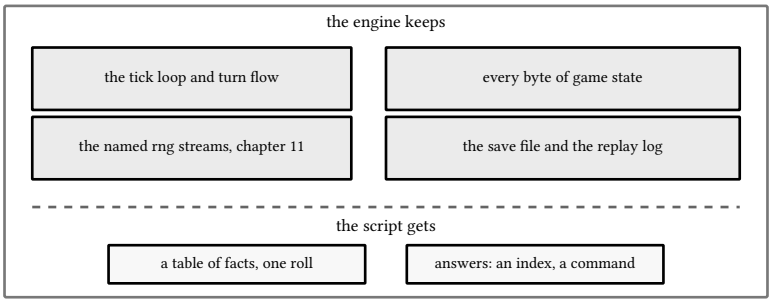



figure 51: the embedding boundary: the engine keeps everything, the script only answers

## 18.2. the host in c#

```
public sealed class LuaHost : IDisposable
{
    private readonly Lua _state = new();

    /// <summary>The raw NLua state, for the sandbox and anything host-side.</summary>
    public Lua State => _state;

    /// <summary>Runs a chunk and returns whatever it returns.</summary>
    public object[] DoString(string chunk) => _state.DoString(chunk);

    /// <summary>Reads one global back out, null when the script never set it.</summary>
    public object? Global(string name) => _state[name];

    /// <summary>Calls a global Lua function with arguments, results and all.</summary>
    public object[] Call(string function, params object[] args) =>
        _state.GetFunction(function)?.Call(args)
            ?? throw new InvalidOperationException(
                $"no lua function named '{function}'");

    /// <summary>Hands a C# function to the script under a global name.</summary>
    public void Register(string name, Delegate function) => _state[name] = function;

    /// <summary>Runs one of the scripts embedded in this assembly, by file name.</summary>
    public object[] RunScript(string file)
    {
        string path = $"{typeof(LuaHost).Namespace}.Scripts.{file}";
        using Stream stream = Assembly.GetExecutingAssembly()
            .GetManifestResourceStream(path)
            ?? throw new FileNotFoundException("no embedded lua script", file);
        using var reader = new StreamReader(stream);
        return DoString(reader.ReadToEnd());
    }
```

listing 58: the smallest useful host: evaluate, read a global, call a function, hand one back (source: game-systems/samples/src/Ch18/LuaHost.cs, lines 12-43)

The host is nlua 1.7.9 from nuget, riding keralua 1.4.9, the minimum version nlua asks for, which embeds lua 5.4 and ships the native runtime as `lua54.dll` under `runtimes/win-x64/native`, with linux and macos libraries beside it, so one package reference runs the same interpreter on all of them. The api surface is four moves: `DoString` evaluates a chunk and returns whatever it returned as an `object[]`, the indexer reads and writes globals, `GetFunction` plus `LuaFunction.Call` invokes a script function with arguments, and assigning a delegate to the indexer hands a c# function to the script. The lua book teaches the other side of this seam, the stack discipline every host rides, in book 9, ch 14 (lua 5.5, the c api, hands on), and closes with a c host of its own over allegro 5 in book 9, ch 18 (lua 5.5, a c host: allegro 5 with lua embedded).

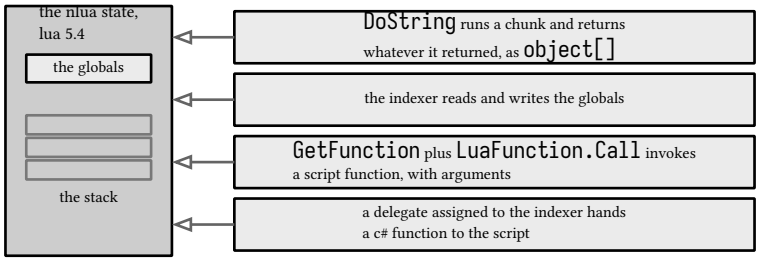



figure 52: the host's four moves over one nlua state, the stack shape the lua book teaches in c

The marshaling edges are measured, not promised. A global the script set to an integer reads back through the indexer as a `double`. The same value returned from a called function arrives as a `long`. Tables come back as `LuaTable`. nlua documents none of this in its readme, so the tests pin it: evaluate `21 * 2`, read the global, call an `add` that returns an integer sum, and register a delegate the script calls with a string. When the capstone needs every number to stay a lua 5.4 integer in both directions, it steps past this object layer entirely, and the last section of this chapter shows why.

## 18.3. the sandbox

```
public static class Sandbox
{
    private static readonly string[] Removed =
        ["io", "os", "debug", "package", "load"];

    /// <summary>Reshapes a fresh state into the sandbox, in place.</summary>
    public static void BuildSandbox(Lua lua)
    {
        foreach (string name in Removed)
            lua[name] = null;
        lua.DoString("math.random, math.randomseed = nil, nil");
    }
}
```

listing 59: the sandbox is removal, not wrapping: nil the shelves, keep arithmetic (source: game-systems/samples/src/Ch18/Sandbox.cs, lines 12-24)

```
local attempts = {
  io = function() return io.open("score.txt") end,
  os = function() return os.time() end,
  debug = function() return debug.getinfo(1) end,
  package = function() return package.loadlib("probe", "*") end,
  load = function() return load("return 1") end,
  random = function() return math.random() end,
}

local report = {}
for name, attempt in pairs(attempts) do
  report[name] = pcall(attempt)
end
return report
```

listing 60: the probe asks each removed library for something real and reports false under pcall (source: game-systems/samples/src/Ch18/Scripts/sandbox_probe.lua, lines 4-17)

The sandbox works by deletion because nothing else works: a script cannot reach what was never granted. `io`, `os`, `debug`, `package`, and `load` go nil, and `math` keeps everything except `random` and `randomseed`, which are removed by name. Randomness is the point of that last cut. Chapter 11 gave the campaign named streams so a replay draws the same futures, and a script calling `math.random` would fork a second, invisible source of chance that no seed reproduces. The probe proves absence the only honest way, by asking each removed library for something real inside `pcall` and asserting every answer fails. The sandbox shape is the one the lua book teaches among its book 9, ch 16 (lua 5.5, idioms), and it is the same shape warcraft iii ships, stripped shelves and all, as the comparison below records.

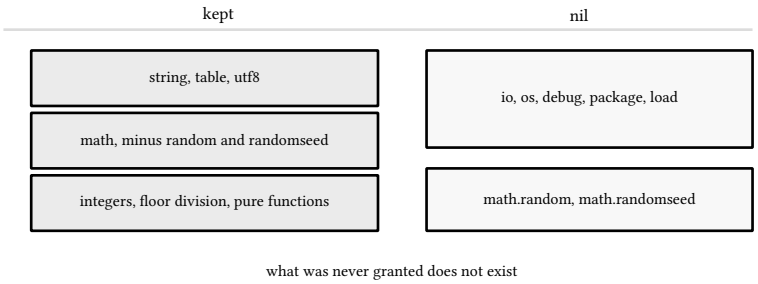



figure 53: what the script sees: arithmetic and tables, nothing that touches the world

## 18.4. events in, decisions out

```
/// </summary>
public sealed class HookLoop : IDisposable
{
    private readonly LuaHost _host = new();
    private readonly Rng _rng;

    public HookLoop(ulong seed)
    {
        _rng = new Rng(seed);
        Sandbox.BuildSandbox(_host.State);
        _host.RunScript("hooks.lua");
    }

    /// <summary>One decision: state and one host roll in, a command out.</summary>
    public string Decide(HookState state)
    {
        int roll = _rng.NextInt(100);
        LuaTable table = (LuaTable)_host.DoString("return {}").Single()!;
        table["turn"] = state.Turn;
        table["health"] = state.Health;
        table["roll"] = roll;
        object[] returned = _host.Call("on_decide", table);
        return (string)returned.Single()!;
    }

    public void Dispose() => _host.Dispose();
}
```

listing 61: the host owns the loop and the rng, the hook is a pure function called once per decision (source: game-systems/samples/src/Ch18/HookLoop.cs, lines 15-41)

```lua
-- the script half of the host-owned loop. on_decide is a pure function:
-- state in, one command string out. the only randomness is state.roll,
-- drawn by the host, because the sandbox has no math.random to call.
function on_decide(state)
  if state.health < 30 and state.roll < 60 then
    return "heal"
  end
  if state.turn < 2 and state.roll < 30 then
    return "power_shot"
  end
  return "attack"
end
```

listing 62: the policy: state and one roll in, one command out, no other source of chance (source: game-systems/samples/src/Ch18/Scripts/hooks.lua, lines 1-12)

The loop never crosses the seam. Per decision the c# side draws one roll from its splitmix64 stream, builds a fresh table of `turn`, `health`, and `roll`, calls `on_decide` exactly once, and applies the command string it gets back. There are no coroutines and no script-side state, so the hook is a function of its arguments and nothing else, which is what makes the replay story survive the embedding: two hosts seeded the same walk the same 25 decisions, and the same seed with different health inputs walks differently. The thresholds in `hooks.lua` are ordinary policy, heal below 30 health when the roll is under 60, open with a power shot early, and editing them changes behavior without touching the engine or any test that does not assert the policy itself.

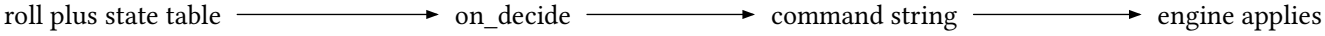



figure 54: one decision, end to end: the host supplies the roll, lua only answers

## 18.5. warcraft iii reforged: lua in the world editor

Warcraft iii added lua to its world editor in patch 1.31, whose official notes carry the line “(Beta) Added Lua as a supported scripting language.” The notes survive as an archived mirror on warcraft.wiki.gg because the original blizzard forum thread did not survive the 2024 forum migration. The patch reached the public test realm on April 24, 2019 and went live May 28, 2019, and the notes are titled as the last major patch for classic before reforged: the reforged line starts at 1.32.0 and shipped January 28, 2020. The language is a per-map either-or, chosen in the editor under `Scenario > Map Options > Script Language`, and the notes say maps save in the selected language, so a map is jass or lua, never both, and a map with custom script must disable its custom triggers before converting.

Around the pinned notes sits a community record, and this chapter labels it as such wherever it leans on it. The embedded version is community-reported as lua 5.3, “a modified version of 5.3.4” in the ptr thread, with no blizzard statement existing either way. The engine’s natives, `CreateUnit`, `GetTriggerUnit`, the `Blz` family, are exposed as plain global lua functions out of the classic `common.j` api, per hive workshop guides whose code calls them directly. The sandbox is community-documented too: `os` is cut down to `clock`, `date`, `time`, and `difftime`, `io`, `file`, `package`, and `debug` are disabled, and the `collectgarbage` functions were removed because influencing the collector could desync a match. Determinism is the same worry this book’s chapter 11 and chapter 12 answer with streams and input logs. Warcraft iii is lockstep multiplayer and every client runs the map script, so the community rules around `GetLocalPlayer` read like a desync catalog: local code must not touch net traffic or shared state, and iterating a table with unordered `pairs` can land differently per client and desync the match. The official notes share the mindset, recording that the editor stopped saving file times to keep maps deterministic.

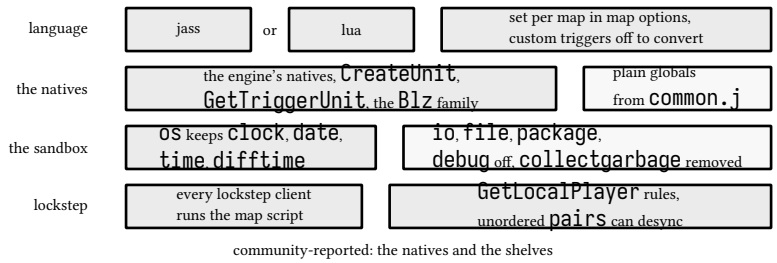



figure 55: warcraft iii: one script language per map, natives as globals, shelves stripped

## 18.6. dota 2: vscript in source 2

Dota 2 custom games run on vscript, and the valve wiki states the shape plainly: "Scripting in Dota 2 is handled by the VScript virtual machine using the Lua programming language," launched at run time when the game loads the addon. No valve page states the lua version. Community tooling is the evidence: the moddota typescript addon template compiles through typescript-to-lua with `"luaTarget": "JIT"`, a 5.1 semantics target, so this chapter says lua 5.1-family and stops there. The unit of authoring is the addon, built in the free workshop tools. Game logic lives under `scripts/vscripts` with `addon_game_mode.lua` as the entry point, whose `Activate` function the game calls on load. From there the flow is event driven: `ListenToGameEvent` takes an event name and a function and the game calls that function every time the event happens, and community code wraps the handler in `Dynamic_Wrap` so the `script_reload` console command can rebind listeners without a restart. Behavior attaches through registration calls like `LinkLuaModifier(className, fileName, LuaModifierType)`, which links a lua-defined modifier class to its file. The ui is a different stack entirely: panorama is valve's html5, css, and javascript framework living under `content/dota_addons/<name>/panorama`, so a custom game is lua logic on the game side and web tech on the screen side. Source 2 itself arrived with the reborn beta in 2015, "Dota 2 is now powered by the Source 2 engine," and the scripting model rode across.

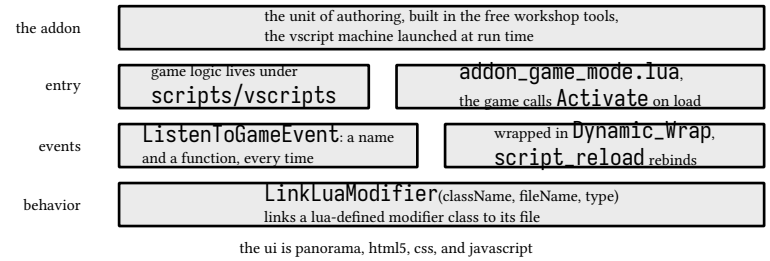



figure 56: dota 2 vscript: the addon's entry point, its event listeners, the linked modifier classes

## 18.7. our host against theirs

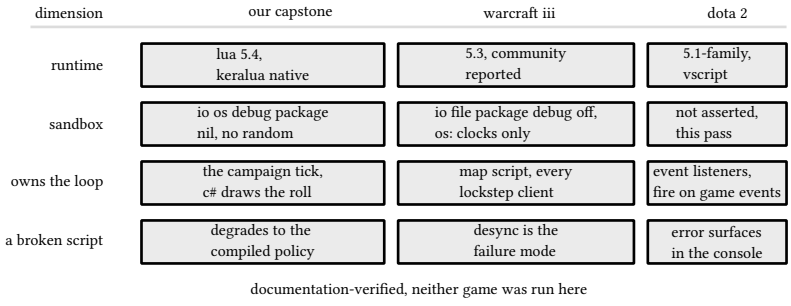



figure 57: three hosts mapped: runtime, sandbox, loop ownership, failure mode

The map is the payoff of the two surveys. All three hosts draw the same boundary this chapter drew, state and randomness stay with the engine and the script answers questions, but they draw it at different depths. Warcraft iii hands the script the whole map, runs it on every lockstep client, and polices determinism with desync rules because a script gone wrong does not throw, it silently diverges the match. Dota 2 keeps the script inside event

listeners and registration calls with a web-tech ui beside it. Ours is the smallest of the three and the only one where a broken script is a non-event: the host degrades to the compiled policy and the battle finishes. That guarantee is not luck, it is the fallback latch the next section builds.

**NOTE: documentation-verified, neither game was run**

Neither warcraft iii reforged nor dota 2 was installed or run for this chapter. Every claim about them is read from vendor pages (the archived 1.31 patch notes, the valve developer wiki, the blizzard release announcement) or labeled community-reported from hive workshop threads, jassdoc, and the moddota template, all accessed 2026-09-23 and pinned in the coverage sources with urls. The comparison maps what those documents state. No number in this book was measured against a live copy of either game.

## 18.8. the capstone wires it in

```
namespace GameSystems.Capstone;

/// <summary>
/// The campaign's engine options: scripts on loads the lua policies
/// for the enemy sweep and the pet's skill choice, scripts off runs
/// the compiled brains only. Both roads are behavior-identical by
/// construction and pinned by the equivalence suite.
/// </summary>
public sealed record CampaignOptions(bool Scripts = true);
```

listing 63: one switch: scripts on loads the lua policies, off runs the compiled brains (source: game-systems/capstone/src/CampaignOptions.cs, lines 1-9)

The campaign takes options beside its seed. Scripts on, the default, builds one script host for the campaign's life and hangs two brains off it, the enemy sweep's scorer and the pet's skill chooser. Scripts off constructs the compiled brains directly, and the seed-only constructor every existing call site uses means the whole golden corpus of chapters 19 through 22 runs unmodified either way. The player side never crosses the seam in either world: the golden hand stays the compiled water-constrained policy, so equivalence tests isolate exactly the enemy brain and the pet choice.

```
public sealed class ScriptHost : IDisposable
{
    private const string SandboxSource = """
        io = nil
        os = nil
        debug = nil
        package = nil
        load = nil
        math.random = nil
        """;

    private readonly NLua.Lua _lua = new();
    private readonly Func<string, string?> _loader;
    private bool _fallback;

    /// <summary>True once a script failed: every call now degrades to compiled policy.</summary>
    public bool Fallback => _fallback;

    public ScriptHost(Func<string, string?>? scriptLoader = null)
    {
        _loader = scriptLoader ?? LoadEmbedded;
        try
        {
            _lua.DoString(SandboxSource);
            Load("pet_brain");
            Load("ai_weights");
        }
        catch (Exception)
        {
            _fallback = true;
        }
    }
```

listing 64: one interpreter per campaign, the two scripts embedded as resources, the fallback latch (source: game-systems/capstone/src/Scripting/ScriptHost.cs, lines 16-47)

```
    /// <summary>
    /// One batched call: hook(table, scalar), every value an integer
    /// both ways, so lua 5.4 integer math is the only math the scripts
    /// ever see. Answers the integer the hook returned, or null when
    /// the host is in fallback or the hook misbehaved; the stack is
    /// restored either way.
    /// </summary>
    public long? Call(string hook, ReadOnlySpan<long> table, long scalar)
    {
        if (_fallback)
            return null;
        Lua? state = _lua.State;
        if (state is null)
        {
            // the runtime is gone: degrade, never throw
            _fallback = true;
            return null;
        }
        int top = state.GetTop();
        try
        {
            if (state.GetGlobal(hook) != LuaType.Function)
                throw new InvalidOperationException($"hook {hook} is not a function");
            state.CreateTable(table.Length, 0);
            for (int i = 0; i < table.Length; i++)
            {
                state.PushInteger(table[i]);
                state.RawSetInteger(-2, i + 1);
            }
            state.PushInteger(scalar);
            if (state.PCall(2, 1, 0) != LuaStatus.OK
                || state.Type(-1) != LuaType.Number
                || !state.IsInteger(-1))
                throw new InvalidOperationException($"hook {hook} misbehaved");
            return state.ToInteger(-1);
        }
        catch (Exception)
        {
            _fallback = true;
            return null;
        }
        finally
        {
            state.SetTop(top);
        }
    }
```

listing 65: one batched call: a table and a scalar pushed as raw lua 5.4 integers, pcalled, restored (source: game-systems/capstone/src/Scripting/ScriptHost.cs, lines 49-94)

The call deliberately steps under nlua's object layer to keralua's raw state. Measured in a probe during the build: nlua's `Call` marshals a `long[]` argument as userdata, which the script cannot index as a table, so the host pushes the candidate table itself, `CreateTable`, `PushInteger`, `RawSetInteger`, pushes the scalar, and `PCalls` the hook with a type check that demands an integer back. Every value crosses as a lua 5.4 integer in both directions, there is no float to drift and no userdata to surprise, and the `finally` restores the stack top whatever the script did. Any failure latches `Fallback` and every later call answers null: a script that throws, a hook that misbehaves, a script that fails to load,

even a disposed host, whose null-state guards degrade instead of throwing. The brains turn null into the compiled policy, so a broken script costs behavior nothing and never crashes a battle.

```
/// <summary>
/// The scripted policy: one batched lua call per sweep, the flat
/// candidate table scored inside the script, the winning index mapped
/// back here with the score computed by the same compiled formula, so
/// the cross weapon comparison in the tree keeps one yardstick. Any
/// script failure hands the answer to the compiled brain, so a broken
/// script costs behavior nothing.
/// </summary>
public sealed class LuaUtilityBrain(ScriptHost host) : IUtilityBrain
{
    private static readonly CompiledUtilityBrain Fallback = CompiledUtilityBrain.Instance;

    public (int Angle, int Power, int Score)? Pick(IReadOnlyList<ShotCandidate> candidates)
    {
        long[] flat = new long[6 * candidates.Count];
        for (int i = 0; i < candidates.Count; i++)
        {
            ShotCandidate c = candidates[i];
            flat[6 * i] = c.Angle;
            flat[6 * i + 1] = c.Power;
            flat[6 * i + 2] = c.FalloffDamage;
            flat[6 * i + 3] = c.SecondaryBonus;
            flat[6 * i + 4] = c.Offense;
            flat[6 * i + 5] = c.MitigationPercent;
        }
        long? index = host.Call("pick", flat, 0);
        if (index is not (> 0 and ≤ int.MaxValue) || index > candidates.Count)
            return Fallback.Pick(candidates);
        ShotCandidate shot = candidates[(int)index - 1];
        return (shot.Angle, shot.Power, CompiledUtilityBrain.Score(shot));
    }
}
```

listing 66: the seam: candidates flattened six ints each, lua answers the winning index, the score stays compiled (source: game-systems/capstone/src/Scripting/UtilityBrain.cs, lines 49-80)

```lua
-- the sweep's scoring table: how much each piece of a candidate shot
-- weighs. pick(cands) scores every candidate, six integers each
-- (angle, power, falloff damage, secondary bonus, offense, mitigation
-- percent) and answers the 1-based argmax, 0 when there is nothing to
-- score. integer math only: the host marshals lua 5.4 integers both
-- ways, and the division is floor division, which matches c#'s
-- truncating int division because every value is non-negative
local weights = {
    falloff = 1,
    secondary = 1,
    offense = 5,
    mitigation_half = 2,
}

function pick(cands)
    local best = nil
    local best_index = 0
    for i = 1, #cands // 6 do
        local base = 6 * (i - 1)
        local score = weights.falloff * cands[base + 3]
            + weights.secondary * cands[base + 4]
            + weights.offense * cands[base + 5]
            - cands[base + 6] // weights.mitigation_half
        if best == nil or score > best then
            best = score
            best_index = i
        end
    end
    return best_index
end
```

listing 67: the sweep's scoring table in lua: integer math, floor division, the argmax (source: game-systems/capstone/src/Scripts/ai_weights.lua, lines 1-30)

The division of labor keeps one yardstick. Lua scores every candidate with the weights and answers the 1-based index of the best, nothing else, and the c# side maps that index to the candidate and computes the winner's score with the compiled `Score` function, so the behavior tree's argmax never compares a lua-computed number against a compiled one. Ties keep the first candidate in both worlds. The arithmetic is integer end to end, and lua's floor division `//` matches the compiled truncating division because every value in the table is non-negative, the one signedness fact the equivalence suite watches. The ghost flights, the thing that makes the sweep expensive, never leave c#, one batched call per sweep is the entire crossing.

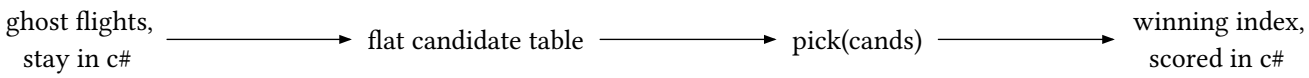



figure 58: the sweep crosses the seam once per decision, the score never leaves

```
    // seeds 42..45, not 42..49: each on/off pair walks twelve battles
    // of ghost-flight sweeps, and the full eight-seed band pushed the
    // capstone suite past its five minute budget while the duel fact
    // already covers the wider seeds at the engine level
    [Fact]
    public void The_golden_battles_hash_identically_with_scripts_on_and_off()
    {
        for (ulong seed = 42; seed ≤ 45; seed++)
        {
            var off = new List<ulong>();
            var on = new List<ulong>();
            using (var compiled = new Campaign(seed, new CampaignOptions(Scripts: false)))
                Walk(compiled, off);
            using (var scripted = new Campaign(seed))
                Walk(scripted, on);
            Assert.True(on.Count > 0, "the walk never saw a battle");
            Assert.Equal(off.Count, on.Count);
            Assert.Equal(off, on);
        }
    }
```

listing 68: the equivalence walk: both expeditions, scripts on and off, per-turn state hashes must match (source: game-systems/capstone/tests/ScriptEquivalenceTests.cs, lines 55-74)

**VERIFY: scripts on, scripts off, one trace**

The equivalence suite walks both expeditions twice per seed, once with scripts on and once off, and asserts the per-turn `Match.StateHash` traces are identical, not the outcomes: campaign seeds 42 through 45, plus mirror duels, both brains scripted against both compiled, on seeds 7, 21, and 42. A full eight-seed band was trimmed for suite budget and the trim is recorded in a comment in `ScriptEquivalenceTests.cs`. The walks demand no wins, deliberately: the compiled water hand loses expedition 1 stage 3 on seeds 44, 46, and 47 and expedition 2 stage 3 on seed 48, a pre-existing compiled-world fact, so the only win assertions live in the seed-42 golden test. The balance matrix and the renderer's bots construct compiled ai only, so the committed balance report is untouched by construction.

sources: the warcraft iii facts cite the archived mirror of the official 1.31 patch notes on warcraft.wiki.gg, the original blizzard forum thread having died in the 2024 migration, and the reforged release announcement on news.blizzard.com, both accessed 2026-09-23. the embedded 5.3 version, the natives-as-globals shape, the stripped library list, and the desync rules are community-reported from hive workshop threads and jassdoc, same date, labeled as such in the text. the dota 2 facts cite developer.valvesoftware.com pages, the scripting page fetched live and the rest through wayback snapshots of a site sitting behind bot protection, plus the moddota typescript template for the 5.1-family tooling evidence, all accessed 2026-09-23. nlua and keralua facts cite nuget, the nlua readme, and the lua 5.4 manual, accessed 2026-09-23, with the marshaling behavior measured by this book's own tests rather than cited. Verified by `make test-gs-samples` and `make test-gs-capstone`: 8 tests in `GameSystems.Samples.Tests.Ch18`, 3 in `ScriptHostTests`, 4 in `ScriptEquivalenceTests`.

# 19. capstone part 1: the battle engine

The eighteen chapters before this one built systems; this chapter assembles the battle that uses them, and this time the assembly is literal. Shells fly as entities in the chapter 3 world, the turn machine is the chapter 5 fsm verbatim, damage and carving come from the chapter 15 weapon table, the pet abilities of chapter 16 gate inside the loop, and every structural change queues and drains between systems the way chapter 3 said it would. The battle is tick stepped: fire launches, then one `Step()` per 60 hertz tick advances wind, flight, impact, and smoke, so a renderer can watch the shell arc under live wind.

## 19.1. components and the machine

```
public readonly record struct Position(Fp X, Fp Y);

public readonly record struct Velocity(Fp Vx, Fp Vy);

/// <summary>The previous tick's position, the source end of the collision segment.</summary>
public readonly record struct Trail(Fp PrevX, Fp PrevY);

/// <summary>What a shell is: the weapon it delivers and who fired it.</summary>
public readonly record struct Payload(int WeaponId, int OwnerId);

/// <summary>Ticks remaining for a particle, decremented by the particle system.</summary>
public readonly record struct Life(int Remaining);

/// <summary>Tag: a particle the wind system should push. Tags carry no data.</summary>
public readonly record struct Ember;

/// <summary>The turn machine, verbatim from the chapter 5 sample tests.</summary>
public enum Phase { Idle, Aiming, Firing, Resolving, GameOver }

/// <summary>Events the turn machine accepts.</summary>
public enum Cmd { Start, Fire, Impact, NextTurn }
```

listing 69: shell components, the ember tag, and the chapter 5 states verbatim (source: game-systems/capstone/src/BattleComponents.cs, lines 13-33)

A shell carries position, velocity, a trail of its previous tick for the collision segment, and a payload naming its weapon and owner. Particles carry position, velocity, and life, and the `Ember` tag marks the ones the wind should push: one marker routes them through the wind system while plain debris falls straight. The phase and command enums are copied name for name from the chapter 5 test machine, so the miniature the fsm chapter taught and the engine this chapter runs are the same object at two scales.

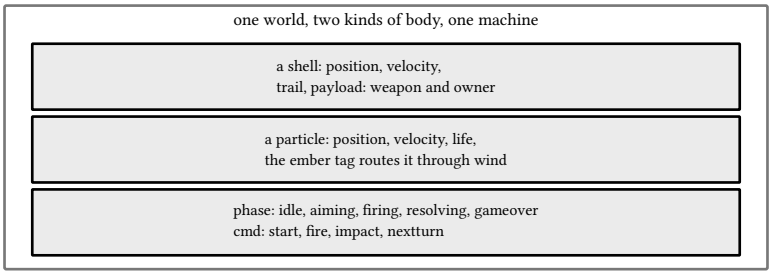



figure 59: the shell, the particle, and the chapter 5 machine verbatim

## 19.2. fire, then the tick owns the flight

```
    public void Fire(int angleDegrees, int power, int weaponId)
    {
        if (Phase ≠ Phase.Aiming)
            throw new InvalidOperationException($"cannot fire while {Phase}");
        WeaponDef weapon = WeaponCatalog.Get(weaponId);
        if (angleDegrees < weapon.MinAngle || angleDegrees > weapon.MaxAngle)
            throw new ArgumentOutOfRangeException(nameof(angleDegrees),
                $"{weapon.Name} fires {weapon.MinAngle} to {weapon.MaxAngle} degrees, got {angleDegrees}");
        CombatantState shooter = CurrentCombatant();
        if (power ≤ 0 || Fp.FromInt(power) > MaxPowerFor(shooter.Id))
            throw new ArgumentOutOfRangeException(nameof(power));

        // negative elevation fires leftward: one signed angle carries
        // direction and elevation and the replay stays four integers
        Fp direction = angleDegrees < 0 ? Fp.FromInt(-1) : Fp.FromInt(1);
        (Fp cos, Fp sin) = Fp.SinCos(Fp.FromInt(Math.Abs(angleDegrees)) * DegreesToRadians);
        Fp v = Fp.FromInt(power);
        Fp launchY = _terrain.HeightAt(shooter.X);

        _spawns.Add(new ShellSpawn(
            shooter.X, launchY, direction * (v * cos), v * sin, weaponId, shooter.Id, false, 0));
        Drain();
        _world.ForEach<Position, Payload>((e, ref pos, ref payload) => _tracked = e);
        _flight.Clear();
        _flight.Add((shooter.X, launchY));
        _shotWind = _currentWind;
        _hasImpact = false;
        _turnDamage = 0;
        _turnImpacts = 0;
        _fsm.Fire(Cmd.Fire);
        _log.Add(new TurnInputV3(
            shooter.Id, angleDegrees, power, weaponId, _turnMoveDm, _turnFlags, _turnTeleportX));
        _turnMoveDm = 0;
        _turnFlags = 0;
        _turnTeleportX = Fp.Zero;
        if (_rallyFor == shooter.Id)
            _rallyFor = 0; // rally lifts exactly one shot
    }
```

listing 70: aim checks, the shell spawn, the fsm event, and the log entry (source: game-systems/capstone/src/Match.Flight.cs, lines 16-53)

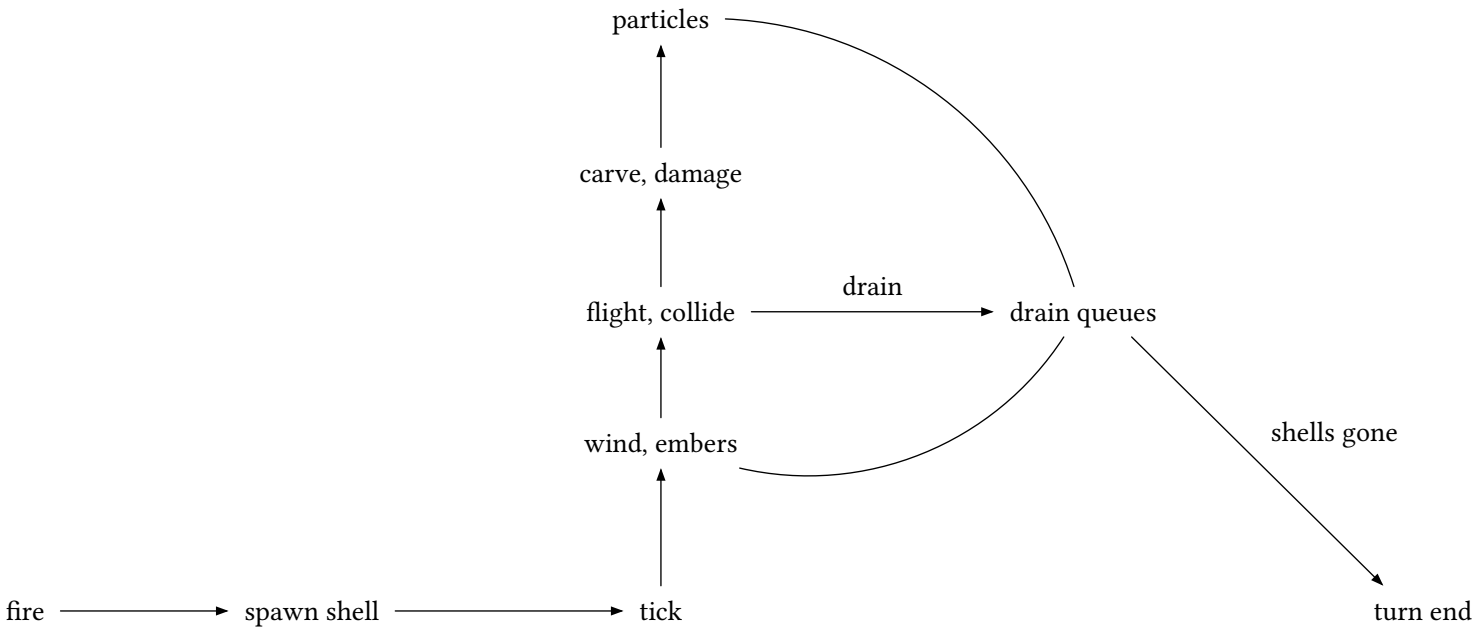



figure 60: one turn through the battle engine's systems

`Fire` validates against the phase, the weapon's own elevation window, and the power ceiling, queues the shell, drains it into existence, fires the fsm's `Fire` event, and appends the turn record the replay needs: shooter, angle, power, weapon, the decimeters walked while aiming, and the flags carrying any paper plane flight. `Step` then runs one tick of the firing phase. The wind system's query is `ForEach<Position, Ember>`: the tag is the routing. The flight system queries `Position` joined with `Payload`, shells and only shells, integrates with the chapter 8 ballistics, and on a hit queues the impact. Impact resolution carves with the weapon's chapter 15 profile, runs the damage pipeline, physical falloff scaled by offense and cut by armor percent, the offhand's raw element channeled through the weapon plus its secondary splash, both cut by resistance, one crit roll per target from a named stream, applies the element's on-hit toll, fire burns for two of the target's turns, water soaks their pools, wind knocks them two meters and grounds their plane, earth carves again scaled by channeled earth damage, queues debris and embers, and a wind bolt queues its five children, all through the deferred lists. The drain applies them between systems, which is the chapter 3 discipline, and the children are why it is not optional: spawning mid iteration would throw. The turn ends when the payload population hits zero, the fsm routes `NextTurn` through its living teams guard, statuses tick at the afflicted's turn start before the machine hands the turn over so a burn death is visible to the guards, the smoke dies with the turn so the world is empty at rest, and the state hash folds terrain, combatants, wind, phase, statuses, and the pet flags.

The engine is no longer a duel. `Match` takes any number of combatants, each on a team, the queue is built once with initiative `40 + 2 * agility` and stable ties by id, a blast spares the shooter's own team, and the match ends when one team remains. During the aiming phase the actor may `Move` along the terrain, two stamina per meter inside a 100 decimeter budget and 8 meters from anyone, and may fly the paper plane `Teleport` once per match for 20 stamina, which ends the walk so a turn is always walk then fly and the replay stays exact.

## 19.3. the weapon table

```
    public const int Earth = 1;
    public const int Water = 2;
    public const int Fire = 3;
    public const int Wind = 4;
    public const int WindChild = 5;

    public static readonly IReadOnlyList<int> Selectable = [Earth, Water, Fire, Wind];

    public static readonly IReadOnlyDictionary<int, WeaponDef> All = new Dictionary<int, WeaponDef>
    {
        // the earth ball bites widest, heavy kind at twice its radius
        [Earth] = new WeaponDef(
            Earth, "earth ball", Element.Earth, Element.Fire, SecondaryBonus: 15,
            new DamageProfile(100, Fp.FromInt(11)),
            new CarveProfile(CarveProfileKind.Heavy, 1),
            MinAngle: -55, MaxAngle: 55),
        // the water droplet splashes one round crater of its full radius
        [Water] = new WeaponDef(
            Water, "water droplet", Element.Water, Element.Wind, SecondaryBonus: 15,
            new DamageProfile(100, Fp.FromInt(10)),
            new CarveProfile(CarveProfileKind.Shell, 1),
            MinAngle: -70, MaxAngle: 70),
        // the fire ball bores a narrow deep shaft, digger kind stacked
        [Fire] = new WeaponDef(
            Fire, "fire ball", Element.Fire, Element.Earth, SecondaryBonus: 15,
            new DamageProfile(100, Fp.FromInt(9)),
            new CarveProfile(CarveProfileKind.Digger, 8),
            MinAngle: -60, MaxAngle: 60),
        // the wind bolt scatters small pocks wide, cluster kind
        [Wind] = new WeaponDef(
            Wind, "wind bolt", Element.Wind, Element.Water, SecondaryBonus: 15,
            new DamageProfile(100, Fp.FromInt(9)),
            new CarveProfile(CarveProfileKind.Cluster, 1),
            MinAngle: -85, MaxAngle: 85),
        [WindChild] = new WeaponDef(
            WindChild, "wind child", Element.Wind, Element.Wind, SecondaryBonus: 0,
            new DamageProfile(40, Fp.FromInt(4)),
            new CarveProfile(CarveProfileKind.Shell, 1),
            MinAngle: -89, MaxAngle: 89),
    };
```

listing 71: four elemental weapons plus the wind child, each its own carve and window, versioned with the binary (source: game-systems/capstone/src/WeaponCatalog.cs, lines 15-54)

The catalog is the balance half of the identity split: the config record carries seed, terrain shape, and power ceiling, everything a replay needs to be itself, while damage numbers and carve shapes ride with the binary. Rebalancing a weapon changes no save file and breaks no replay, the state hash tells the truth about a mismatch, and the wind child exists only as something the wind bolt spawns, never as a player choice. The four weapons are the four elements: the earth ball carves widest, heavy kind at twice its radius, the water droplet splashes one round crater, the fire ball bores a narrow deep shaft, and the wind bolt scatters pocks wide. Each carries 100 base physical damage, its own elevation window, the earth ball stops at 55 degrees while the wind bolt flies near vertical, a secondary element splash, and an on-hit toll that matches its element.

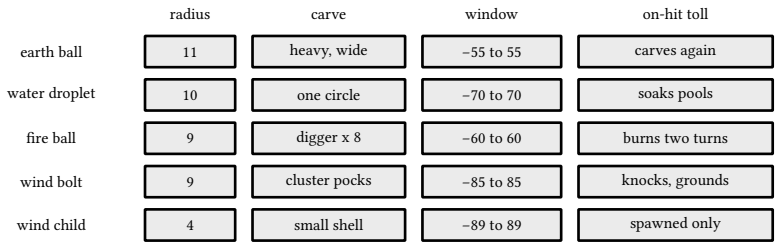



figure 61: four elemental weapons plus the wind child, side by side

## 19.4. the hybrid ai

```
    public AiDecision Decide(Match match, bool isBoss = false, IReadOnlySet<int>? owned = null)
    {
        CombatantState me = match.Combatants.First(c => c.Id == match.CurrentPlayerId);
        // over any number of combatants: the weakest living opponent
        CombatantState target = match.Combatants
            .Where(c => c.Team != me.Team && c.Health > 0)
            .OrderBy(c => c.Health)
            .First();
        Fp distance = target.X - me.X;
        if (distance < Fp.Zero)
            distance = -distance;
        bool Has(int weapon) => owned?.Contains(weapon) ?? weapon == WeaponCatalog.Earth;

        (AiDecision Shot, int Score)? decision = null;
        void Plan(string name, (AiDecision Shot, int Score)? found)
        {
            if (found != null)
            {
                decision = found;
                LastPlan = name;
            }
        }

        var tree = new Selector(
            new Sequence(
                new Condition(() => isBoss && me.Health * 3 ≤ match.Config.StartHealth &&
Has(WeaponCatalog.Earth)),
                new Leaf("desperation earth", () =>
                {
                    Plan("desperation earth", Sweep(match, me, target, WeaponCatalog.Earth));
                    return decision != null ? Status.Success : Status.Failure;
                })),
            new Sequence(
                new Condition(() => Has(WeaponCatalog.Wind) && distance < Fp.FromInt(CloseRange)),
                new Leaf("scatter wind", () =>
                {
                    Plan("scatter wind", Sweep(match, me, target, WeaponCatalog.Wind));
                    return decision != null ? Status.Success : Status.Failure;
                })),
            new Leaf("sweep", () =>
            {
                // the utility leaf owns comparison: the best shot of every
                // owned element, scored by damage, argmax wins
                foreach (int weapon in WeaponCatalog.Selectable)
                {
                    if (!Has(weapon))
                        continue;
                    (AiDecision Shot, int Score)? shot = Sweep(match, me, target, weapon);
                    if (shot != null && (decision == null || shot.Value.Score > decision.Value.Score))
                        Plan("sweep", shot);
                }
                return decision != null ? Status.Success : Status.Failure;
            }));
        tree.Tick();
        return (decision ?? throw new InvalidOperationException("the ai tree produced no decision")).Shot;
    }
```

listing 72: a selector tree over utility leaves, the sweep scores by expected damage (source: game-systems/capstone/src/AiOpponent.cs, lines 31-85)

Chapter 6′s shape exactly: the tree owns priorities and the leaves own comparison. A hurt boss tries desperation earth first, a close fight scatters wind, and otherwise the sweep leaf scores the best shot of every owned element by the damage its falloff would deal where it lands, chapter 15′s curve as the utility function, and fires the argmax. The sweep respects each weapon's own elevation window. `LastPlan` exposes the leaf that won for the tests and this book. The tree is stateless, so the enemy re-solves its whole problem every decision, which is free because aiming is arithmetic.

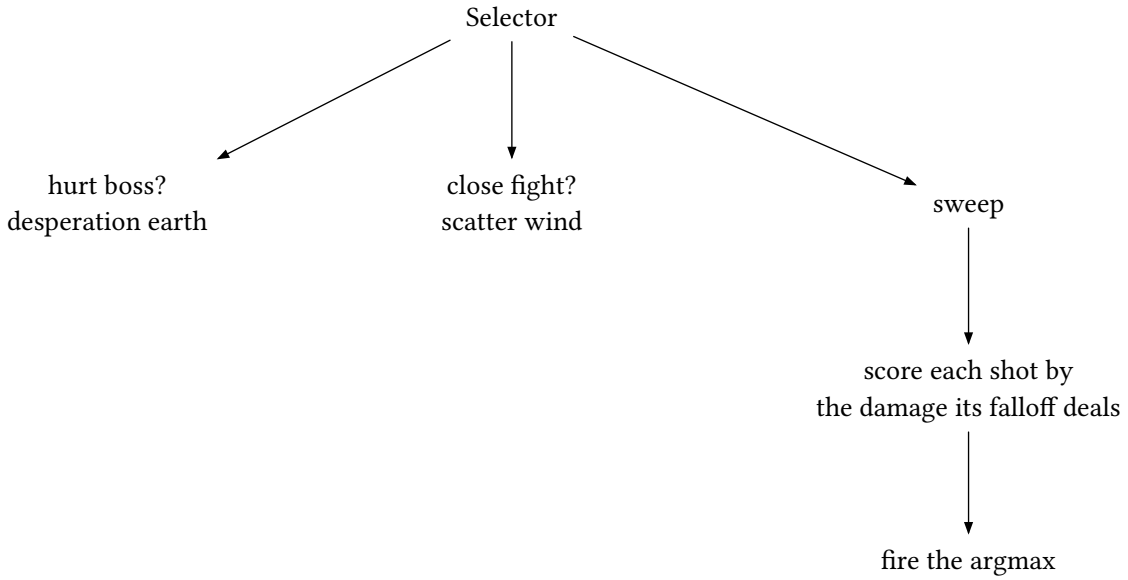



figure 62: the hybrid: a selector over desperation, scatter, and the sweep leaf

## 19.5. the match is still a pure function

The same seed and the same log rebuild the same end state, now with walks, flights, and weapons in the log, magic "GSR3". The tests pin the fsm's refusals, the world populated in flight and empty at rest, wind impacts counting six on a wind bolt, the fire ball deeper and the earth ball wider on identical flights, friendly fire off, team wipes ending the match, agility ordering three actors, movement cost and blocking, the paper plane once per match, burn, soak, knock, and lock timing, the damage pipeline's caps, crit from a named stream, and the shield halving a real incoming hit. The chapter 8 bug story survives the rewrite: the first end to end run still needed the segment collision to skip its launch end, and that regression pin is still in chapter 8′s suite.

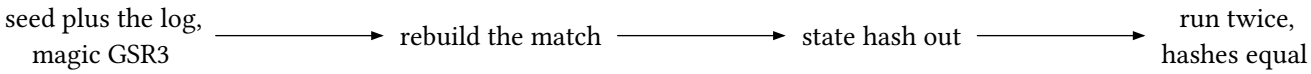



figure 63: seed plus log in, the rebuild, the state hash out, run twice

**NOTE: why the tick matters**

The old engine resolved a shot inside one `Fire` call, which was correct and invisible. Splitting the flight into ticks changes no number, the tick count is a pure function of state, but it lets a window animate the shell instead of its recording, and it is what makes the drain between systems observable rather than decorative.

sources: everything here composes chapters 3, 5, 6, 8, 9, 11, 13, and 15 of this book, each with its own pinned sources. Verified by `dotnet test books/game-systems/capstone/GameSystemsCapstone.slnx`, 95 tests in `GameSystems.Capstone.Tests` covering the engine, the elements, the balance bands, and the ai.

# 20. capstone part 2: the campaign core

Part 1 left the battle a pure function; part 2 spends it. The campaign is the composition root over chapters 14 through 17: a character sheet that levels and wears 14 slots of gear, a shop and a diamond forge, pets that hatch with three mana gated actives and two passives, and expeditions that turn stage records into battles with a boss at the end. One rule holds all of it together, the whole campaign is a pure function of the profile seed plus the command sequence, pinned by a test that replays a scripted run twice and hashes after every command.

## 20.1. the composition root

```
    private readonly ulong _profileSeed;
    private readonly Rng _battleSeeds;
    private readonly Rng _loot;
    private readonly Rng _craft;
    private readonly Rng _hatch;
    private readonly AiOpponent _ai;
    private readonly ScriptHost? _scripts;
    private readonly IPetBrain _petBrain;

    /// <summary>Seed-only construction, scripts on: every existing call site.</summary>
    public Campaign(ulong profileSeed)
        : this(profileSeed, null)
    {
    }

    public Campaign(ulong profileSeed, CampaignOptions? options)
    {
        _profileSeed = profileSeed;
        _battleSeeds = new Rng(profileSeed).Stream("battle");
        _loot = new Rng(profileSeed).Stream("loot");
        _craft = new Rng(profileSeed).Stream("craft");
        _hatch = new Rng(profileSeed).Stream("hatch");
        _scripts = options is { Scripts: false } ? null : new ScriptHost();
        _ai = new AiOpponent(_scripts is null ? null : new LuaUtilityBrain(_scripts));
        _petBrain = _scripts is null ? CompiledPetBrain.Instance : new LuaPetBrain(_scripts);
    }

    /// <summary>Shut the campaign's lua runtime down with it.</summary>
    public void Dispose() => _scripts?.Dispose();

    // the roster of the live battle, combatant id to its enemy definition
    private readonly Dictionary<int, EnemyDef> _enemyDefs = [];

    private readonly int[] _stagesCleared = new int[CampaignCatalog.Expeditions.Count];

    private bool _sandbox;

    public CharacterSheet Sheet { get; private set; } = new(
        "gunner", Level: 1, Xp: 0, Base: new PrimaryBlock(3, 5, 2, 1),
        Loadout: new Loadout((SlotId.Weapon, WeaponCatalog.Water)),
        Upgrades: new int[Slots.All.Length],
        Grid: new InventoryGrid(),
        Gold: 100, Diamonds: 0, ActivePet: 0);
```

listing 73: every stream forked once at construction, and the v2 sheet it all hangs on (source: game-systems/capstone/src/Campaign.cs, lines 32-74)

Chapter 11's discipline at its widest: `"battle"`, `"loot"`, `"craft"`, and `"hatch"` fork at construction, battle seeds derive per expedition and stage without touching the parent, and each battle's own wind forks inside the engine. A replayed profile draws the same loot and hatches the same pets, every time. The sheet is the chapter 15 record: primary attributes, the 14 slot loadout with per slot upgrade levels, the paged grid, and the wallet, all of it folded into the state hash.

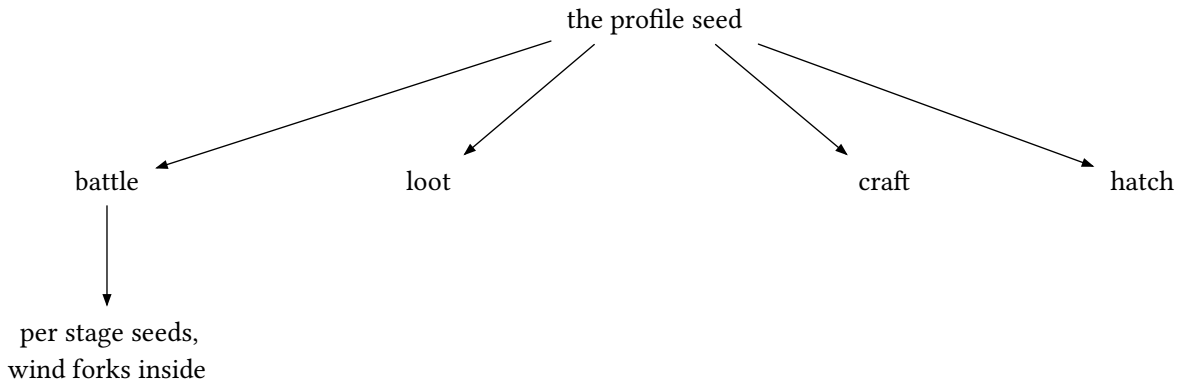



figure 64: the composition root: four streams forked once, stage seeds derive

## 20.2. a stage becomes a battle

```
    private void BeginStage()
    {
        _sandbox = false;
        StageDef stage = CurrentStageDef;
        DerivedStats stats = ComposeStats();
        ulong seed = _battleSeeds.Stream($"stage:{CurrentExpedition}:{CurrentStage}").NextUlong();
        // flat pools scale x3 to the v3 weapon base of 100 physical, the
        // monte carlo pass retires this multiplier when it tunes the table
        int playerHealth = stats.MaxHealth * 3;
        var combatants = new List<CombatantState>
        {
            new(1, playerHealth, Fp.FromInt(32), IsPlayer: true,
                Team: 1,
                Offense: stats.Primaries.Offense, Defense: stats.Primaries.Defense,
                Agility: stats.Primaries.Agility, Luck: stats.Primaries.Luck,
                Armor: stats.Armor,
                MaxHealth: playerHealth,
                Stamina: stats.MaxStamina, MaxStamina: stats.MaxStamina,
                Mana: stats.MaxMana, MaxMana: stats.MaxMana,
                ElemDamage: stats.ElemDamage, ElemRes: stats.ElemRes),
        };
        _enemyDefs.Clear();
        if (stage.Enemies.Count > 0)
        {
            // the roster spawns right to left, 40 meters apart, ids in order
            for (int i = 0; i < stage.Enemies.Count; i++)
            {
                EnemyDef e = stage.Enemies[i];
                int id = 2 + i;
                _enemyDefs[id] = e;
                combatants.Add(new CombatantState(id, e.Health,
                    Fp.FromInt(480 - 40 * i), IsPlayer: false,
                    Team: 2, Agility: e.Agility, Offense: e.Offense, Defense: e.Defense,
                    MaxHealth: e.Health, Stamina: 100, MaxStamina: 100));
            }
        }
        else
        {
            combatants.Add(new CombatantState(2, stage.EnemyHealth, Fp.FromInt(480), IsPlayer: false,
                Team: 2, Offense: stage.EnemyAttack, Defense: stage.EnemyDefense,
```

```
                MaxHealth: stage.EnemyHealth, Stamina: 100, MaxStamina: 100));
        }
        CurrentBattle = new Match(
            new MatchConfig(seed, 512, stage.TerrainOctaves, stage.TerrainAmplitude,
                playerHealth, Fp.FromDouble(120), MaxTurns: 100), combatants);
    }

    /// <summary>
    /// The sheet to battle derivation: every worn item's stats, upgrade level
    /// applied, plus the active pet's legacy bonus, through chapter 14's
    /// pure function.
    /// </summary>
    private DerivedStats ComposeStats()
    {
        EquipmentContrib equipment = EquipmentContrib.None;
        foreach (SlotId slot in Slots.All)
        {
            if (Sheet.Loadout[slot] is not int id)
                continue;
            ItemDef item = CampaignCatalog.Item(id);
            ItemStats scaled = UpgradeMath.Scale(item.Stats, Sheet.UpgradeLevel(slot));
            equipment += scaled.ToContrib();
        }
        PetContrib pet = ActiveSpecies?.PassiveContrib() ?? PetContrib.None;
        return Attributes.Derive(Sheet.Base, Sheet.Level, equipment, pet);
    }
```

listing 74: stats derive through the loadout, the stage arms the enemy, the seed derives (source: game-systems/capstone/src/Campaign.cs, lines 449-514)

`BeginStage` is where the systems meet: `ComposeStats` walks the loadout, scales each worn piece by its upgrade level, adds the active pet's passives, and runs chapter 14's `Attributes.Derive` to produce the primaries, armor, pools, and elemental arrays the player fights with, the stage record arms the enemy with its own numbers, weapon, and terrain, and the derived seed fixes the ground under both. Losing keeps the cleared stages and the retry walks back into the same fight, which is why the unlock chain of chapter 17 is permanent progress rather than a streak.

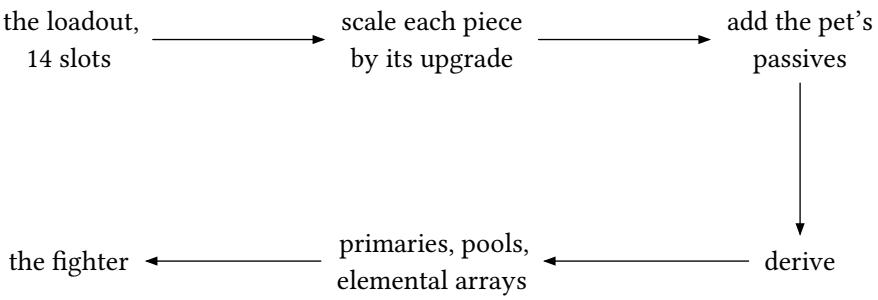



figure 65: a stage becomes a battle: the loadout walks into derive

## 20.3. the payout

```
        var loot = new List<(int, int)>();
        int xp = 0;
        bool leveled = false;
        PetInstance? hatched = null;
        if (won)
        {
            StageDef stage = CurrentStageDef;
            loot.AddRange(Loot.Roll(stage.Rewards, _loot, draws: stage.IsBoss ? 2 : 1));
            xp = stage.IsBoss ? 120 : 40;
            // the loot stream pays the purse after the table rolls, so the
            // gold draw is a pure function of the wins before it
            int gold = stage.GoldLo + _loot.NextInt(stage.GoldHi - stage.GoldLo + 1);
            int diamonds = stage.IsBoss ? 4 : 0;
            (Sheet, leveled) = Growth.GainXp(Sheet, xp);
            Sheet = Sheet with { Gold = Sheet.Gold + gold, Diamonds = Sheet.Diamonds + diamonds };
            foreach ((int itemId, int count) in loot)
                Sheet.Grid.Add(itemId, count, CampaignCatalog.Item(itemId).MaxStack);
            _stagesCleared[CurrentExpedition]++;
            if (stage.IsBoss)
            {
                PetInstance pet = Hatchery.Hatch(_hatch, CampaignCatalog.Species);
                _pets.Add(pet);
                Sheet = Sheet with { ActivePet = _pets.Count };
                hatched = pet;
            }
        }
        CurrentBattle = null;
        return new BattleOutcome(won, loot, xp, leveled, hatched);
    }
```

listing 75: loot rolls, gold and diamonds pay, xp crosses levels, the boss hatches a pet (source: game-systems/capstone/src/Campaign.cs, lines 354-382)

A won battle rolls the stage's weighted table through the `"loot"` stream, then the same stream rolls the gold between the stage's bounds, bosses pay four diamonds on top, xp can cross several levels in one step, and a boss clear hatches from the `"hatch"` stream and becomes the active pet. The gold buys shop gear and the diamonds upgrade it, both compounding into the next battle's derived stats. A lost one pays nothing and keeps the progress. Either way the outcome record carries what the reward screen shows.

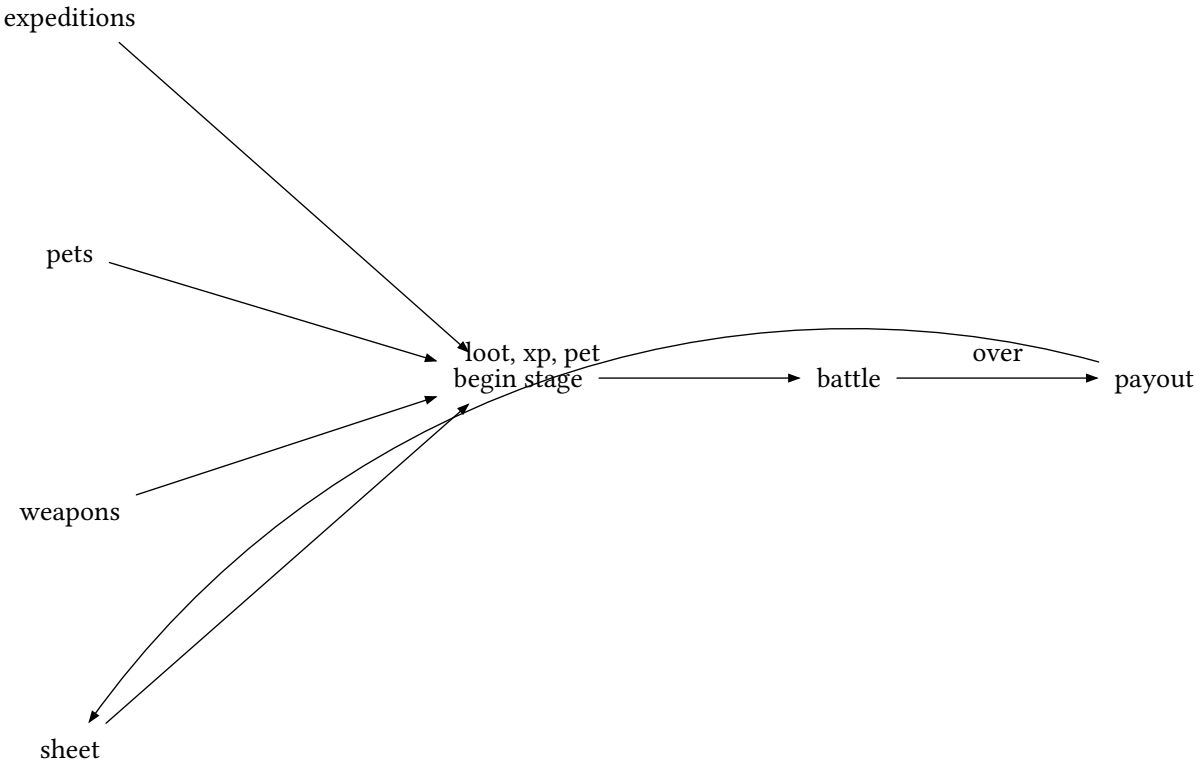



figure 66: the campaign loop, everything feeds the sheet and the sheet feeds back

## 20.4. replay v3

```
/// <summary>
/// One turn's committed input: the shot as four integers, plus the aim
/// phase that led to it, decimeters walked and any paper plane flight, so
/// the replay reproduces movement and positioning exactly.
/// </summary>
public sealed record TurnInputV3(
    int PlayerId,
    int AngleDegrees,
    int Power,
    int WeaponId,
    int MoveDm,
    int Flags,
    Fp TeleportX);

/// <summary>
/// The battle replay, version 3: same discipline as the chapter 12 codec,
/// magic bumped, seven fields per turn, the teleport landing only read when
/// its flag is set. The profile hash ties the file to the character that
/// fired it, the identity versus balance split with the sheet standing in
/// for the seed alone.
/// </summary>
public sealed record BattleReplayV3(
    ulong Seed,
    int Width,
    ulong ProfileHash,
    IReadOnlyList<TurnInputV3> Turns);

public static class ReplayV3
{
    private static readonly byte[] Magic = "GSR3"u8.ToArray();
    private const byte Version = 3;
    private const int HeaderBytes = 29;
    private const int TurnBytes = 32;
```

listing 76: magic bumped, version 3, seven fields per turn, the profile hash beside the seed (source: game-systems/capstone/src/ReplayV3.cs, lines 16-48)

Chapter 12 taught the rule and warned what happens when a format grows: this is the predicted case twice over. Weapons joined the log in version 2, then movement joined: the turn record now carries the shot, the decimeters walked while aiming, and the flags holding any paper plane flight, the magic moved to `GSR3`, the version byte reads 3, and the profile hash rides beside the seed so a file names the character that fired it. The chapter 12 codec still loads version 1 forever; this lineage replaces its own version 2 outright, the binary is book-versioned, and a battle rebuild replays its log through a fresh campaign start to the same state hash.

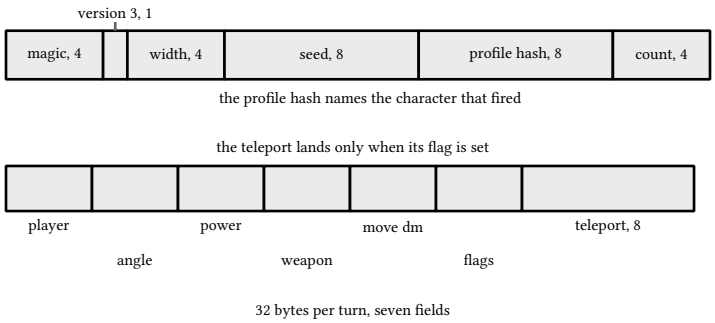



figure 67: replay v3: 29 of header, the profile hash beside the seed, 32 per turn

sources: composed from chapters 11, 14, 15, 16, and 17 of this book, each with its own pinned sources. Verified by `dotnet test books/game-systems/capstone/GameSystemsCapstone.slnx`, 95 tests in `GameSystems.Capstone.Tests` covering the campaign, the economy, the balance bands, and the replay.

# 21. capstone part 3: the playable mvp

Parts 1 and 2 left the game complete, deterministic, and invisible. This part makes it playable: a window through the KNI framework you drive with keyboard and mouse, home hub to battle to reward, and the thin htmx companion beside it. Both frontends obey the one rule that keeps them thin, they hold no game logic: every keystroke, click, and post translates into a campaign call, every pixel and tag comes from the render model or a state fragment.

## 21.1. kni, taught before used

KNI is a cross platform c# game framework, a fork of MonoGame that keeps the XNA surface, `Game`, `GraphicsDeviceManager`, `SpriteBatch`, `Vector2`, and adds backends from Windows DirectX 11 through desktop OpenGL to the browser over Blazor and WebGL. Everything asserted here was verified against release 4.3.9001 on .NET 10, in this repository, including one fact the first build missed.

The shape that matters is the `Game` lifecycle chapter 1 described in the abstract: subclass `Game`, a `GraphicsDeviceManager` owns the window and device, the framework calls `Update` zero or more times then `Draw` exactly once per frame, driven by `IsFixedTimeStep` and `TargetElapsedTime`, which this renderer sets to a sixty hertz fixed step. Drawing goes through a `SpriteBatch`, and the honest shape primitive is a one pixel white `Texture2D` stretched into every rectangle. No content pipeline and no asset files: shapes are that pixel, and the one asset-adjacent thing, the font, is a vendored ttf rasterized at runtime, the next section.

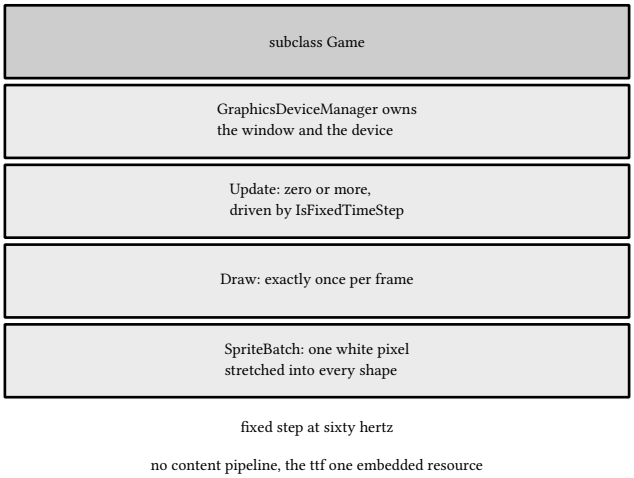



figure 68: the kni Game contract stacked, who owns what each frame

Onboarding is two facts and one trap. Fact one, the `PackageReference` `nkast.Kni.Platform.WinForms.DX11` 4.3.9001 pulls the framework and the DirectX backend. Fact two, the target framework must be `net10.0-windows`, because the platform package ships its windows backends under windows tfms only: a plain `net10.0` target restores the framework references, compiles clean, and then dies at startup inside `GameFactory` because the platform assembly that registers it was never copied to the output. That trap was found by running, not by building, and the csproj comment in this repository now pins it.

## 21.2. the runtime font

```
/// <summary>
/// The three text sizes in final pixels at the 1080p window: caption for
/// panel furniture, body for rows and hud lines, title for headings.
/// </summary>
public enum FontTier
{
    Caption = 18,
    Body = 24,
    Title = 44,
}

/// <summary>
/// The game font: Iosevka Term Nerd Font, embedded from the repo's
/// fonts/ dir and rasterized on demand through FontStashSharp, so the
/// whole glyph set including lowercase ships inside the exe. No content
/// pipeline, no copied asset files; the atlas textures bind themselves to
/// the drawing batch's device on first use, and Load from LoadContent
/// simply swaps in a fresh system.
/// </summary>
public static class Font
{
    private static FontSystem? _system;

    public static void Load(GraphicsDevice device)
    {
        _system?.Dispose();
        _system = new FontSystem();
        using Stream ttf = OpenEmbeddedTtf();
        _system.AddFont(ttf);
    }

    public static int Measure(string text, FontTier tier = FontTier.Body) =>
        (int)_system!.GetFont((int)tier).MeasureString(text).X;
```

listing 77: three tiers in final pixels, one font system loaded from the embedded ttf (source: game-systems/capstone/src/renderer/Font.cs, lines 8-40)

```
    public static void DrawText(
        SpriteBatch? batch, int x, int y, string text, Color color, FontTier tier = FontTier.Body)
    {
        batch?.DrawString(_system!.GetFont((int)tier), text, new Vector2(x, y), color);
    }

    /// <summary>
    /// The old bitmap-font signature kept for the screens that still pass a
    /// pixel and an integer scale: 1 reads as body text, 2 and up as title.
    /// </summary>
    public static void DrawText(
        SpriteBatch? batch, Texture2D pixel, int x, int y, string text, Color color, int scale = 1)
    {
        DrawText(batch, x, y, text, color, scale ≥ 2 ? FontTier.Title : FontTier.Body);
    }

    private static Stream OpenEmbeddedTtf()
    {
        string name = "IosevkaTermNerdFont-Regular.ttf";
        Assembly assembly = Assembly.GetExecutingAssembly();
        string[] all = assembly.GetManifestResourceNames();
        string? resource = Array.Find(all, r => r.EndsWith(name, StringComparison.Ordinal));
        return resource is null
            ? throw new InvalidOperationException($"{name} is not embedded in {assembly.GetName().Name}")
            : assembly.GetManifestResourceStream(resource)!;
    }
```

listing 78: one draw call per text, the legacy pixel-and-scale signature mapped onto the tiers (source: game-systems/capstone/src/renderer/Font.cs, lines 42-67)

Still no content pipeline, but there is a font now. The repo already vendors the Iosevka Term Nerd Font ttfs for the pdf theme, and the renderer embeds the regular cut, 13,230,756 bytes, straight from `fonts/` as a resource, five directories up in the csproj with a `Link` and nothing copied to the output. Rasterization is runtime work through FontStashSharp.Kni 1.6.1 from nuget: `Load` from `LoadContent` builds one `FontSystem` and feeds it the embedded stream, and from then on a size is a dynamic sprite font whose glyphs rasterize into atlas textures on first use, the atlas binding itself to the drawing batch's device. The old 5x7 bitmap could not be made bigger or given lowercase without hand-drawing hundreds of glyphs, and the ttf gives both for free.

Three sizes cover every screen, named tiers in final pixels at the 1080p window: caption 18 for panel furniture, body 24 for rows and hud lines, title 44 for headings. The layout grid scales at `ViewBox.Scale`, 1080 over 400, so 2, and the tiers were pitched against that grid in a re-spacing pass over the screens. Lowercase renders now, and each call site chooses its own casing instead of a forced `ToUpperInvariant` at the font. A compatibility overload keeps the old pixel-and-scale signature compiling, 1 reading as body and 2 and up as title, and four call sites still ride it. Fixed sizes and a fixed draw sequence make the StbTrueTypeSharp rasterization deterministic, and the golden walkthrough in the last section asserts it the blunt way, two captures of the same beat in two processes demanding byte-identical pngs.

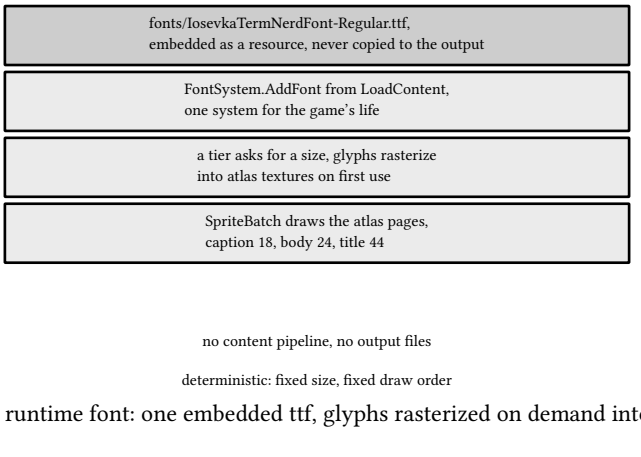



figure 69: the runtime font: one embedded ttf, glyphs rasterized on demand into atlas pages

## 21.3. the screens and the input

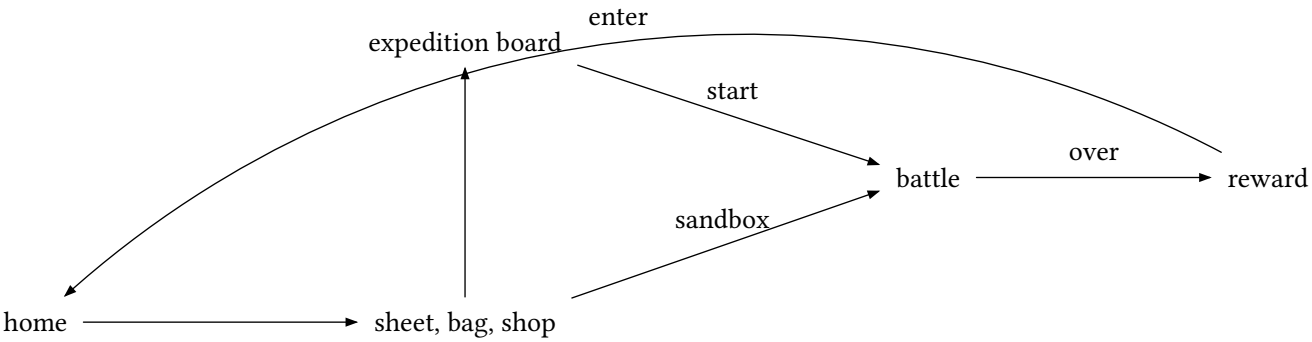



figure 70: the screen router, the hub feeds every screen and every screen walks home

The home hub is a static map with three doors, the sandbox range, the expedition board, and the shop: clicks and the 1 to 3 keys route, and the screen owns nothing but the routing. The board lists each expedition's stages under its name, the boss stage marked and last, locked or cleared. The unified sheet carries three tabs, character, pet, crafting: the paper doll wears all 14 slots around the wearer holding the equipped weapon, the level and xp bar above, the power number under the hands, and the full stat grid below, stamina, armor, health, mana, the four elemental damages, the four resistances, and the four primaries. Beside the doll sits the bag, pages of 16x16 cells at 64 pixels, one item per cell, arrows walking the cursor, q and e turning pages, r sorting stacks, enter equipping. The crafting tab is the diamond bench, each level costs 2 to the n diamonds and compounds every stat on the item, with the material slots strip under it. In battle a and d walk the terrain on stamina, w and s aim, 1 through 4 switch the element, space charges and releases, e fires the pet's recommended active, the same choice the pet brain of chapter 18 makes, i opens the sheet over the fight, t arms the paper plane and a minimap click flies it, b toggles the aim bot's dotted arc drawn through the live wind. The window opens at 1920x1080 and every hud element scales with it: run it with `dotnet run --project books/game-systems/capstone/src/renderer`, escape leaves the home.

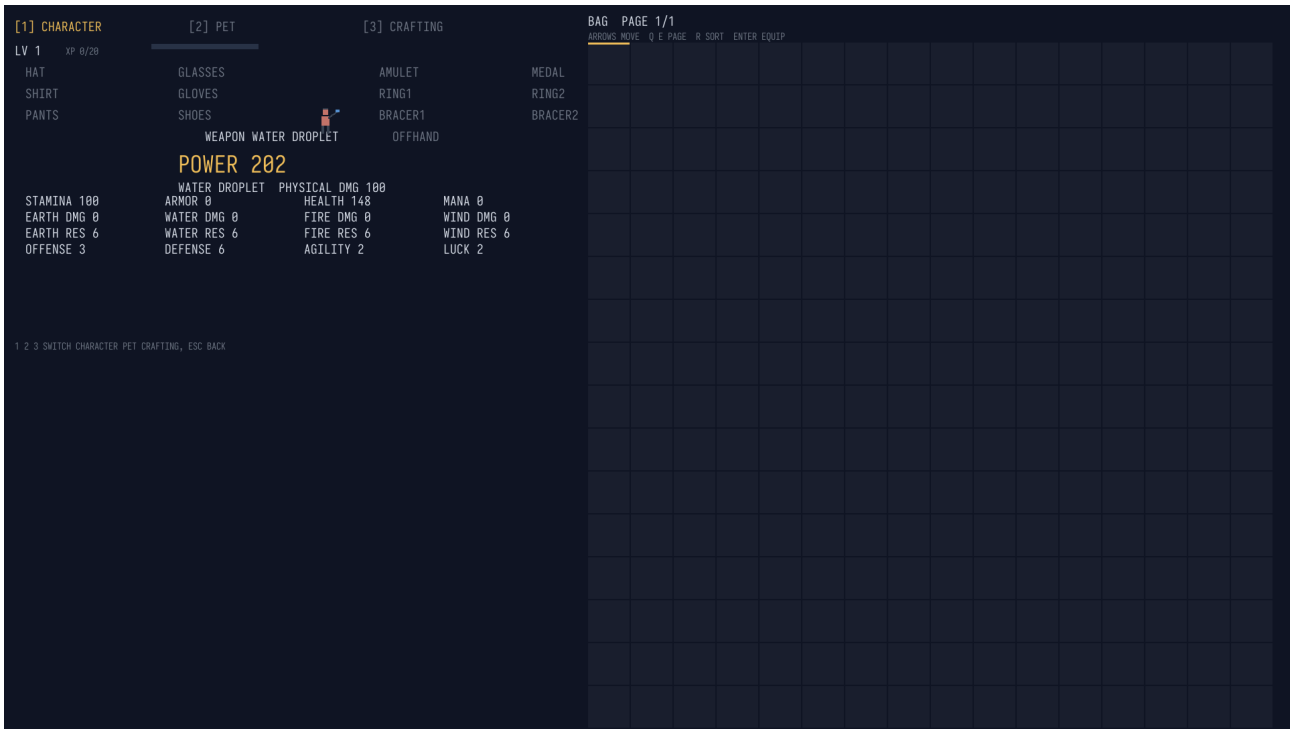



figure 71: the unified sheet as the exe captures it, the paper doll, the power number, and the bag beside it




figure 72: the home hub as the exe captures it, three doors on a static map, the purse and power on the header

The exe also produces frames without a player: `--screenshot <scene> --out frame.png` drives the same seed 42 profile to that scene, captures the backbuffer once the batch ends, writes the png, and exits 0. The script behind a scene steps once per drawn frame, so the launch burst of catch-up updates cannot shift it and the same scene writes the same frame bytes on every run. A scene that never arrives trips a tick guard and exits 1, a bad flag exits 2 with the scene list in the message, and the smoke suite runs all of those cases, so a renderer that compiles but cannot actually run fails `dotnet test`. `make screenshots` wraps the same flag: it asks the exe's usage error for the scene list, captures every scene plus the web companion into `output/screenshots`, and fails if any frame is missing or malformed.

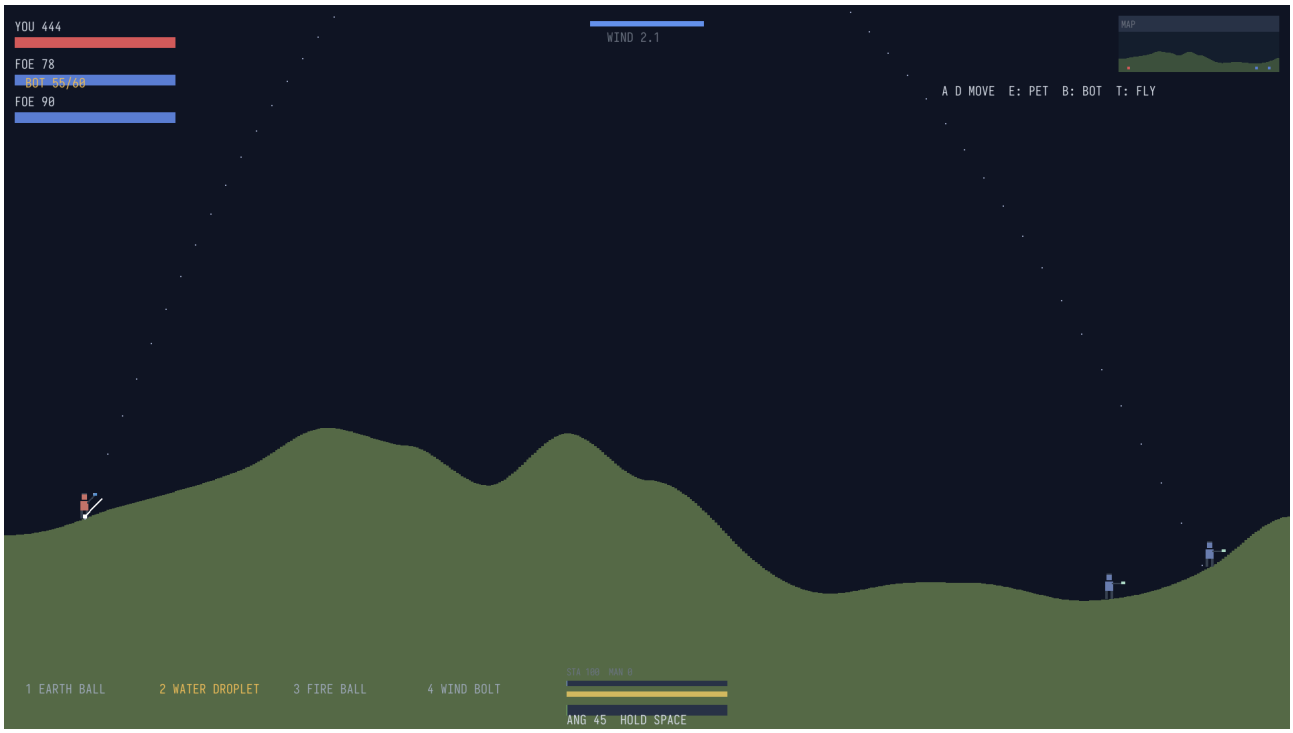



figure 73: the battle scene as the exe captures it, humanoids holding their elements, the minimap top right, the aim bot armed

## 21.4. htmx, taught before used

The web companion is the same campaign served as html fragments. htmx 4 is the hypermedia approach: the server owns the markup, small attributes on ordinary html elements declare that this element fetches and swaps, and no application javascript is written. The 4.0 release replaced the transport with `fetch()`, made attribute inheritance explicit opt-in, and renamed events to `htmx:phase:action`, but the surface this chapter uses is the same as 2.x: `hx-get`, `hx-post`, `hx-trigger`, `hx-target`, `hx-swap`.

The vendoring recipe is book 10's, reused exactly. The `next`-tagged npm package `htmx.org@4.0.0` ships `dist/htmx.min.js`, 36,716 bytes, its SHA-384 digest matches the published SRI value byte for byte, and the file lives in the project's `wwwroot/js` with the integrity attribute in the script tag. The web test downloads it through the running app and recomputes the digest, so a corrupted or half-updated vendor file fails the suite instead of the page.

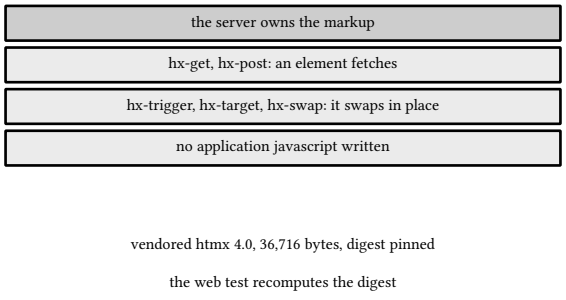



figure 74: the htmx contract: the server owns markup, elements fetch and swap

## 21.5. the thin companion

```
app.UseStaticFiles();

app.MapGet("/", () => Results.Text(IndexHtml, "text/html"));

// the whole ui: state polls in, the fire button posts, htmx swaps the
// fragment, no client script of our own exists
app.MapGet("/api/state", (CampaignHost host) => Results.Text(StateFragment(host), "text/html"));

// one post, one turn: the player's shot when it is ours, the enemy's
// otherwise, and a finished battle collects and reopens the next stage
app.MapPost("/api/fire", (CampaignHost host) =>
{
    Campaign campaign = host.Game;
    if (campaign.CurrentBattle is { } battle && !battle.Over)
    {
        if (battle.CurrentPlayerId == 1)
        {
            AiDecision d = host.Ai.Decide(battle, owned: new HashSet<int> { WeaponCatalog.Water });
            campaign.Fire(d.Angle, d.Power, WeaponCatalog.Water);
        }
        else
        {
            campaign.EnemyTurn();
        }
    }
    else if (campaign.CurrentBattle is { Over: true })
    {
        campaign.FinishBattle();
        campaign.StartExpedition(campaign.CurrentExpedition + 1); // same id, next stage
    }
    return Results.Text(StateFragment(host), "text/html");
});
```

listing 79: one post advances one turn, a finished battle opens the next stage (source: game-systems/capstone/src/web/Program.cs, lines 19-50)

The page polls `/api/state` every two seconds and the fire button posts to `/api/fire`, both swapping the same fragment div, and that is the entire client. The fragment carries the battle, the health bars, and one compact sheet line, level, weapon, pet, power, gold, diamonds. The replay link serves the battle's v3 file as base64, decodable by `ReplayV3` and replayable by any campaign with the same binary.




figure 75: the companion in a browser, one fire click into stage 1

**NOTE: two frontends, one truth**

The window and the page run the same campaign core and agree because neither owns state: the window translates keys and clicks into campaign calls and reads the render model, the page translates posts into the same calls and renders fragments. The determinism suite is what lets both exist without drifting.

## 21.6. the golden run

The last walk is the golden run: one campaign on the seed 42 profile played start to finish through public calls only, both expeditions cleared boss by boss, then the shop, the equip, and the bench, with the purse asserted at every step. The renderer ships the walk as twelve beats, five from the first fight of expedition 1 and seven from the economy run after it, each a fresh process walking to its beat with real campaign calls and capturing one frame. The economy math it pins: the starter purse is 100 gold, the six battles of the two passes pay at least 15 + 15 + 40 gold twice, so 240 at worst, and the hat at 40 plus the egg at 100 always fit. Diamonds: two boss wins pay 8, two weapon levels cost 2 then 4, and 2 remain. The core test checks affordability before each buy, and a seed that ever ran short would earn the difference as extra boss clears on the already cleared pass, never a balance edit.

The combat half opens the first fight of expedition 1 under the standing constraint that the scripted hand owns water only, and slices it into five beats. Move is three fixed one-meter walks toward the foe, and the call is `Move(10)` three times, not `Move(1)`: stamina is charged per full meter, so a one-decimeter crawl would spend nothing and the stamina line would not move. Aim holds the scripted hand's own solve on the hud, the angle the water-constrained sweep picked, with the power bar charging at two thirds of the ceiling. Flight is the fired shell half way through its arc, the flown trail fading behind it. Impact is the tick after the resolve, the carved crater, the health drop, and the hud's `LAST HIT` line reading the damage the shot dealt. Over plays the battle out, and on seed 42 the player wins stage 1.

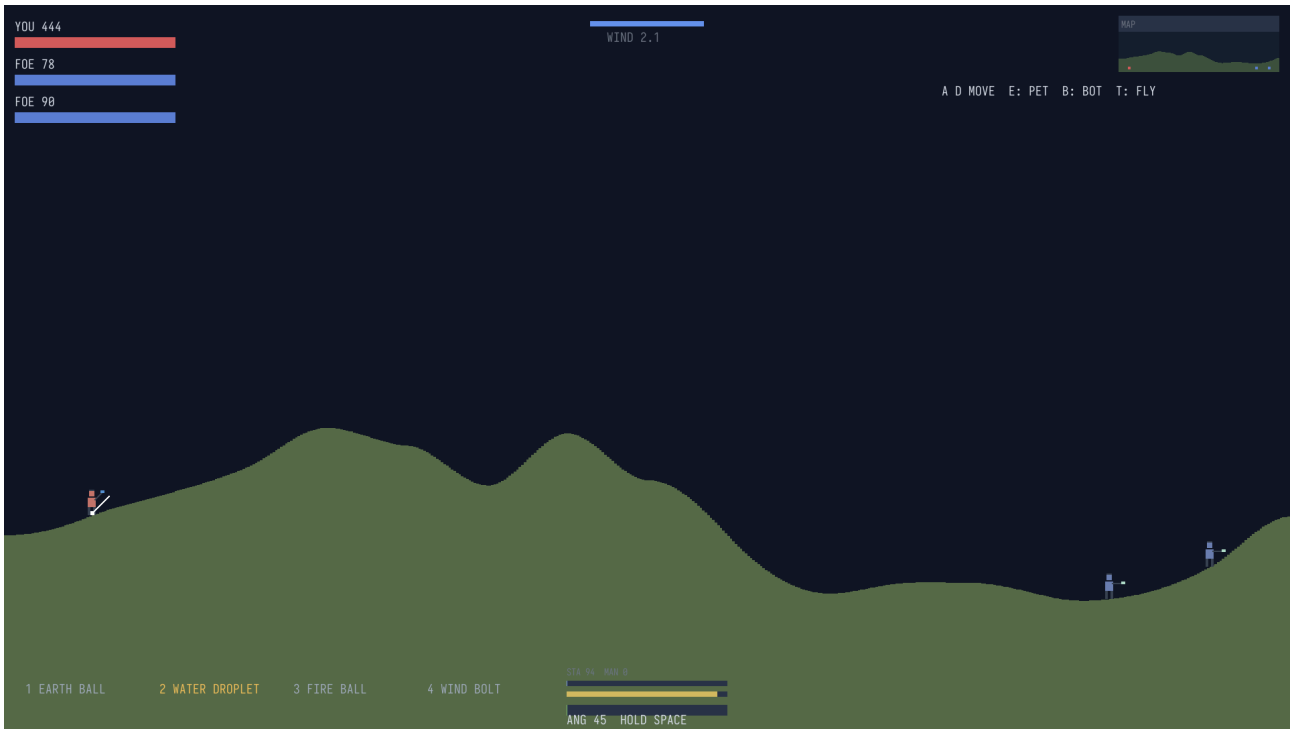



figure 76: the move beat, three fixed one-meter walks, six stamina gone from the line

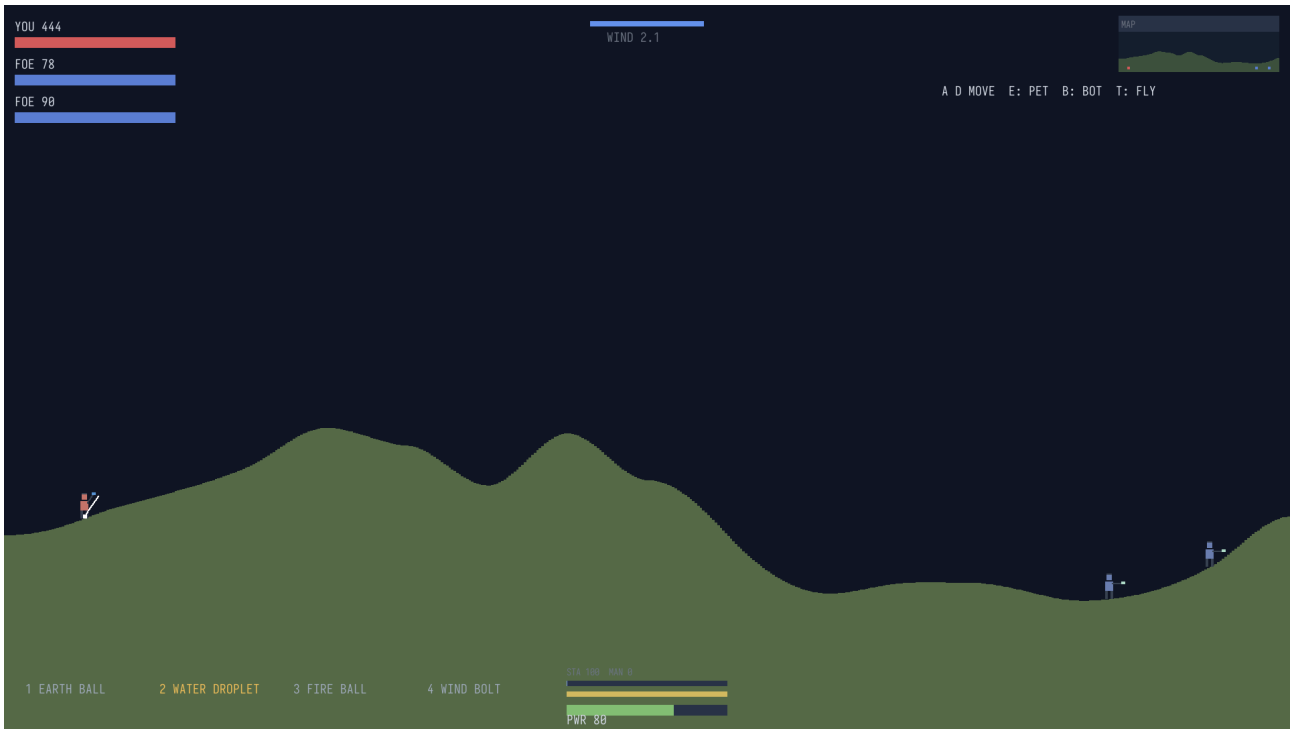



figure 77: the aim beat, the scripted solve's angle held, the power bar at two thirds

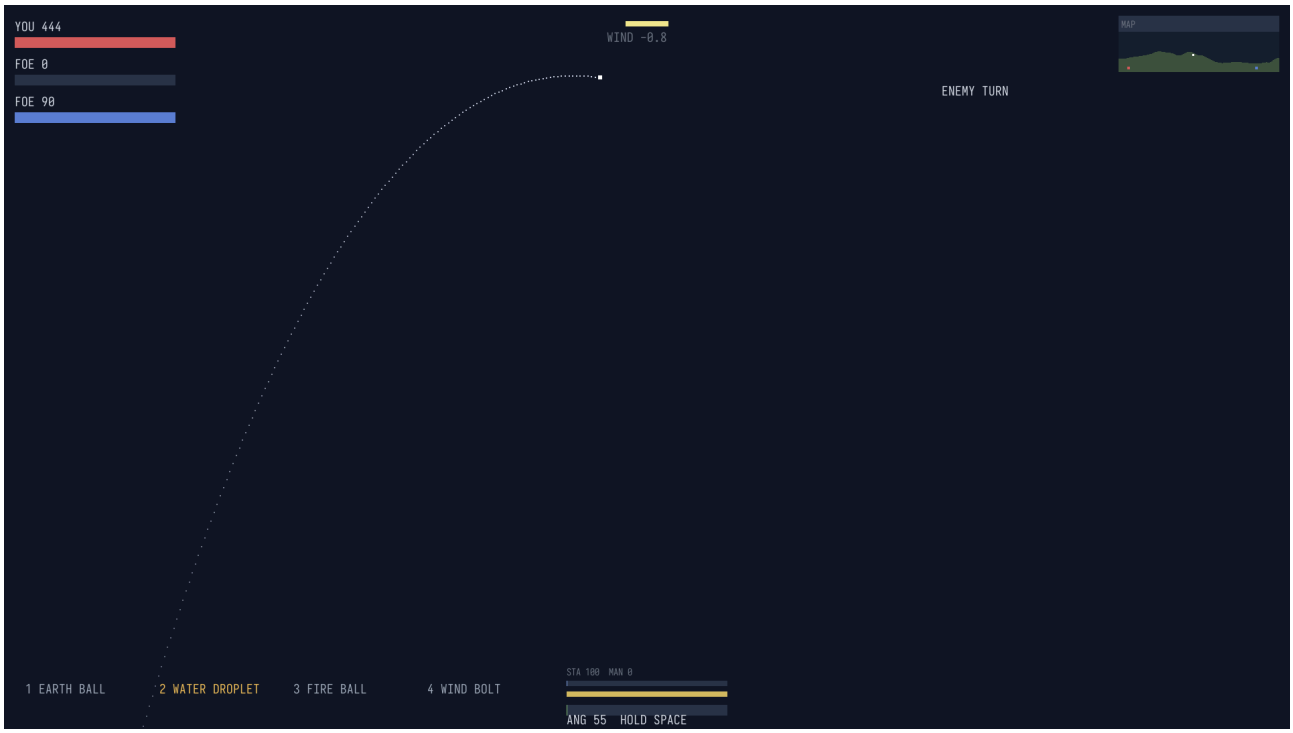



figure 78: the flight beat, the shell mid arc at half its flight, the trail fading behind

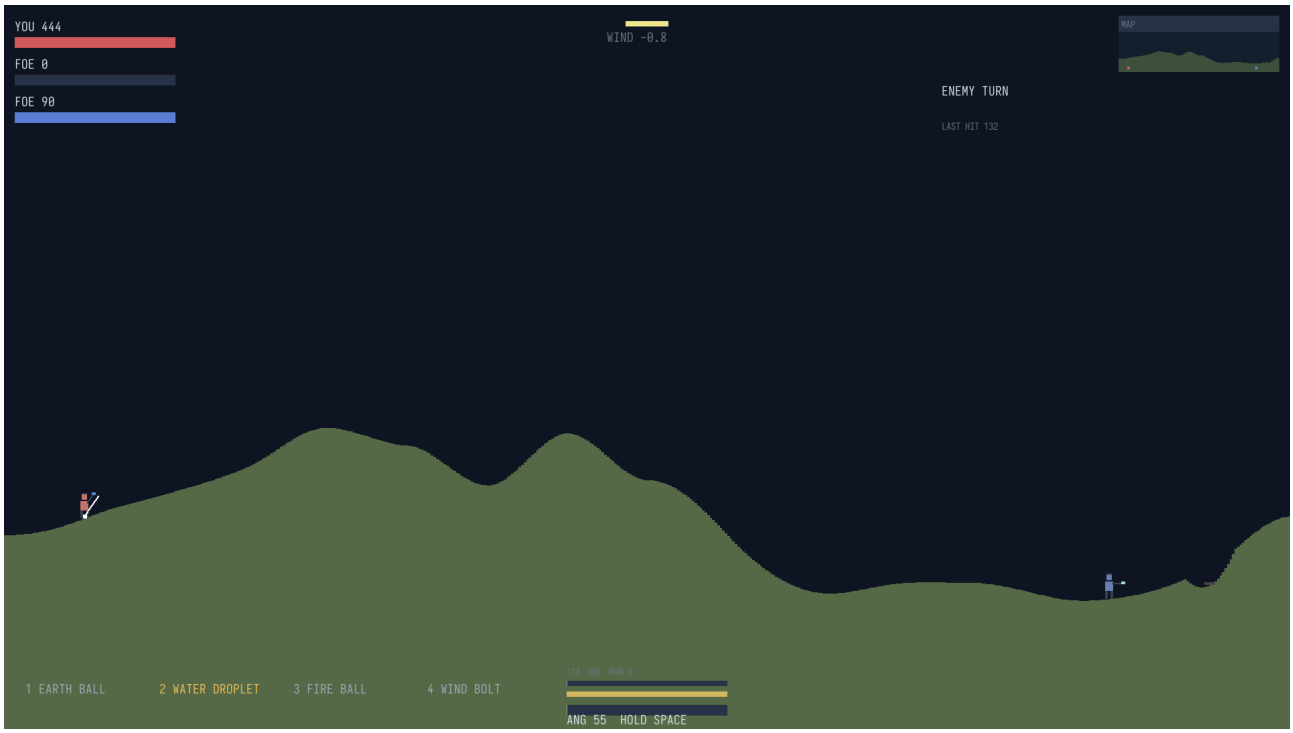



figure 79: the impact beat, the crater carved, the health dropped, the last hit readout

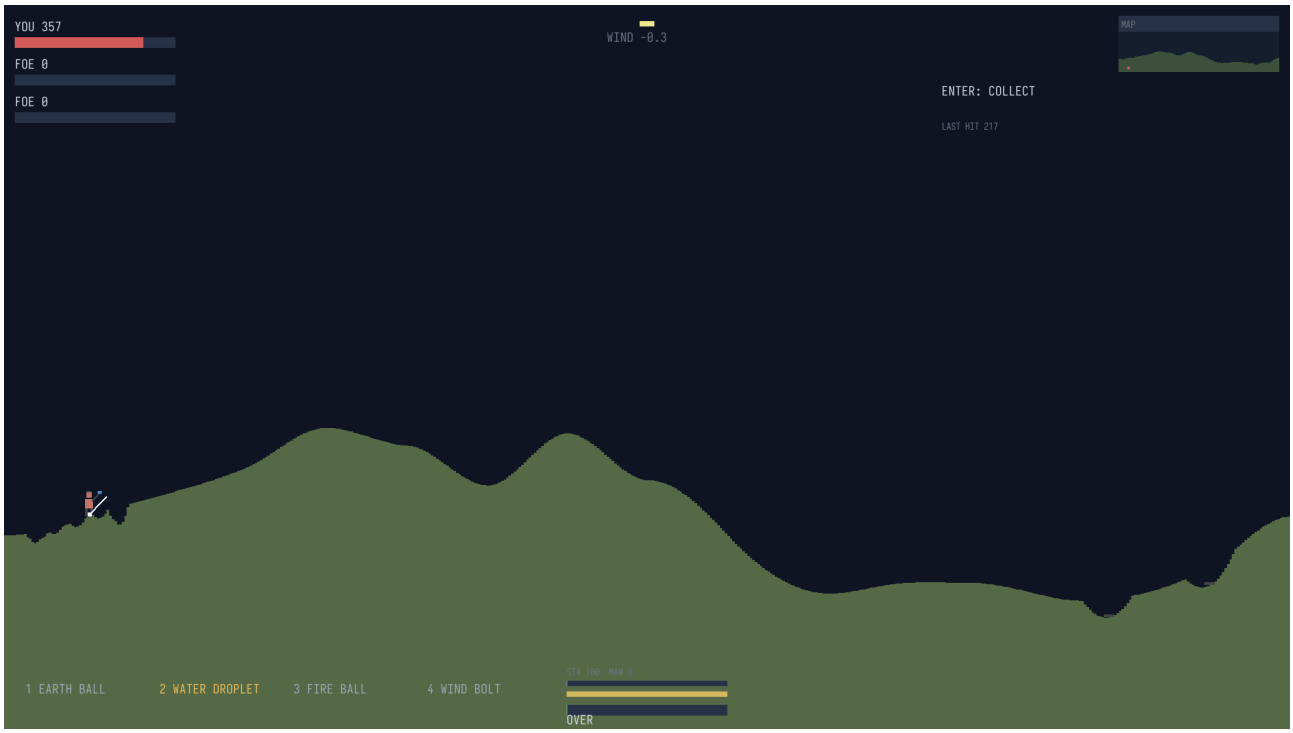



figure 80: the over beat, the foes dead, the stage 1 win on seed 42

The five beats run on chapter 18′s seam: the campaign constructs with scripts on by default, so the enemy sweep is scored by the lua weights and the pet's skill choice comes from the pet brain, while the player side stays the compiled water-constrained hand. That is a default instead of a risk because the equivalence suite walked it: per-turn `Match.StateHash` traces are identical scripts on and off over campaign walks on seeds 42 through 45, and mirror duels, both brains scripted against both compiled, agree on seeds 7, 21, and 42. The seed does not generalize: the compiled water hand loses expedition 1 stage 3 on seeds 44, 46, and 47 and expedition 2 stage 3 on seed 48. Seed 42 wins both expeditions, which is why the golden run walks 42.




figure 81: the board after the golden battles, both passes clear, pass 3 unlocked




figure 82: the alpha digger's payout, the second boss purse, 120 xp, scrap, and a hatch

The economy half is the seven steps after the battles, `golden-board` through `golden-craft`, walked the same way: the board after the clears, the boss payout, the shop with the 8 diamond purse, the bag holding the bought hat while the egg hatched at the counter, the doll wearing the sun hat, the pet tab with two hatches plus the egg, and the bench holding a water droplet +2 over 2 leftover diamonds. The smoke suite runs all twelve beats like every other scene. The sheet itself now speaks the upgrade language: a worn piece names itself with its level, "water droplet +2", and a stat prints its base plus the bench's bonus in parens, "100 + (44)", the bonus being exactly what the compounding scale added over the base.




figure 83: the golden shop, 8 diamonds and the purse that covers the hat and the egg

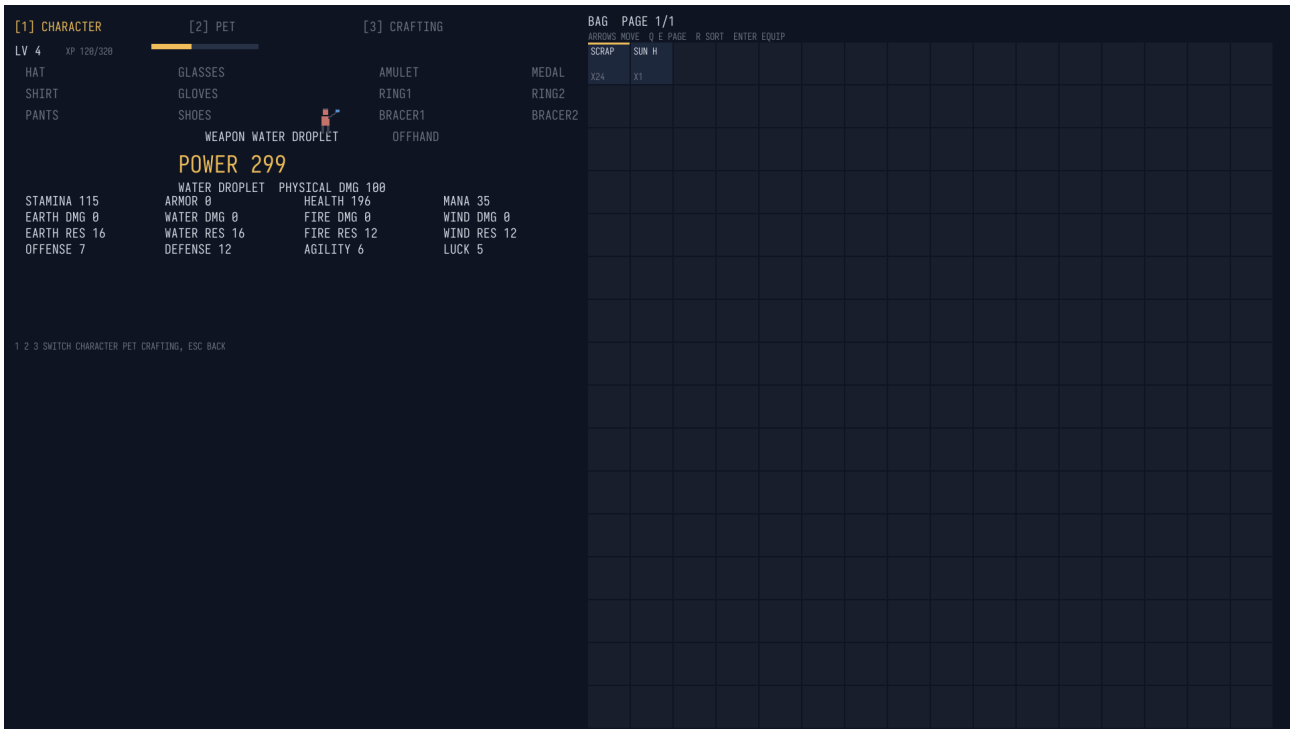



figure 84: the buy beat, the bag holding the bought hat while the egg hatched at the counter

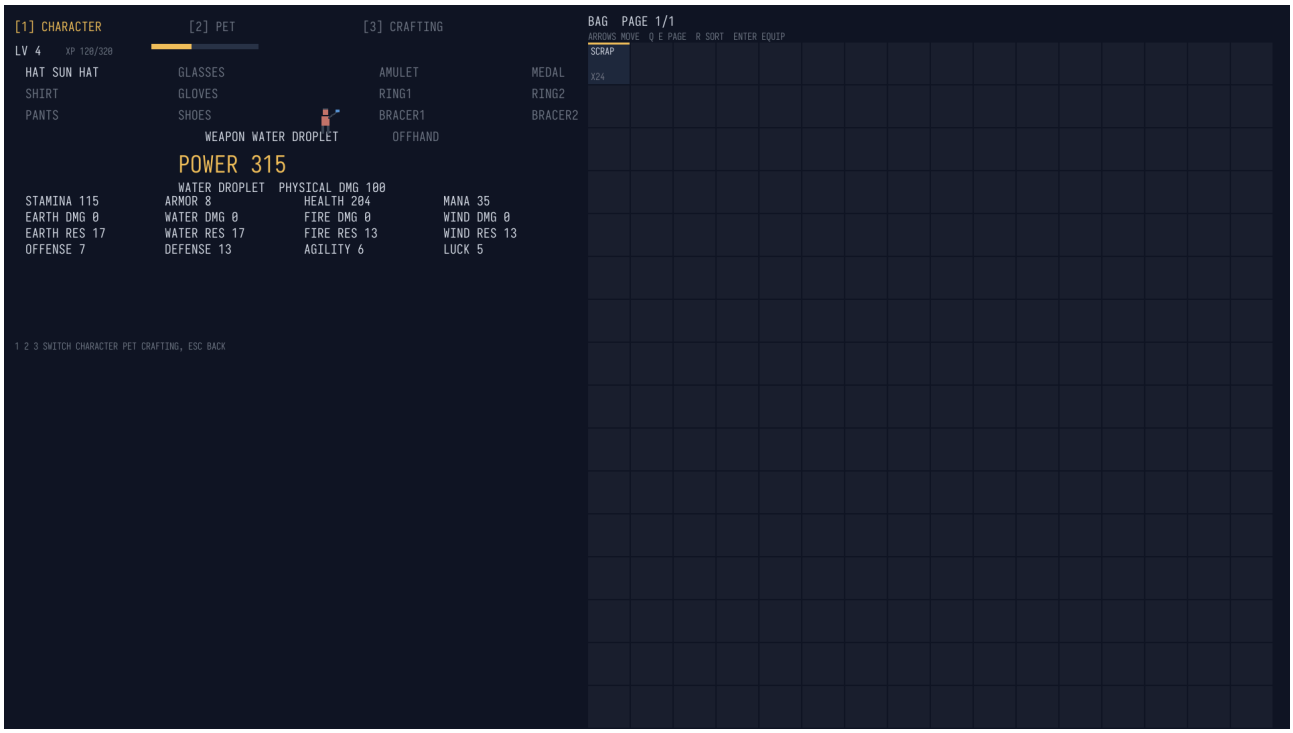



figure 85: the equip beat, the doll wearing the sun hat




figure 86: the pets beat, the pet tab with two hatches plus the egg

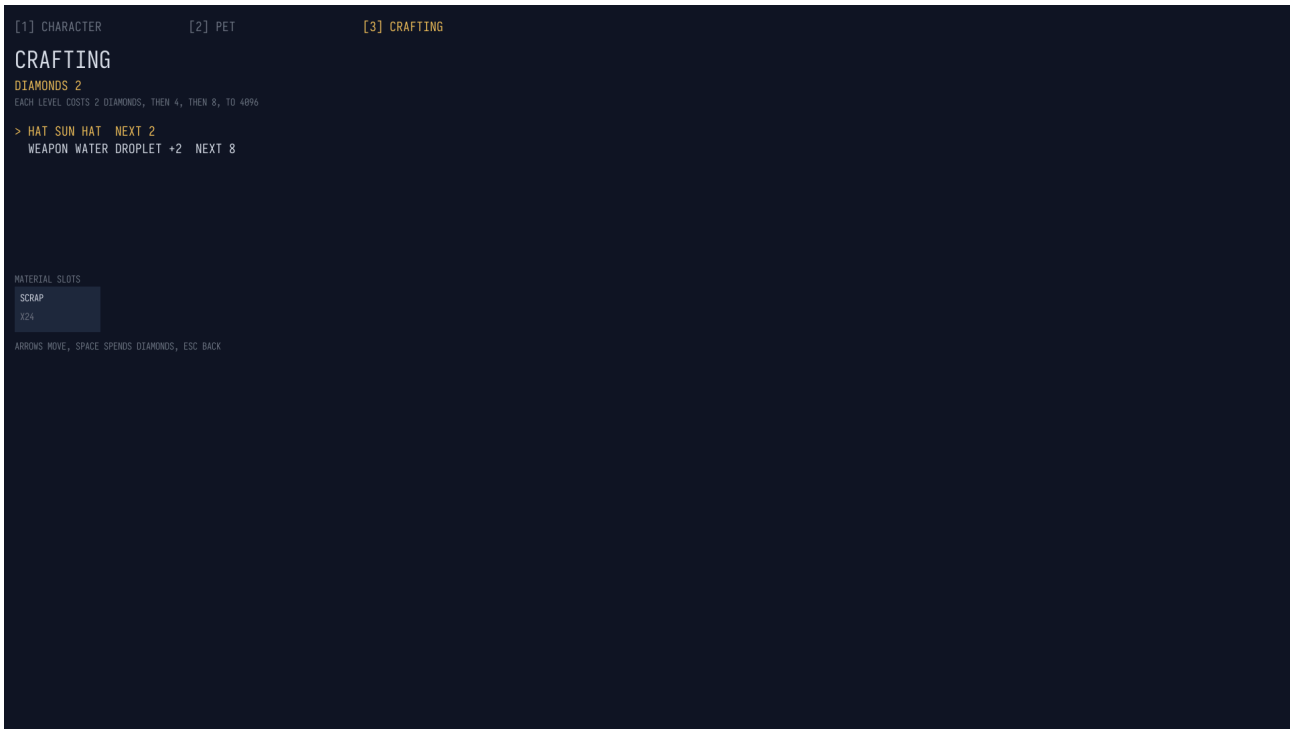



figure 87: the bench after the golden run, the weapon line reading +2 with 2 diamonds left

The web companion grew the same economy: `/api/buy` takes a stock id, `/api/equip` an item id, `/api/upgrade` a slot, each one campaign call and the same fragment back out, unknown ids answer 400, and the page carries buttons for the hat, the egg, the equip, and the weapon slot. The web test plays the golden flow over HTTP, fires to two boss wins, buys, equips, upgrades twice, and asserts the power delta of each bench visit is exactly the weapon's scale line, 100 to 120 to 144.

The run also exists as footage. `--record battle --out-dir <dir>` replays the seed 42 first battle deterministically, the same scripted play-out the over beat walks, and captures every second tick: sixty hertz of simulation becomes

thirty frames a second, so the encode plays at real-time speed. Stage 1 runs 260 ticks, 130 frames, about 4.3 seconds. The walkthrough encodes that segment at its capture cadence in yuv420p, then holds each of the twelve validated beats for two seconds, about 28.3 seconds total, and commits the result at `figures/golden-run.mp4`. The encoder measured here is ffmpeg 9.0.1, and mp4 bytes shift across encoder versions, so the mp4 is not a byte gate: the ordered validated pngs carry the byte-exact contract, the mp4 answers a measured size floor and a duration band.

`make golden` wraps the whole walkthrough: it builds both frontends, cross-checks the beat list against the exe's usage error, captures the twelve frames in walking order as `output/screenshots/golden-01-move.png` through `golden-12-craft.png`, validates each png's signature, 1920x1080 frame, and a measured byte floor, proves the frames deterministic by capturing the board beat twice in two processes and demanding byte equality, records and encodes the battle, and appends the holds. The encoder is mandatory: ffmpeg on the PATH first, a one-shot container second, and a machine with neither fails the target with a plain message.

sources: github.com/kniEngine/kni for the framework, release 4.3.9001, verified by the restore, compile, and run probes in this repository; nuget.org/packages/FontStashSharp.KNI for the runtime font rasterizer, FontStashSharp.Kni 1.6.1, and fontstashsharp.github.io for its api shape, both accessed 2026-09-23; the vendored Iosevka Term Nerd Font ttf and its OFL license live in this repository's fonts/, from the github.com/ryanoasis/nerd-fonts releases; htmx.org and the npm registry for htmx 4.0.0 with the SRI digest, accessed 2026-09-08. Verified by `dotnet test books/game-systems/capstone/GameSystemsCapstone.slnx`, 10 web tests, and the renderer smoke suite, 28 scene runs and 3 usage errors plus the record lane's bounded dump, every scene an exe run that must write a 1920x1080 png, plus `make golden` for the ordered walkthrough, the double capture, and the motion mp4.

# 22. capstone part 4: balance by simulation

Every number before this chapter was chosen by hand: weapon radii, health pools, the price of a hat. This chapter replaces the hand with a monte carlo harness. The deterministic ai plays both sides of scripted duels over fixed seeds, the suite pins bands on the results, and the tuning loop walks: measure, change one constant, measure again. Nothing here could flake, a failing band is a design regression, because every battle is a pure function of the binary and the seed span.

## 22.1. the matrix

```
    /// </summary>
    private static Match Duel(ulong seed, bool gearTeam1 = false)
    {
        int health = DuelPool;
        var combatants = new List<CombatantState>
        {
            new(1, health, Fp.FromInt(32), IsPlayer: true,
                Team: 1, Agility: 4, Offense: gearTeam1 ? 6 : 3, Defense: 5, Luck: 2,
                Armor: gearTeam1 ? 8 : 0,
                MaxHealth: health, Stamina: 100, MaxStamina: 100),
            new(2, health, Fp.FromInt(480), IsPlayer: false,
                Team: 2, Agility: 4, Offense: 3, Defense: 5, Luck: 2,
                MaxHealth: health, Stamina: 100, MaxStamina: 100),
        };
        return new Match(new MatchConfig(seed, 512, 3, Fp.FromDouble(30),
            health, Fp.FromDouble(120), MaxTurns: 100), combatants);
    }

    /// <summary>Plays one duel out with the sweep ai on both sides.</summary>
    private static (int WinnerTeam, int Turns) PlayOut(Match match, int weapon)
    {
        var ai = new AiOpponent();
        var owned = new HashSet<int> { weapon }; // one element per duel
        for (int guard = 0; guard < 200 && !match.Over; guard++)
        {
            AiDecision d = ai.Decide(match, owned: owned);
            match.FireAndResolve(d.Angle, d.Power, d.WeaponId);
        }
        return (match.WinnerTeam ?? 0, match.TurnNumber);
    }

    /// <summary>Mirror win rate for team 1 over a seed span, one element.</summary>
    public static (int Team1Wins, int Battles) Mirror(int seeds, int weapon)
    {
```

listing 80: mirrored duels on fixed seeds, the sweep ai on both sides (source: game-systems/capstone/src/BalanceMatrix.cs, lines 19-52)

Three questions, three runs. The mirror asks whether the first mover wins by coin flip: identical combatants, one element per duel, team 1 win rate pooled over the elements should sit inside forty to sixty percent. The parity run asks whether any element trivially dominates: mean turns to resolve, the suite bands each element within a third of the earth ball at eight seeds per element. The ladder asks whether the power number means anything: the same seeds twice, team 1 bare once and geared once, and gear must win more.

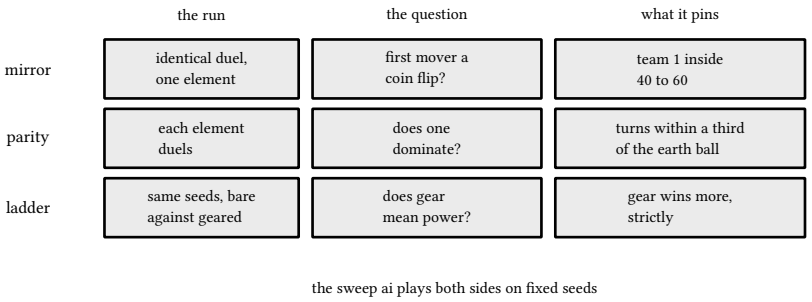



figure 88: the three questions as three runs: mirror, parity, ladder

## 22.2. the tuning loop

The first measurement was brutal: the first mover won 72 to 84 percent of mirrors, and the earth ball took three and a half times longer to resolve a duel than the wind bolt. Two constants moved. The second mover's compensation gives whoever acts last in the opening round 16 percent more health, and the mirror band is what pinned that number: 20 percent overshot into the low forties, 12 undershot into the sixties. The earth ball's elevation window widened from 45 to 55 degrees and its radius grew from 8 to 11, because a weapon that cannot aim steep cannot land close, and a weapon that cannot land close cannot kill.

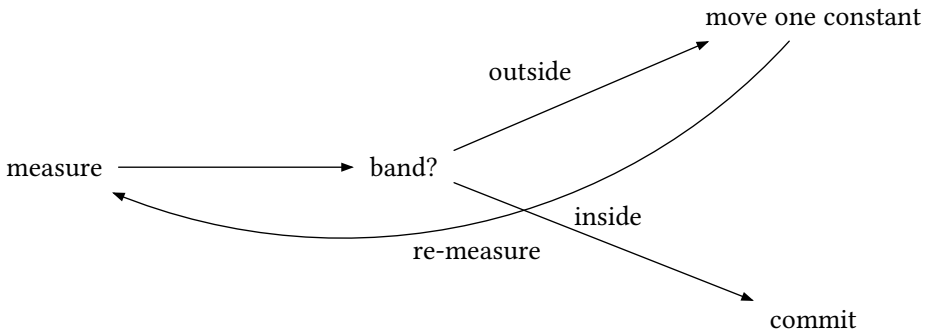



figure 89: the tuning loop, one constant moves per lap

## 22.3. the bands and the report

```
public class BalanceTests
{
    private const int MirrorSeeds = 16;
    private const int ElementSeeds = 8;
    private const int LadderSeeds = 16;

    [Fact]
    public void The_mirror_pools_to_a_forty_sixty_band_across_the_elements()
    {
        // per weapon rates swing hard at suite sized samples, so the band
        // pools all four elements: 64 duels is small enough to run and
        // wide enough to mean something
        int wins = 0, battles = 0;
        foreach (int weapon in WeaponCatalog.Selectable)
        {
            (int w, int b) = BalanceMatrix.Mirror(MirrorSeeds, weapon);
            wins += w;
            battles += b;
        }
        Assert.InRange(wins, battles * 0.40, battles * 0.60);
    }

    [Fact]
```

listing 81: pooled bands sized to the suite's runtime, never per weapon point estimates (source: game-systems/capstone/tests/BalanceTests.cs, lines 12-34)

The bands are sized to their sample. A 16 seed per weapon mirror swings twelve points either side of its mean, so a per weapon forty sixty band would fail on noise alone. The suite pools all four elements into 64 duels and bands the pool, keeps the parity band loose at a third of the anchor, and demands the ladder strictly rise. The wide report runs the same code at 128 mirror seeds and 32 per element and commits its output, `make balance` rewrites `coverage/balance-report.md`, and the committed table is the changelog: today it reads the pooled mirror at 0.58, the earth ball at 10.5 mean turns against the wind bolt's 5.8, and the fire ball's first mover edge at 0.69. The parity band holds on the suite's eight seeds and not on the wide run: the wind bolt resolves at 0.55 of the earth anchor and the water droplet at 0.61, both outside the third, documented findings and the next tuning targets rather than hidden constants.

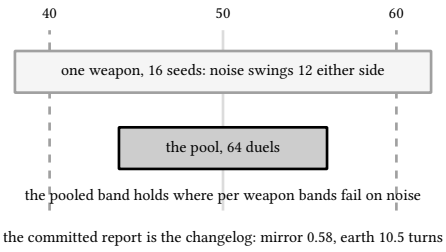



figure 90: the pooled band against the per weapon noise swing

**NOTE: why determinism is the precondition**

A monte carlo harness over `Random.Shared` battles would need thousands of runs per band for the same confidence and would still flake. The same determinism that makes replays byte identical makes one seed worth a hundred, and a failing band always means the design moved, never that the weather did.

sources: the method is the repo's own, the runs above, measured by `make balance` and pinned by `dotnet test books/game-systems/capstone/GameSystemsCapstone.slnx`, 3 tests in `GameSystems.Capstone.Tests` under the balance namespace.

# 23. appendices: topic matrix and sources

The topic matrix is this book's audit trail: every mechanism the chapters build, what it costs, and where the wider world ships it. The build enforces the rest, every chapter listing reads a real file that the pinned toolchain compiles and tests, 156 sample tests and 137 capstone tests, zero skipped, and the capstone's battle engine, balance harness, renderer, and web companion all build from one solution.

## 23.1. topic coverage

127 topics across the 22 teaching and capstone chapters, each with its honest cost and its counterpart in the ecosystem. The rows say "none in bcl" where the base class library genuinely ships nothing, which is most of this book, a game loop, an ecs, cordic trig, a turn queue are all structures you own, and knowing which parts have no library answer is the practical skill the column encodes.

| topic | chapter | cost | ecosystem counterpart |
|---|---|---|---|
| fixed timestep accumulator | ch 1 | sim and render decouple | kni Game IsFixedTimeStep |
| spiral of death clamp | ch 1 | backlog dropped on stall | framework catch up budget |
| interpolation alpha | ch 1 | render state in prev/current pairs | manual lerp |
| variable timestep hazards | ch 1 | replay divergence | none, avoid for sim |
| entity as id handle | ch 2 | storage owned elsewhere | unity entities concept |
| component as plain data | ch 2 | no behavior attached | IComponentData |
| system over a query | ch 2 | coupling by data shape | ISystem/SystemBase |
| generation slot recycling | ch 2 | stale handle rejection | entity versioning |
| archetype vs sparse set storage | ch 2 | churn vs iteration trade | archetype chunks |
| tag components | ch 2 | one bit of routing | zero sized components |
| sparse set store | ch 3 | iteration probes per component | unity uses archetypes instead |
| ref returns into dense arrays | ch 3 | no boxing, aliased state | ref struct accessors |
| component bitmask queries | ch 3 | 64 types per ulong | none, roll your own |
| swap remove | ch 3 | order not preserved | list backfill idiom |
| deferred structural changes | ch 3 | spawn and kill queues | entity command buffers |
| structure of arrays | ch 4 | field at a time access | parallel arrays discipline |
| array of objects cost | ch 4 | pointer per entity, measured 2.1x | none, layout choice |
| struct padding and alignment | ch 4 | 7 bytes lost per wide struct | StructLayout and Pack |
| subset scans pay subset memory | ch 4 | needs soa layout | columnar reading |
| transition table fsm | ch 5 | refusals are silent | state value over flag pile |
| guards routing events | ch 5 | row order matters | first passing row wins |
| terminal states | ch 5 | no exits means stuck | audit the table |
| behavior tree grammar | ch 6 | tree re-walked per tick | none in bcl |
| sequence and selector composites | ch 6 | priority is position | logical and/or |
| running status | ch 6 | spans ticks, no memory here | task statuses |
| utility response curves | ch 6 | curve design is the work | clamp and lerp |
| argmax option picking | ch 6 | scores must be comparable | MaxBy |
| Q31.32 fixed point | ch 7 | finite range, throws on overflow | decimal, slow and 96 bit |
| cordic sin cos | ch 7 | 32 step loop per call | Math.Sin, not portable |

| topic | chapter | cost | ecosystem counterpart |
|---|---|---|---|
| frozen literal tables | ch 7 | hand generated constants | none |
| integer newton sqrt | ch 7 | iterations not cycles | Math.Sqrt, not portable |
| determinism vs exactness | ch 7 | identical bits, not true values | none |
| semi-implicit euler | ch 8 | O(dt) but energy bounded | none, hand rolled |
| wind as acceleration | ch 8 | one extra term | none |
| segment vs heightfield collision | ch 8 | one probe per column crossed | none |
| tunneling prevention | ch 8 | continuous segment test | none |
| crater carving | ch 8 | integer parabola per column | none |
| heightfield terrain | ch 9 | one height per column | arrays of floats in engines |
| value noise from hash | ch 9 | needs avalanche finalizer | Perlin in libraries |
| smoothstep interpolation | ch 9 | none, 3 mults | MathHelper smoothstep |
| fBm octave stacking | ch 9 | octaves × width work | library noise |
| seed as map identity | ch 9 | craters need replay events | none |
| initiative queue | ch 10 | one sort, then removals | none |
| stable tie breaking | ch 10 | relies on stable sort | OrderBy is stable |
| mid round removal | ch 10 | cursor fixups | none |
| round counting | ch 10 | full cycles only | none |
| splitmix64 generator | ch 11 | own the arithmetic | Random, not version stable |
| rejection sampling | ch 11 | occasional redraws | Random.Next, biased by none |
| named rng streams | ch 11 | fork points are contract | none |
| fnv plus finalizer seeding | ch 11 | raw fnv shares high bits | none |
| input log replays | ch 12 | sim must be pure function | none |
| magic and version framing | ch 12 | a few header bytes | BinaryPrimitives |
| round trip byte determinism | ch 12 | no default reflection | System.Text.Json for saves |
| checkpoint snapshots | ch 12 | kbytes per checkpoint | none |
| state hashing | ch 13 | needs raw word state | none |
| avalanche criterion tests | ch 13 | fixed inputs, no flakes | none |
| headless determinism runs | ch 13 | sim purity required | none |
| counting harness | ch 13 | one dict entry per system | none |
| scenario invariants | ch 13 | turn caps and bounds | none |
| unified character sheet | ch 14 | one record to save and hash | none |
| stat growth by level | ch 14 | derived, never stored per level | none |
| xp curve and multi level awards | ch 14 | one award crosses levels | none |
| composed effective stats | ch 14 | gear and pet bonuses merge | none |
| profile binary codec | ch 14 | field naming errors | System.Text.Json would drift |
| weapon as data record | ch 15 | catalog versioned with binary | none |
| carve profiles per weapon type | ch 15 | compositions over one primitive | none |
| linear falloff owned by the weapon | ch 15 | integer raw units only | none |
| recipe crafting with success rolls | ch 15 | consume on failure, roll from stream | none |
| pet stat bonuses | ch 16 | compose onto the sheet | none |
| stream driven hatching | ch 16 | one draw per egg | none |
| once per battle abilities | ch 16 | gate in the engine, pure effects | none |

| topic | chapter | cost | ecosystem counterpart |
|---|---|---|---|
| staged expedition defs | ch 17 | one record per fight | none |
| weighted loot tables | ch 17 | one draw per reward | none |
| unlock chains | ch 17 | strict order, boss gated | none |
| signed elevation angles | ch 19 | one int carries direction | none |
| blast falloff damage | ch 19 | horizontal distance model | none |
| ai ghost flight search | ch 19 | lattice finer than blast | none |
| identity versus balance split | ch 19 | balance is versioned | none |
| match state hash | ch 19 | folds whole world to 64 bits | none |
| tick stepped battle loop | ch 19 | one step per frame | none |
| shells as ecs entities | ch 19 | payload joins for free | none |
| deferred spawn and kill drain | ch 19 | cluster children need it | none |
| weapon catalog in battle | ch 19 | carve and damage per type | none |
| campaign composition root | ch 20 | streams fork once | none |
| stage to battle wiring | ch 20 | stats compose, seeds derive | none |
| replay v3, weapons and movement in the log | ch 20 | magic GSR3, version 3 | none |
| kni game subclass | ch 21 | window owns no logic | Game, Update, Draw |
| spritebatch pixel stretching | ch 21 | no content pipeline | SpriteBatch |
| render model projection | ch 21 | floats on the skin only | none |
| htmx fragment ui | ch 21 | no client js written | minimal api endpoints |
| vendored htmx with sri | ch 21 | digest pinned in test | wwwroot static files |
| replay download endpoint | ch 21 | base64 of the log | none |
| backbuffer frame capture | ch 21 | get data, save as png, exit | none |
| primary attribute derivation | ch 14 | one pure function, every number derived | none |
| exclusive raw stat sources | ch 15 | the catalog never sets an illegal field | none |
| power number composite | ch 14 | one figure stands for the sheet | none |
| fourteen slot loadout | ch 14 | one legality table | none |
| paged grid inventory | ch 15 | one item per cell, deterministic sort | none |
| compounding diamond upgrades | ch 15 | fixed point 1.2 a level, 2^n a price | none |
| element channeling | ch 15 | the offhand fuels the weapon's element | none |
| elevation windows per weapon | ch 19 | fire validates the window | none |
| elemental damage pipeline | ch 19 | falloff, offense, resistances, crit, all integer | none |
| status ticks at turn starts | ch 19 | burn, soak, lock, armor up | none |
| n combatant teams | ch 19 | initiative from agility, teams gate the blast | none |
| aim phase movement | ch 19 | stamina a meter, budget a turn | none |
| paper plane teleport | ch 19 | one flight a match, the walk ends | none |
| second mover compensation | ch 22 | health pins the mirror band | none |

| topic | chapter | cost | ecosystem counterpart |
|---|---|---|---|
| mana gated pet actives | ch 16 | pools regen, passives decide depth | none |
| creep scripts versus utility bosses | ch 17 | one deterministic, one tree | none |
| gold and diamond economy | ch 20 | drops feed shop and forge | none |
| sandbox stage | ch 20 | practice pays nothing | none |
| home hub click zones | ch 21 | mouse routes to campaign calls | Mouse.GetState |
| draggable minimap | ch 21 | title bar drag, click to fly | none |
| aim bot ghost arc | ch 21 | the sweep drawn through live wind | none |
| paper doll sheet screen | ch 21 | fourteen slots, one power number | none |
| monte carlo balance bands | ch 22 | deterministic seeds, no flake | none |
| committed balance report | ch 22 | the binary writes its own table | none |
| embedded lua runtime | ch 18 | policy hot, state stays compiled | none, NLua is a package |
| script host lifetime | ch 18 | one interpreter per campaign, disposed with it | IDisposable |
| sandbox by nil assignment | ch 18 | what was never granted cannot be reached | none |
| host-fed randomness | ch 18 | math.random removed, rolls arrive as arguments | none |
| hook as pure function | ch 18 | no coroutines, no script state | none |
| integer marshaling both ways | ch 18 | raw state pushes, long arrays trap as userdata | KeraLua native bindings |
| degrade to compiled policy | ch 18 | a broken script costs behavior nothing | none |
| warcraft iii lua maps | ch 18 | either-or per map, community-documented sandbox | none, world editor |
| dota 2 vscript addons | ch 18 | event listeners, ui in panorama js | none, workshop tools |
| on off state hash equivalence | ch 18 | walks assert traces, never wins | none |

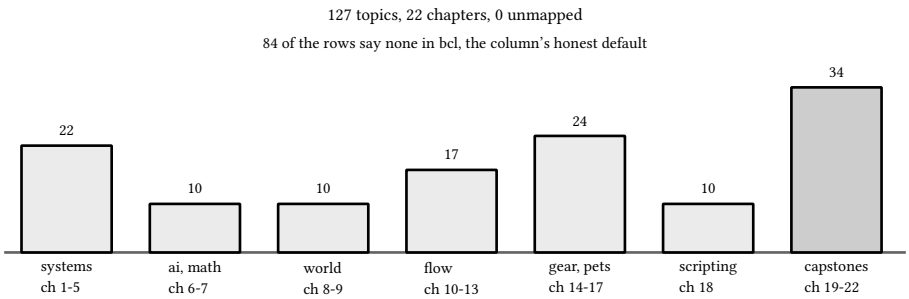



figure 91: where the 127 topics landed, none unmapped

## 23.2. pinned sources

Every chapter ends with its own sources line naming the pages consulted while writing it. The book level pins are:

| topic | url | accessed |
|---|---|---|
| ddtank customizable characters and weapon upgrading | https://ddtank-us.fandom.com/wiki/Games | 2026-09-09 |
| ddtank weapons, normal and special, drops and events | https://ddtank-us.fandom.com/wiki/Weapons | 2026-09-09 |
| ddtank pet raising system | https://baike.baidu.com/en/item/DDtank/3467170 | 2026-09-09 |
| ddtank adventure mode overview | https://www.gamesindustry.biz/ddtank-a-bit-more-about-it | 2026-09-09 |
| kni engine, release 4.3.9001 | https://github.com/kniEngine/kni | 2026-09-08 |
| kni nuget packages and templates | https://www.nuget.org/packages?q=nkast.Kni | 2026-09-08 |
| monogame game loop, IsFixedTimeStep and TargetElapsedTime | https://docs.monogame.net/articles/getting_to_know/whatis/game_loop/index.html | 2026-09-08 |
| fix your timestep, accumulator and alpha | https://gafferongames.com/post/fix_your_timestep/ | 2026-09-08 |
| integration, semi-implicit euler | https://gafferongames.com/post/integration_basics/ | 2026-09-08 |
| ecs faq, entity component system definitions | https://github.com/SanderMertens/ecs-faq | 2026-09-08 |
| unity entities, archetypes and chunks | https://docs.unity3d.com/Packages/com.unity.entities@1.0/manual/concepts-archetypes.html | 2026-09-08 |
| state, boolean pile against state value | https://gameprogrammingpatterns.com/state.html | 2026-09-08 |
| behavior trees for ai, how they work | https://www.gamedeveloper.com/programming/behavior-trees-for-ai-how-they-work | 2026-09-08 |
| improving ai decision modeling through utility theory | https://www.gdcvault.com/play/1012410/Improving-AI-Decision-Modeling-Through | 2026-09-08 |
| System.Math.Sin remarks, c runtime portability | https://learn.microsoft.com/en-us/dotnet/api/system.math.sin?view=net-10.0 | 2026-09-08 |
| System.Random remarks, cross version instability | https://learn.microsoft.com/en-us/dotnet/api/system.random?view=net-10.0 | 2026-09-08 |
| cordic algorithm and constants | https://en.wikipedia.org/wiki/CORDIC | 2026-09-08 |
| splitmix, fast splittable pseudorandom number generators | https://dl.acm.org/doi/10.1145/2714064.2660235 | 2026-09-08 |
| value noise and fbm treatments | https://iquilezles.org/articles/morenoise/ | 2026-09-08 |
| BinaryPrimitives span contract | https://learn.microsoft.com/en-us/dotnet/api/system.buffers.binary.binaryprimitives?view=net-10.0 | 2026-09-08 |
| htmx 4.0.0, npm next tag and dist | https://www.npmjs.com/package/htmx.org | 2026-09-08 |
| htmx attributes and philosophy | https://htmx.org/docs/ | 2026-09-08 |
| local probes on .NET 10 windows/amd64: restore and compile probes of nkast.Kni.Platform.WinForms.DX11 4.3.9001, the aos versus soa benchmark, and the htmx sri digest | | 2026-09-08 |
| warcraft iii patch 1.31 notes mirror, lua as a supported scripting language | https://web.archive.org/web/20251216024804/https://warcraft.wiki.gg/wiki/Warcraft_III/Patch_1.31.0 | 2026-09-23 |
| hive workshop comprehensive lua mapping guide, embedded 5.3 and the sandbox library list | https://www.hiveworkshop.com/threads/a-comprehensive-guide-to-mapping-in-lua.341880 | 2026-09-23 |
| hive workshop 1.31 ptr thread, modified 5.3.4 and natives in common.j | https://www.hiveworkshop.com/threads/warcraft-iii-patch-1-31-ptr.314856/ | 2026-09-23 |

| topic | url | accessed |
|---|---|---|
| hive workshop wc3 reforged lua thread, map options and gc desync | https://www.hiveworkshop.com/threads/wc3-reforged-lua.326354/ | 2026-09-23 |
| hive workshop getting started with lua scripting in wc3 maps | https://www.hiveworkshop.com/threads/getting-started-with-lua-scripting-in-wc3-maps.315970/ | 2026-09-23 |
| jassdoc, GetLocalPlayer local code and desync rules | https://github.com/lep/jassdoc | 2026-09-23 |
| warcraft iii reforged release date announcement | https://news.blizzard.com/en-us/article/23237899/warcraft-iii-reforged-releases-on-january-28-2020 | 2026-09-23 |
| dota 2 workshop tools scripting, vscript runs lua | https://developer.valvesoftware.com/wiki/Dota_2_Workshop_Tools/Scripting | 2026-09-23 |
| dota 2 listening to game events, ListenToGameEvent and Dynamic_Wrap | https://web.archive.org/web/2024id_/https://developer.valvesoftware.com/wiki/Dota_2_Workshop_Tools/Scripting/Listening_To_Game_Events | 2026-09-23 |
| dota 2 scripting api, LinkLuaModifier | https://web.archive.org/web/2024id_/https://developer.valvesoftware.com/wiki/Dota_2_Workshop_Tools/Scripting/API | 2026-09-23 |
| dota 2 panorama ui, html5 css js under content/dota_addons | https://web.archive.org/web/20240105013815id_/https://developer.valvesoftware.com/wiki/Dota_2_Workshop_Tools/Panorama/en | 2026-09-23 |
| dota 2 workshop tools, addons framing | https://web.archive.org/web/2024/https://developer.valvesoftware.com/wiki/Dota_2_Workshop_Tools | 2026-09-23 |
| dota 2 reborn, source 2 announcement | https://web.archive.org/web/20151231150727/http://www.dota2.com/reborn/part3/ | 2026-09-23 |
| moddota typescript addon template, tstl luaTarget evidence | https://github.com/ModDota/TypeScriptAddonTemplate | 2026-09-23 |
| nlua 1.7.9 on nuget | https://www.nuget.org/packages/NLua | 2026-09-23 |
| nlua readme, state DoString indexer LuaFunction.Call | https://raw.githubusercontent.com/nlua/NLua/main/README.md | 2026-09-23 |
| keralua on nuget, 1.4.9 is what nlua 1.7.9 resolves, embeds lua 5.4 | https://www.nuget.org/packages/KeraLua | 2026-09-23 |
| keralua 1.4.10 package download, native runtimes layout inspected | https://www.nuget.org/api/v2/package/KeraLua/1.4.10 | 2026-09-23 |
| lua 5.4 manual, integer and float subtypes | https://www.lua.org/manual/5.4/manual.html | 2026-09-23 |
| fontstashsharp kni on nuget, runtime font rasterizer 1.6.1 | https://www.nuget.org/packages/FontStashSharp.KNI | 2026-09-23 |
| fontstashsharp api, FontSystem AddFont GetFont and the spritebatch overloads | https://fontstashsharp.github.io | 2026-09-23 |
| nerd fonts releases, source of the vendored iosevka term ttf | https://github.com/ryanoasis/nerd-fonts | 2026-09-23 |

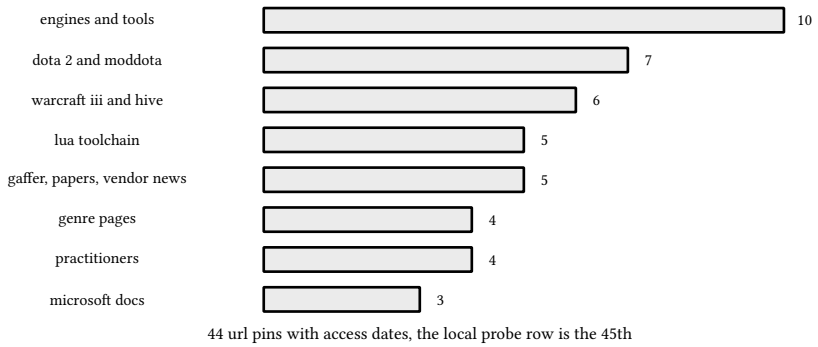



figure 92: the 44 url pins by origin, the table's one local probe row sits outside them

**NOTE: version drift rule**

These pins were verified 2026-09-08 against kni 4.3.9001 and htmx 4.0.0 on windows/amd64 under .NET 10, including a restore and compile probe of the kni package before the renderer was written and a byte level SRI check of the vendored htmx. The 2.0 additions were verified 2026-09-23 the same way: NLua 1.7.9 over KeraLua 1.4.9 carrying lua54.dll, and FontStashSharp.Kni 1.6.1 rasterizing the repo's vendored iosevka term ttf, each restored, compiled, and exercised by its own test lane. The measured claims, the 2.1x soa ratio, the cordic accuracy, the replay size gap, the on-off state hash equivalence, are all reproducible from the test suite, and this table is the changelog if a future release moves any of them.